



TRAILHEAD

Advanced Pricing

Exercise Guide

Version 1.0

TABLE OF CONTENTS

Preface	5
Overview	5
What You Will Learn	5
Prerequisites	5
Advanced Pricing Overview	6
Exercise 9-1: Create Attribute-Based Pricing	8
Attribute-Based Pricing	9
Attribute-Based Pricing Expression Sets and Lookup Tables	11
Attribute-Based Pricing and Pricing Plans	13
Task 1: Review Salesforce Industries Process Library	14
Task 2: Configure post-install steps	15
Task 3: Validate Apex classes	18
Task 4: Set up AttributePricingMatrix	21
Task 5: Set up RangeAttributePricingMatrix	25
Task 6: Set up SourceTargetAttributePricingMatrix	28
Task 7: Set up AttributePricingProcedure Expression Set	31
Task 8: Set up RangeAttributePricingProcedure Expression Set	38
Task 9: Set up SourceTargetAttributePricingProcedure Expression Set	41
Task 10: Set up custom pricing steps	42
Task 11: Refresh the platform cache	49
Task 12: Add products to the Cart to initiate attribute-based pricing	50
Exercise 9-2: Create Loyalty Pricing	55
Loyalty Pricing	56
Creating Pricing Elements for Loyalty Pricing	56
Task 1: Enable loyalty pricing	58
Task 2: Create pricing elements for loyalty point charges	59
Task 3: Assign loyalty pricing to products	61
Task 4: Refresh the cache	63
Task 5: Create orders using loyalty pricing	64
Exercise 9-3: Work with Repricing	67

Repricing	68
Task 1: Activate the RepricingElementServiceImplementation	69
Task 2: Reprice order line items to reflect a price effectivity change	70
Task 3: Reprice an asset to reflect a context rule condition for an account	77
Exercise 9-4: Create a Pricing Hook	83
What is an Event Hook?	84
How can I use an event hook in Industries CPQ?	84
How is the CpqAppHandlerHook invoked?	85
What do I need to look out for when using a hook?	86
Examples of using a hook	87
Task 1: Review the billing zip code custom field	88
Task 2: Create an attribute pricing matrix for billing zip code	89
Task 3: Create an attribute pricing procedure expression set for billing zip code	92
Task 4: Create a custom pricing plan step for billing zip code	95
Task 5: Create a new Apex class	97
Task 6: Create the CpqAppHandlerHookImplementation	99
Task 7: Add a new column to the Cart	101
Task 8: Add billing zip code as a custom field in the Cart	108
Task 9: Test in the Cart	110
Troubleshooting Guide	112
Modifying the Cart	113
Exercise 9-5: Create Usage Pricing	114
What is Usage Pricing?	115
Task 1: Enable usage pricing	118
Task 2: Create charge measurements and assign usage pricing to products	119
What is an attribute binding?	124
What is a Vlocity Object?	126
Task 3: Configure an attribute and an attribute binding for estimated usage quantity	127
Task 4: Test usage pricing in the Cart	133
Task 5: Configure attribute-based pricing for usage pricing	135
Task 6: Edit an order to use attribute-based pricing	143
Exercise 9-6: Extending Attribute-Based Pricing Challenge	145
Task 1: Review product attributes	146

Task 2: Set up attribute-based pricing matrix	148
Task 3: Test attribute-based pricing in the Cart	149

Preface

These training exercises are based on the Spring '22 release of Salesforce Industries Communications, Media, and Energy & Utilities Clouds. For additional information about the topics covered in this module, see the documentation available in Vlocity Success Community at <https://success.vlocity.com>.

Overview

This module includes concepts and exercises regarding:

- Attribute-based pricing
- Loyalty pricing
- Repricing orders and assets
- Event hooks
- Usage pricing

What You Will Learn

When you complete this training, you will be able to:

- Price products using product attributes.
- Price products using loyalty points.
- Create an event hook and explain when it is invoked.
- Explain how repricing works using multiple price list entries.
- Price energy commodities (products) based on estimated usage

Prerequisites

The prerequisites for this training include successful completion of the Pricing and EPC modules from the Build Salesforce Industries Configure, Price, Quote (CPQ) Solutions training course. In addition, a solid understanding of basic Salesforce concepts and functionality is required. You should also have a working knowledge of telecommunications, media, or energy and utilities industry business objectives.

Advanced Pricing Overview

In the Pricing module, we covered how Salesforce Industries' pricing solution offers:

- A component-oriented system with reusable items.
- Types of pricing, such as penalties, charges, and adjustments to existing charges.
- Settings that determine the frequency that the charge occurs.
- Pricing components which live independently of products.
- Transitioning from older pricing to newer pricing for less expense and disruption.

Now, let's dig a little deeper. Salesforce Industries CPQ provides standard and advanced pricing capabilities for you to:

- Offer customers the option of paying with loyalty points.
- Change the price of a product automatically over time.
- Customize the pricing process by adding pricing plan steps or invoking an event hook.
- Automatically reprice products, draft items in the Cart, and assets.
- Price energy commodities (products) based on estimated usage

As we progress through this module you will be exposed to advanced functionality. To understand these features, we need to start with the underlying structure. The core of Salesforce Industries open architecture is the interface and implementation paradigm. An interface manages the application and its call out to business logic; an implementation is that business logic. Salesforce Industries offers multiple implementations for each interface. You can choose the implementation that best fits your needs, or you can create your own implementation. When used judiciously, you can define interfaces once and deploy them many times to consistently maintain and apply business logic.

Salesforce Industries provides a variety of interfaces and implementations that support functionality across Salesforce Industries clouds and change the default behavior for asset-based ordering, contracts, context rules, advanced rules, and pricing. For example, the PricingInterface communicates with the system to ask, “How should I price this product?” The implementation responds with the appropriate business logic to evaluate the configuration, price lists, and rules and determine the correct price for the product.

Exercise 9-1: Create Attribute-Based Pricing

Scenario

Eliza Young, Infiwave's Developer, is quickly learning that with all the products Infiwave offers and the possible combinations of those products that the product catalog is going to grow rapidly, increasing the time needed on maintenance and administration. She thinks to herself, there must be a more efficient way to price the hundreds of products in the catalog. In her search for a solution, she comes across Attribute-Based Pricing. Implementing Attribute-Based Pricing allows Eliza to create a single product, a set of attributes and use a matrix to price each combination of attributes. Hooray! Eliza is ecstatic that less time will be spent on maintaining and administering the product catalog.

Goal

- Explain the concept of attribute-based pricing
- List the types of attribute-based pricing
- Set up attribute-based pricing using pricing plans
- Add data to lookup tables
- Create expression sets to price the line item in the Cart
- Add a new step to the default pricing plan

Tasks

1. Review Salesforce Industries Process Library
2. Configure post-install steps
3. Validate Apex classes
4. Set up AttributePricingMatrix
5. Set up RangeAttributePricingMatrix
6. Set up SourceTargetAttributePricingMatrix
7. Set up AttributePricingProcedure Expression Set
8. Set up RangeAttributePricingProcedure Expression Set
9. Set up SourceTargetAttributePricingProcedure Expression Set
10. Set up custom pricing steps
11. Refresh the platform cache
12. Add products to the Cart to initiate attribute-based pricing

Time: 25 mins



ALERT:

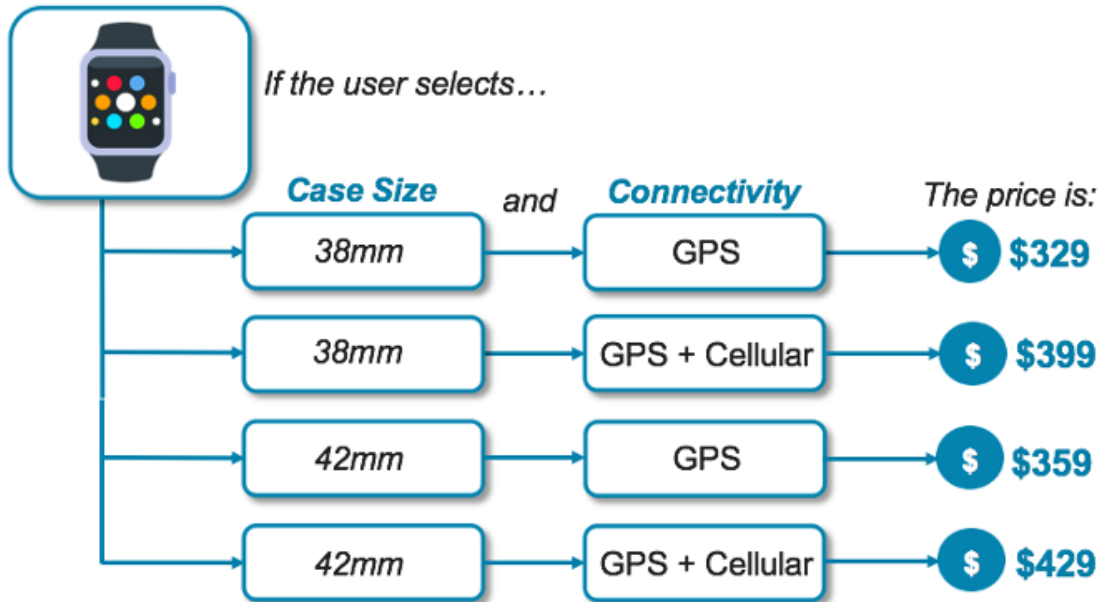
If you've just received your training playground, add your email address to the system administrator profile to ensure you receive all system notifications. In the upper-right, click on the **Avatar** and select **Settings**. Enter your email address in the **Email** field on the Personal Information page and click **Save**.

Attribute-Based Pricing

Assigning attributes to products provides a powerful and flexible way to describe product characteristics. Consider a smartwatch with two product attributes – case size and connectivity. Without attribute-based pricing you would need to create four products for each configuration so that each is priced correctly.

	Product 1	Product 2	Product 3	Product 4
				
Case Size	38mm	38mm	42mm	42mm
Connectivity	GPS	GPS + Cellular	GPS	GPS + Cellular
	\$ \$329	\$ \$399	\$ \$359	\$ \$429

But, with attribute-based pricing you create a single product, a set of attributes and use a matrix to price each combination of attributes.



Salesforce Industries provides three types of attribute-based pricing:

- Standard – prices a product using attributes.
- Source/Target – prices a product based on the attributes of another product.
- Range – prices a product when the value falls within the defined range for each attribute and field in the matrix.

You can implement attribute-based pricing using pricing plans, expression sets, and lookup tables.

Attribute-Based Pricing Expression Sets and Lookup Tables

In this exercise, you'll use preconfigured lookup tables and create expression sets to execute attribute-based pricing. There are two forms of lookup tables: a decision matrix or a decision table. Decision matrices are exact-match based lookup tables that match your inputs to values in a row, and return the row's output. Decision tables are complex lookup tables that match your inputs to values in multiple rows, if applicable, and return multiple matching outputs. An expression set is a procedure that performs a series of calculations on matrix lookups and user-defined variables and constants. Together, expression sets and lookup tables comprise the Business Rules Engine.



NOTE:

Expression sets and lookup tables are similar in function to calculation procedures and calculation matrices, and are part of Salesforce core in Summer '22 and later releases. Calculation procedures and matrices are still supported and included in Industries packages, though we recommend using expression sets and lookup tables for advanced pricing scenarios.

Salesforce provides a DataPack that can be downloaded from the Salesforce Industries Process Library. This DataPack includes preconfigured matrices to support attribute-based pricing. The Attribute/Volume Based Pricing DataPack has already been installed in your training playground, and it imported the following records.

Decision Matrix (Lookup Table)	DataPack
AttributePricingMatrix	AttributePricingDataPack.json
SourceTargetAttributePricingMatrix	SourceTargetAttributePricingDataPack.json
RangeAttributePricingMatrix	RangeAttributePricingDataPack.json

The preconfigured decision matrices contain the following columns:

Header Name	Header Type	Data Type	Description
SourceProductName	Input	Text	Product Name
SourceProductCode	Input	Text	Product Code is the key for the matching algorithm used in the PricingPlanHelper.cls.
CharacteristicName	Input	Text	Attribute name associated with the product. Specify multiple attributes using a semi-colon delimited list.
CharacteristicValue	Input	Text	Attribute value associated with the product attribute. Specify multiple attributes using a semi-colon delimited list.
MRC	Output	Currency	Recurring monthly charge. Does not need to match an existing pricing element value in the price list that is used for the order.
NRC	Output	Currency	Non-recurring charge or one-time charge. Does not need to match an existing pricing element value in the price list that is used for the order.
Target Product Code	Output	Text	Used only for Source/Target or Range pricing. Specifies the target output name to which the pricing override is applied.



ALERT:

Do not change the header names of the columns in the decision matrices. The PricingPlanHelper.cls expects these column names and will fail if they are not found.

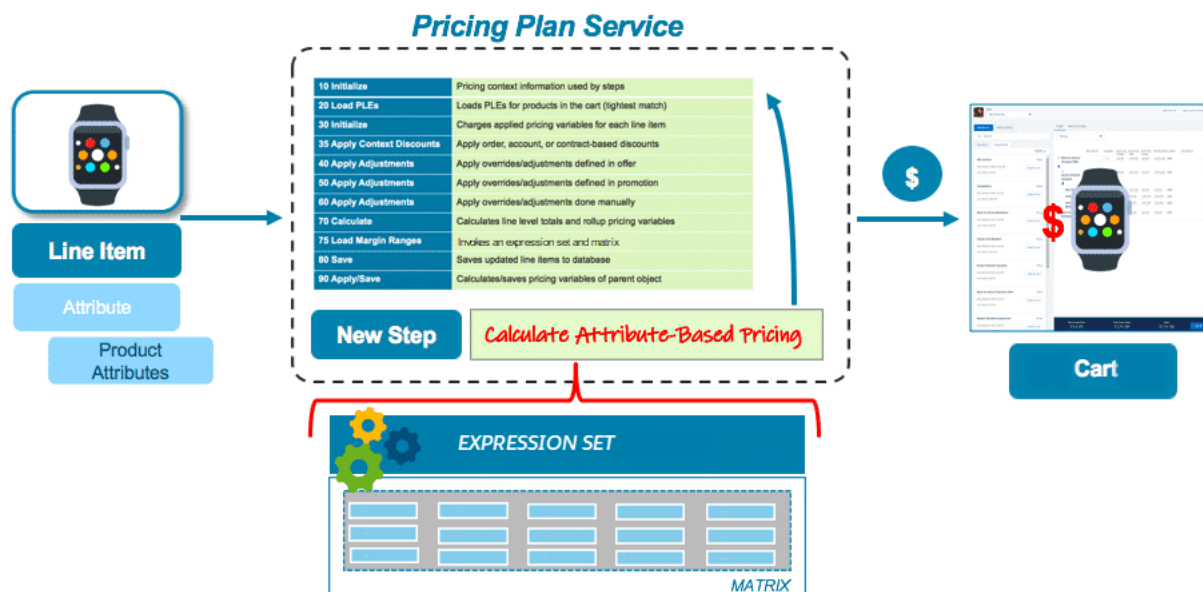
The Range Attribute Pricing matrix also includes a Text column type to store the Quantity API field name.

Attribute-Based Pricing and Pricing Plans

You can invoke the expression sets and lookup tables from a pricing plan. A pricing plan is a series of steps in sequence, and each step invokes a process, function, or procedure. Pricing plans are defined in the Vlocity Product Console. You can use pricing plans to perform a wide-variety of calculations such as promotion discounts, taxes, and more—including attribute-based pricing.

When implementing attribute-based pricing using pricing plans, you need to create a new step in the default pricing plan. This step will call an expression set and lookup table that stores the product attributes and pricing information. At runtime, the PricingPlanService implementation passes the cart line items to the calculation service and retrieves the correct price.

Attribute-Based Pricing: Pricing Plan Service + Expression Set



Pricing plans are implemented by the PricingInterface and require the PricingPlanService implementation.



RESOURCE:

To optimize performance for large lookup tables or pricing plans that have many attribute-based pricing steps refer to the topic, Caching Calculation Matrices for Performance, in documentation on [Vlocity Success Community](#).

Task 1: Review Salesforce Industries Process Library

1. Navigate to the [Salesforce Industries Process Library](#) and select **Communications**.
2. In the Search Communications field, enter `attribute` and select **Attribute/Volume Based Pricing**.
3. Notice under the **Process Package Includes** section the files that are included in the DataPack.

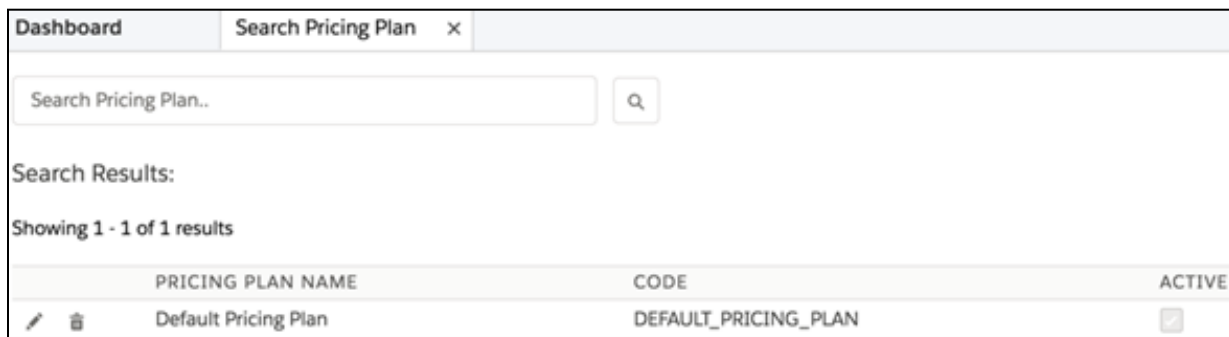


NOTE:

These files are required to be downloaded and installed before implementing attribute-based pricing. We've already installed this DataPack in your training playground.

Task 2: Configure post-install steps

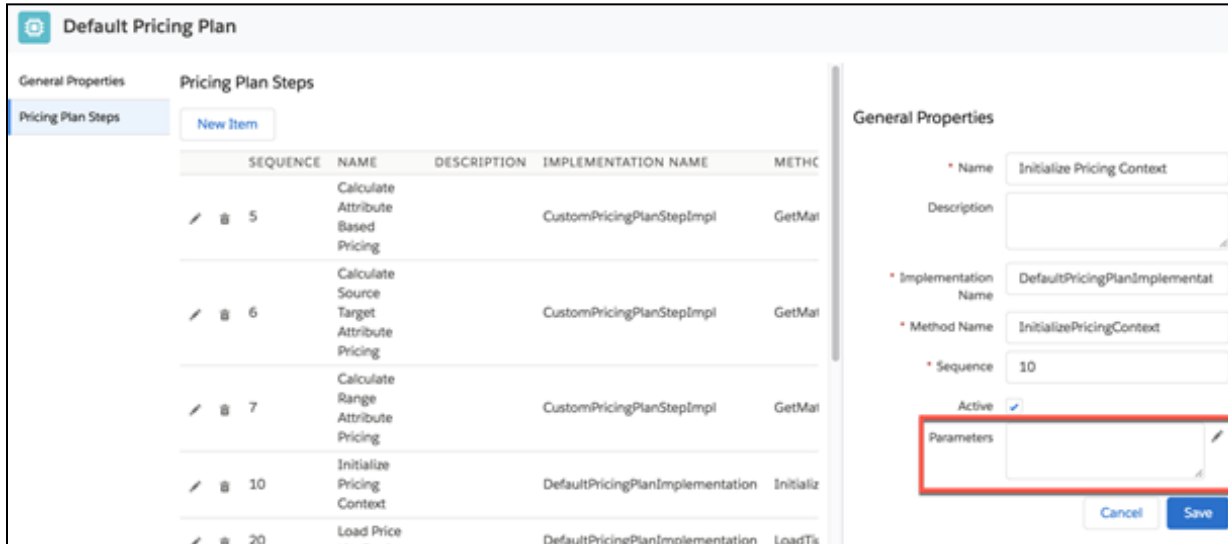
1. Verify the **Default Pricing Plan** is installed.
 - a. Using the Lightning App Launcher, click the **Configure Price Quote (CPQ)** app.
 - b. Navigate to the **Vlocity Product Console**.
 - c. Next to **Pricing Plan** in the Pricing area, click the search icon .
 - d. In the **Search Pricing Plan...** dialog box, press the return or enter key to display the Default Pricing Plan.



The screenshot shows a 'Search Pricing Plan' dialog box. At the top, there is a search input field with the placeholder text 'Search Pricing Plan..' and a search icon. Below the input field, it says 'Search Results:' and 'Showing 1 - 1 of 1 results'. A table displays the search results with three columns: 'PRICING PLAN NAME', 'CODE', and 'ACTIVE'. The table contains one row for 'Default Pricing Plan' with the code 'DEFAULT_PRICING_PLAN' and an active status indicated by a checked checkbox. To the left of the table, there are icons for edit and delete.

PRICING PLAN NAME	CODE	ACTIVE
Default Pricing Plan	DEFAULT_PRICING_PLAN	☒

2. Verify the **Parameters Field** displays.
 - a. Click on the **Default Pricing Plan**.
 - b. Click on the **Pricing Plan Steps** facet.
 - c. Click on any step.
 - d. In the **General Properties** pane, verify the **Parameters** field displays.



Default Pricing Plan

General Properties

Pricing Plan Steps

New Item

SEQUENCE	NAME	DESCRIPTION	IMPLEMENTATION NAME	METHC
5	Calculate Attribute Based Pricing		CustomPricingPlanStepImpl	GetMal
6	Calculate Source Target Attribute Pricing		CustomPricingPlanStepImpl	GetMal
7	Calculate Range Attribute Pricing		CustomPricingPlanStepImpl	GetMal
10	Initialize Pricing Context		DefaultPricingPlanImplementation	Initializ
20	Load Price		DefaultPricingPlanImplementation	LoadTi

General Properties

Name: Initialize Pricing Context

Description:

Implementation Name: DefaultPricingPlanImplementat

Method Name: InitializePricingContext

Sequence: 10

Active: ☒

Parameters:

Cancel Save

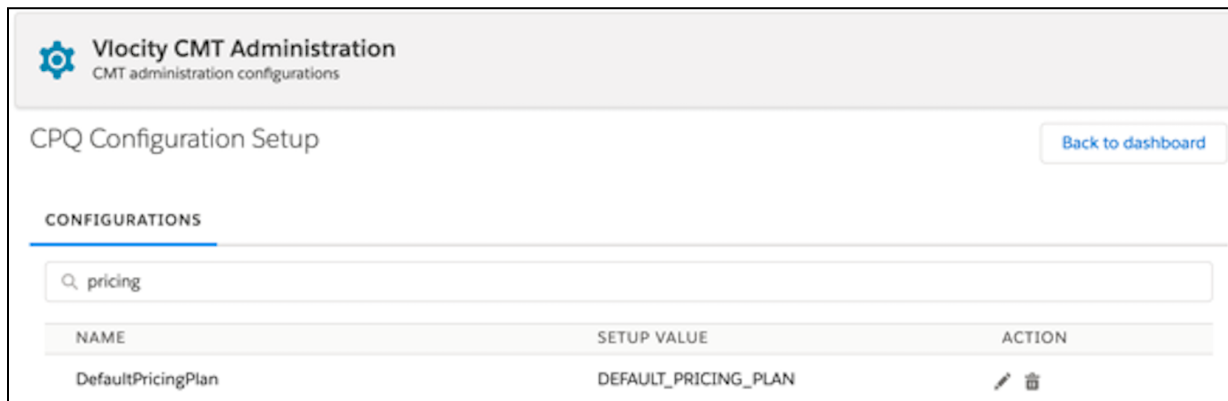
3. Verify the **DefaultPricingPlan** custom setting has been created.



NOTE:

This custom setting is used to indicate the default pricing plan used in the org.

- Using the Lightning App Launcher , click **CME Admin**.
- Under **Custom Settings**, click **CPQ Configuration Setup**.
- In the Search dialog box under **CONFIGURATIONS**, enter `pricing`.
- The **DefaultPricingPlan** custom setting displays. Notice that the **SETUP VALUE** is set to the pricing plan's code.



Vlocity CMT Administration
CMT administration configurations

CPQ Configuration Setup

[Back to dashboard](#)

CONFIGURATIONS

Q pricing

NAME	SETUP VALUE	ACTION
DefaultPricingPlan	DEFAULT_PRICING_PLAN	 

4. Review the **PricingPlanService** implementation.



NOTE:

The PricingPlanService implementation is required when using pricing plans. It is the same implementation as the PricingElementServiceImplementation but the PricingPlanService allows you to manage all of its pricing steps using a pricing plan defined in Vlocity Product Console.

- Click the **Interface Implementations** tab.
- Change the List Views to **All**.
- Click **PricingInterface**.
- On the **Related** tab, verify the **Active** and **Default** boxes are checked for the **PricingPlanService**.

Interface Implementation PricingInterface		
Related Details		
Interface Implementation Detail (4)		
Available Implementation	Active	Default
DefaultPricingImplementa...	☐	☐
PricingRulesImplementation	☐	☐
PricingElementServiceImp...	☐	☐
PricingPlanService	☒	☒

Task 3: Validate Apex classes

1. Verify the **PricingPlanHelper.cls** Apex class from the DataPack is installed.



NOTE:

This Apex class is invoked by CustomPricingPlanStepImpl.cls which analyzes the matrix and invokes the calculation procedure or expression set.

Due to code dependencies, when setting up these Apex classes they must be created in the order demonstrated.

- a. Click on the **Setup** icon  and select **Setup**.
- b. In the **Quick Find** search box, enter and select **Apex Classes**.
- c. On the Apex Classes page, verify the **View** is set to **Pricing Plan Classes**.
- d. Verify **PricingPlanHelper** displays in the **Name** column.


SETUP
Apex Classes

Apex Classes

Apex Code is an object oriented programming language that allows developers to develop on-demand business applications on the Lightning Platform.


Percent of Apex Used: 4.73%
 You are currently using 283,647 characters of Apex Code (excluding comments and @isTest annotated classes) in your organization, out of an allowed limit of 6,000,000 characters. Note that the amount in use includes both Apex Classes and Triggers defined in your organization.

Estimate your organization's code coverage 

Compile all classes 

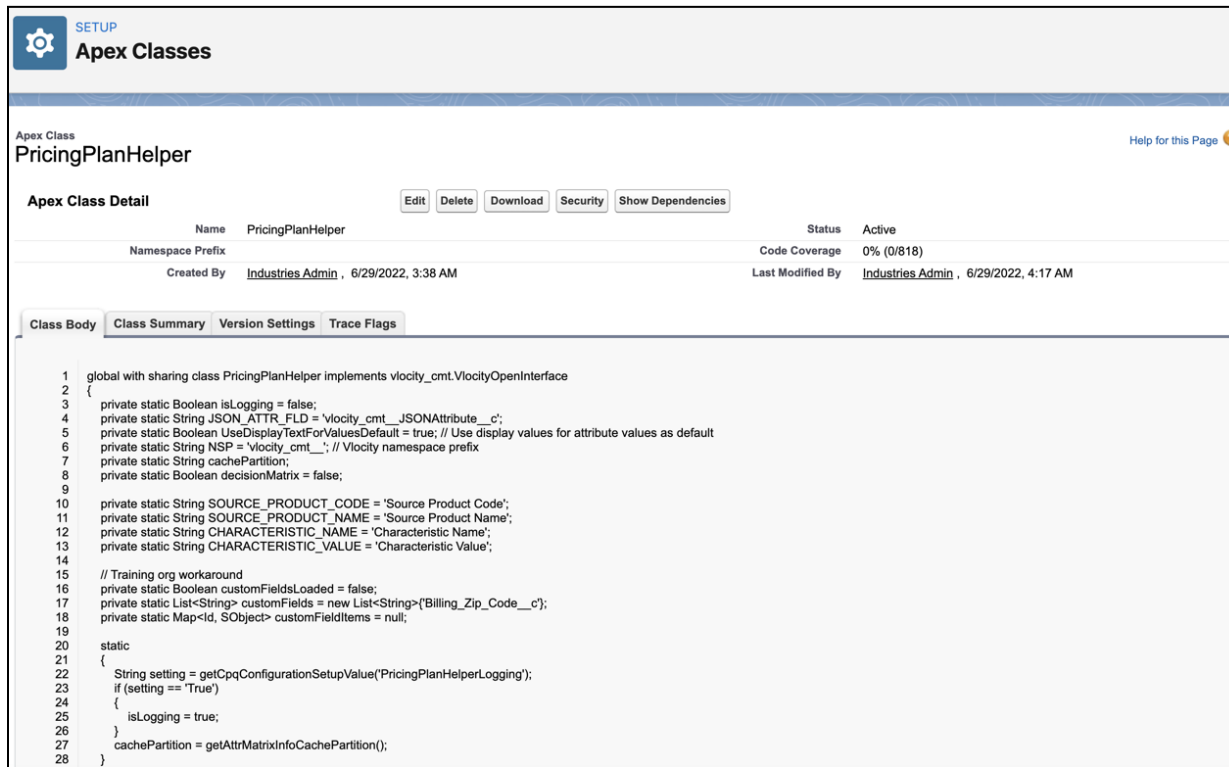
View: Pricing Plan Classes Edit Create New View

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All

Action	Name	Namespace Prefix	Api Version	Status	Size Without Comments	Last Modified By	Has Trace Flags
Edit Security	B2CCmxCopyBaseChargePricingPlanStepImpl	viocity_cmt	49.0	Active	3,987	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Security	CachedBasketPricingPlanImpl	viocity_cmt	52.0	Active	33,913	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	CachedBasketPricingPlanImplTest	viocity_cmt	52.0	Active	61	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Del Security	CustomPricingPlanStepImpl		43.0	Active	2,131	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Del	CustomPricingPlanStepImplTest		43.0	Active	50,208	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Security	DefaultPricingPlanImplementation	viocity_cmt	52.0	Active	149,913	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Security	EPCPricingPlanStepHandler	viocity_cmt	52.0	Active	1,420	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	EPCPricingPlanStepHandlerTest	viocity_cmt	52.0	Active	6,048	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Del Security	PricingPlanHelper		43.0	Active	68,654	Industries Admin, 6/29/2022, 4:17 AM	☐
Edit Security	PricingPlanService	viocity_cmt	52.0	Active	17,133	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	PricingPlanServiceTest	viocity_cmt	52.0	Active	40,970	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	PricingPlanServiceTestData	viocity_cmt	52.0	Active	65,873	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Security	PricingPlanStepTriggerHandler	viocity_cmt	52.0	Active	1,405	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	PricingPlanStepTriggerHandlerTest	viocity_cmt	52.0	Active	1,450	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Security	PricingPlanTriggerHandler	viocity_cmt	52.0	Active	3,322	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	PricingPlanTriggerHandlerTest	viocity_cmt	52.0	Active	3,041	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit Security	QBPDDiscountPricingPlanStepImpl	viocity_cmt	48.0	Active	71,037	Industries Admin, 6/29/2022, 3:38 AM	☐
Edit	QBPDDiscountPricingPlanStepImplTest	viocity_cmt	50.0	Active	13,269	Industries Admin, 6/29/2022, 3:38 AM	☐

Show me fewer records per list page

- e. Right-click to open the **PricingPlanHelper** in a new tab to review the Apex code.



The screenshot shows the Salesforce Apex Classes interface. At the top, there's a 'SETUP' button and the title 'Apex Classes'. Below this, the 'Apex Class' section displays 'PricingPlanHelper'. To the right of the class name are buttons for 'Edit', 'Delete', 'Download', 'Security', and 'Show Dependencies'. Below these buttons is a table with details about the class:

Name	PricingPlanHelper	Status	Active
Namespace Prefix		Code Coverage	0% (0/818)
Created By	Industries Admin , 6/29/2022, 3:38 AM	Last Modified By	Industries Admin , 6/29/2022, 4:17 AM

Below the table, there are tabs for 'Class Body', 'Class Summary', 'Version Settings', and 'Trace Flags'. The 'Class Body' tab is selected, showing the Apex code for the class. The code starts with a global declaration and implements the `VlocityOpenInterface`. It includes several static variables and a static method `getAttrMatrixInfoCachePartition()` at the bottom.

- f. Review lines 100-103. What is the name of the method that retrieves the price from the expression set (calculation procedure)?

- g. Review lines 375-422 and lines 441-492. What is the code doing?

- h. Review lines 1329-1360, notice the code is invoking an expression set (calculation procedure). We will explore this further.

- i. Review lines 1015-1034, what is the name of the method that sets the price to return to the pricing plan service?
 - j. Close the browser tab with the PricingPlanHelper Apex code.
2. Verify the **CustomPricingPlanStepImpl.cls** and **CustomPricingPlanStepImplTest.cls** Apex classes from the DataPack are installed.



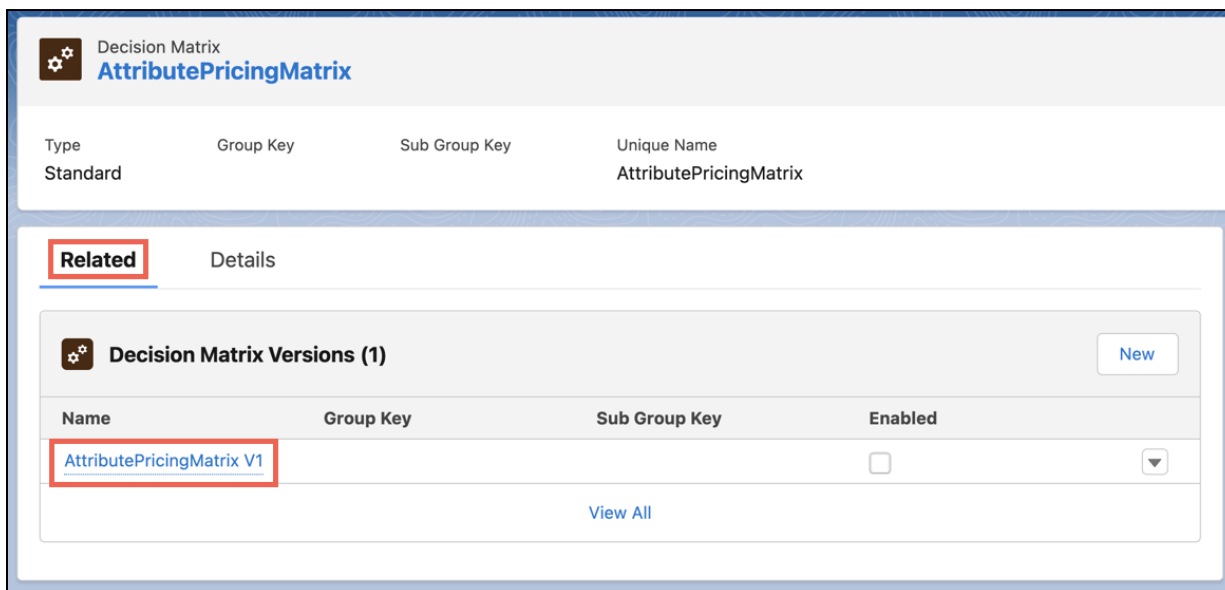
NOTE:

The CustomPricingPlanStepImpl.cls Apex class exposes the GetMatrix method and invokes the PricingPlanHelper class. The CustomPricingPlanStepImplTest.cls is an Apex test class.

- a. Verify that the **CustomPricingPlanStepImpl** and **CustomPricingPlanStepImplTest** display in the **Name** column.
- b. Right-click to open the **CustomPricingPlanStepImpl** in a new tab to review the Apex code.
- c. Review line 8, notice the method name. We will explore this further.
- d. Close the browser tabs with the Apex codes.

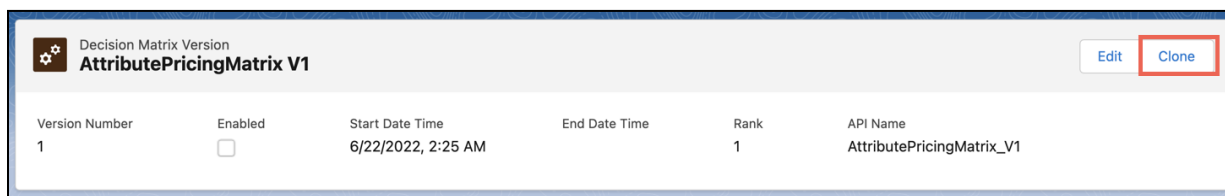
Task 4: Set up AttributePricingMatrix

1. Using the Lightning App Launcher, search for and select **Lookup Tables**.
2. Change the List Views to **All Decision Matrices**.
3. Click **AttributePricingMatrix**, which was installed from the DataPack.
4. Select the **Related** tab and click **AttributePricingMatrix V1**.



The screenshot shows the 'AttributePricingMatrix' page. At the top, there's a header with a gear icon and the text 'Decision Matrix AttributePricingMatrix'. Below this is a table with columns: Type, Group Key, Sub Group Key, and Unique Name. The 'Type' column has the value 'Standard' and the 'Unique Name' column has the value 'AttributePricingMatrix'. Below the table, there are two tabs: 'Related' (which is selected and highlighted with a red box) and 'Details'. Under the 'Related' tab, there's a section titled 'Decision Matrix Versions (1)' with a 'New' button. Below this is a table with columns: Name, Group Key, Sub Group Key, and Enabled. The 'Name' column has the value 'AttributePricingMatrix V1' (highlighted with a red box). The 'Enabled' column has a checkbox that is currently unchecked. Below the table, there's a 'View All' link.

5. Click **Clone** in the **AttributePricingMatrix V1** header.



The screenshot shows the 'AttributePricingMatrix V1' header. At the top, there's a header with a gear icon and the text 'Decision Matrix Version AttributePricingMatrix V1'. To the right of the header are two buttons: 'Edit' and 'Clone' (highlighted with a red box). Below the header is a table with columns: Version Number, Enabled, Start Date Time, End Date Time, Rank, and API Name. The 'Version Number' column has the value '1'. The 'Enabled' column has a checkbox that is currently unchecked. The 'Start Date Time' column has the value '6/22/2022, 2:25 AM'. The 'Rank' column has the value '1'. The 'API Name' column has the value 'AttributePricingMatrix_V1'.

6. Change the **Name** to `AttributePricingMatrix V2` and change the **Version Number** to 2.
7. Select **Save**.

New Decision Matrix Version

Information

Name

AttributePricingMatrix V2

Version Number

2

Decision Matrix

AttributePricingMatrix

Enabled

☐

Start Date Time

Date

6/22/2022

Time

2:25 AM

End Date Time

Date

Time

Rank

1

Configuration

Group Key

Sub Group Key

Group Key Value

Sub Group Key Value

Cancel

Save & New

Save

8. On the Decision Matrix Version screen, notice the preconfigured columns showing the Input and Output Data.



NOTE:

If more than one matrix version is enabled, the Rank field in the New Decision Matrix Version configuration dialog determines which version to use. The version with the higher number in the rank field will be used for calculation procedures.

9. Click **Add Row** four times to add four rows to the matrix.
10. Enter the following information to populate the matrix rows.



ALERT:

Data must be entered exactly as displayed, including spacing and proper case. These columns match the product and its attributes; therefore, they must be exact.

SourceProduct Name (Text)	SourceProduct Code (Text)	Characteristic Name (Text)	Characteristic Value (Text)	MRC (Currency)	NRC (Currency)
DSL Service	C-DSL-001	Download Speed; Upload Speed	20 Mbps; 2 Mbps	50.00	0
DSL Service	C-DSL-001	Download Speed; Upload Speed	20 Mbps; 896 Kbps	45.00	0
DSL Service	C-DSL-001	Download Speed; Upload Speed	40 Mbps; 2 Mbps	55.00	0
DSL Service	C-DSL-001	Download Speed; Upload Speed	40 Mbps; 896 Kbps	45.00	0

Matrix						
4 records • Sorted by SourceProduct Name						
Input Data				Output Data		
☐	SourceProduct Name (Text) ↑	SourceProduct Code (Text)	Characteristic Name (Text)	Characteristic Value (Text)	MRC (Currency)	NRC (Currency)
☐	DSL Service	C-DSL-001	Download Speed; Upload Speed	20 Mbps; 2 Mbps	\$50	\$0
☐	DSL Service	C-DSL-001	Download Speed; Upload Speed	20 Mbps; 896 Kbps	\$45	\$0
☐	DSL Service	C-DSL-001	Download Speed; Upload Speed	40 Mbps; 2 Mbps	\$55	\$0
☐	DSL Service	C-DSL-001	Download Speed; Upload Speed	40 Mbps; 896 Kbps	\$45	\$0

11. Select **Save**.

12. In the AttributePricingMatrix V2 header, select **Edit**.

13. Check the **Enabled** checkbox and select **Save**.

Edit AttributePricingMatrix V2

Information

Name

AttributePricingMatrix V2

Version Number

2

Matrix Type

Standard

Decision Matrix

AttributePricingMatrix

Load Process Status

Enabled

☒

Start Date Time

Date

6/22/2022

Time

2:25 AM

End Date Time

Date

Time

Rank

1

Configuration

Group Key

Sub Group Key

Cancel

Save & New

Save

TIPS:

Notice a mistake in the matrix after enabling? Click **Clone** from the dropdown arrow menu and correct the mistake in the new version of the matrix. Make sure that the new version is Enabled once you're done.

© Copyright 2022 Salesforce.com, inc. All rights reserved.

24

Task 5: Set up RangeAttributePricingMatrix

1. Navigate to the **Lookup Tables** tab. You may need to search for it again using the **Lightning App Launcher**.
2. Click the **RangeAttributePricingMatrix**, which was installed from the DataPack.
3. Select the **Related** tab and click **RangeAttributePricingMatrix V1**.
4. Click **Clone**.
5. Change the **Name** to `RangeAttributePricingMatrix V2` and change the Version Number to 2.
6. Click **Save**.
7. On the Decision Matrix Version screen, click **Add Row** two times to add two rows to the matrix.
8. Enter the following data in the new matrix rows:

SourceProductNa me (Text)	SourceProd uctCode (Text)	Characteristic Name (Text)	Characteristic Value (Text)	Quantity (Text)	MRC (Curr ency)	NRC (Curre ncy)
Home Hub Modem	C-MOD-002	Grade	Best	1-4	0	65.00
Home Hub Modem	C-MOD-002	Grade	Best	5-9	0	50.00



TIPS:

Notice that the Quantity column uses a Text column type instead of a Number Range column type. While either format is acceptable, it's recommended that you use Text column types for numeric ranges because they're easier to manage and maintain consistency when using several ranges in a single matrix.

Decision Matrix Version

RangeAttributePricingMatrix V2

EditClone

Version Number

2

Enabled

☐

Start Date Time

6/22/2022, 2:12 AM

End Date Time

Rank

1

API Name

RangeAttributePricingMatrix_V2

Matrix

Related

Details

2 records • Sorted by SourceProductName

Upload CSV FileGenerate CSV FileAdd ColumnAdd RowDelete RowsSave

Input Data						Output Data	
☐	SourceProductN... ↑	SourceProductCod...	CharacteristicNam...	CharacteristicValu...	Quantity (Text)	MRC (Currency)	NRC (Currency)
1	☐ Home Hub Modem	C-MOD-002	Grade	Best	1-4	\$0	\$65
2	☐ Home Hub Modem	C-MOD-002	Grade	Best	5-9	\$0	\$50

9. Select **Save**.

10. In the RangeAttributePricingMatrix V2 header, click **Edit**.

11. Check the **Enabled** checkbox and select **Save**.

Edit RangeAttributePricingMatrix V2

Information

Name

RangeAttributePricingMatrix V2

Version Number

2

Matrix Type

Standard

Decision Matrix

RangeAttributePricingMatrix

Load Process Status

Enabled

☒

Start Date Time

Date

6/22/2022

Time

2:12 AM

End Date Time

Date

Time

Rank

1

Cancel

Save & New

Save

© Copyright 2022 Salesforce.com, inc. All rights reserved.

27

Task 6: Set up SourceTargetAttributePricingMatrix

1. Navigate to the **Lookup Tables** tab. You may need to search for it again using the **Lightning App Launcher**.
2. Click the **SourceTargetAttributePricingMatrix**, which was installed from the DataPack.
3. Select the **Related** tab and click **SourceTargetAttributePricingMatrix V1**.
4. Click **Clone**.
5. Change the **Name** to `SourceTargetAttributePricingMatrix V2` and change the Version Number to 2.
6. Click **Save**.
7. On the Decision Matrix Version screen, click **Add Row** to add a row to the matrix.
8. Enter the following data in the new matrix rows:

SourceProduct Name (Text)	SourcePro ductCode (Text)	Characteristic Name (Text)	Characteristic Value (Text)	Target Product Name (Text)	MRC (Curre ncy)	NRC (Curre ncy)
Installatio n	C-INS-00 1	Installatio n Type	Full Service Install	Path1	0	0



TIPS:

You can enter the Target Product Name simply using the product name, which will then price all occurrences of that target product throughout the order. However, if you want to only price a specific occurrence of a product within a bundle, you can use a path parameter as shown in this example. The path parameter is described in the next task.

Decision Matrix Version

SourceTargetAttributePricingMatrix V2

EditClone

Version Number

2

Enabled

☐

Start Date Time

5/20/2022, 12:00 AM

End Date Time

Rank

1

API Name

SourceTargetAttributePricingMatrix_V2

Matrix

Related

Details

1 record • Sorted by SourceProductName

Upload CSV FileGenerate CSV FileAdd ColumnAdd RowDelete RowsCopyFilterSave

Input Data				Output Data		
☐ SourceProductNa... ↑ ↓	SourceProductCode ... ↓	CharacteristicName (... ↓	CharacteristicValue (... ↓	TargetProductName ... ↓	MRC (Currency) ↓	NRC (Currency)
1 ☐ Installation	C-INS-001	Installation Type	Full Service Install	Path1	\$0	\$0

9. Select **Save**.

10. In the SourceTargetAttributePricingMatrix V2 header, click **Edit**.

11. Check the **Enabled** checkbox and select **Save**.

Edit SourceTargetAttributePricingMatrix V2

Information

Name

SourceTargetAttributePricingMatrix V2

Version Number

2

Matrix Type

Standard

Decision Matrix

SourceTargetAttributePricingMatrix

Load Process Status

Enabled

☒

Start Date Time

Date

6/22/2022

Time

2:19 AM

End Date Time

Date

Time

Rank

1

Cancel

Save & New

Save



NOTE:

Due to the way that a lookup table requires input data to be unique across all rows in the matrix, it's not currently possible to use source-target attribute-based pricing to change more than one child product in a bundle.

Task 7: Set up AttributePricingProcedure Expression Set

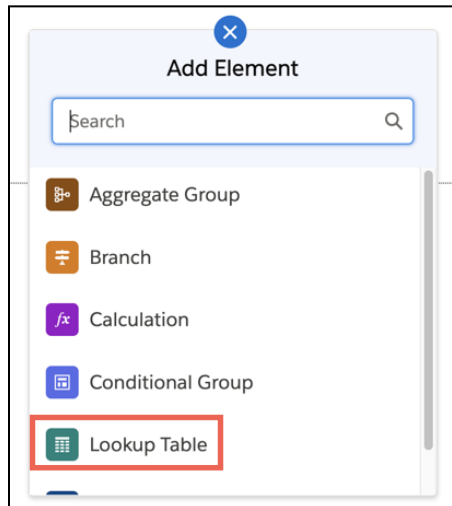
Now that Eliza has set up the lookup table, she'll create expression sets to perform calculations using the data entered in the matrices.

1. Using the **Lightning App Launcher**, search for and select **Expression Sets**.
2. Change the List Views to **All Expression Sets** and pin it so that All Expression Sets display instead of Recently Viewed.
3. Click **New**.
4. In the New Expression Set dialog box, enter `AttributePricingProcedure` in the **Name** field and select **Save**.
5. Click the **Related** tab.
6. Select the dropdown arrow  next to **AttributePricingProcedure V1** and click **Edit**.
7. In the **Rank** field, enter 1 and click **Save**.
8. Click **AttributePricingProcedure V1**.
9. Click **Open in Expression Set Builder** in the top right of the **AttributePricingProcedure V1** header.

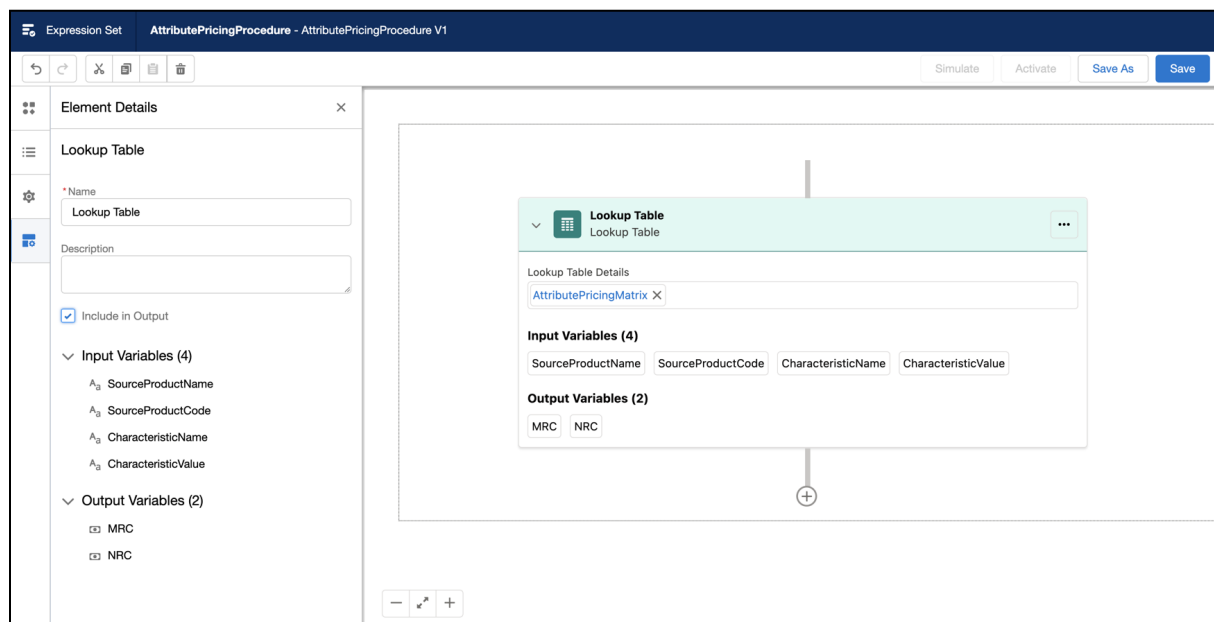


Version Number	Active	Start Date Time	End Date Time	Rank
1	☐	6/30/2022, 8:08 AM		1

10. Create the Lookup Table element.
 - a. In the **Expression Set Builder** window, select the + symbol to open the **Add Element** dialog box.
 - b. Select **Lookup Table** from the list below the **Search** box to add a Lookup Table element to the expression set.



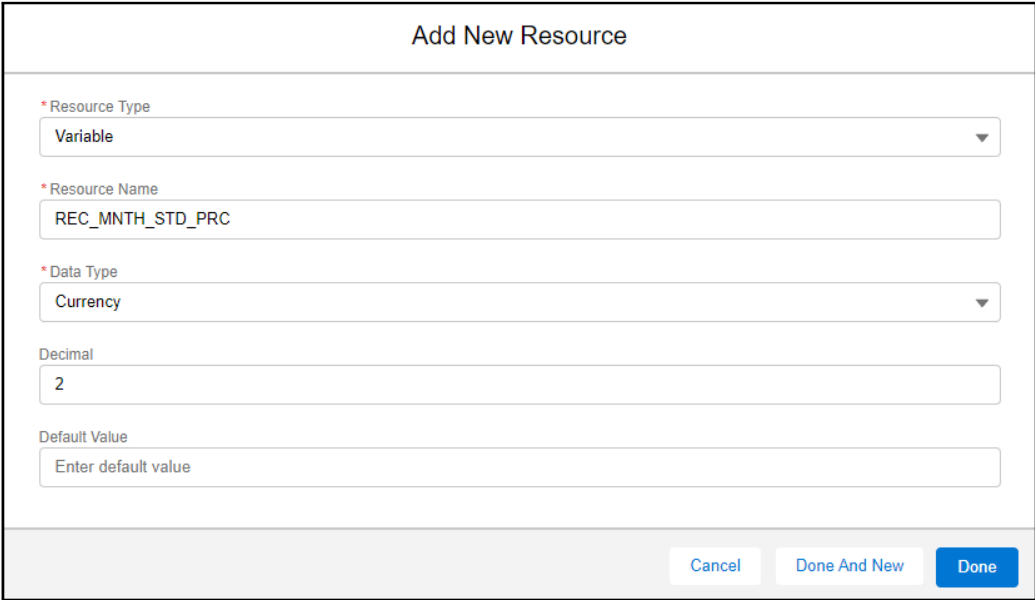
- c. In the Lookup Table Details field, search for and select **AttributePricingMatrix**.
- d. With the Lookup Table element selected, click the **Element Details** tab  in the left panel, and check the **Include in Output** checkbox.



11. Create the recurring monthly standard price variable.

- Click the **Resource Manager** tab  and click **Add Resource**.
- In the **Add New Resource** dialog, enter the following information to create the recurring monthly standard price variable:

Field	Entry
Resource Type	Variable
Resource Name	REC_MNTH_STD_PRC
Data Type	Currency
Decimal	2



- Click **Done**.

12. Create the one-time standard price variable.

- Click **Add Resource**.
- In the **Add New Resource** dialog, enter the following information to create the recurring monthly standard price variable and click **Done**:

Field	Entry
Resource Type	Variable
Resource Name	OT_STD_PRC
Data Type	Currency
Decimal	2

13. Create the recurring monthly standard price calculation element.

- Under the Lookup Table element, click the + symbol to open the **Add Element** dialog box, and select **Calculation**.
- In the **Formula** field of the new Calculation element, search for and select **AttributePricingMatrix__MRC**.
- In the **Output Variable** field, search for and select **REC_MNTH_STD_PRC**.



- With the new Calculation element selected, click the **Element Details** tab



in the left panel, and check the **Include in Output** checkbox.

14. Create the one-time standard price calculation element.

- Under the Lookup Table element, click the + symbol to open the **Add Element** dialog box, and select **Calculation**.
- In the **Formula** field of the new Calculation element, search for and select **AttributePricingMatrix__NRC**.
- In the **Output Variable** field, search for and select **OT_STD_PRC**.



Calculation
Calculation

Formula: AttributePricingMatrix_NRC = Output Variable: OT_STD_PRC

d. With the new Calculation element selected, click the **Element Details** tab



in the left panel, and check the **Include in Output** checkbox.

15. Select **Save** in the top right corner of the Expression Set Builder, and then click **Activate**.

16. Simulate the expression set to test out the procedure.

- Click **Simulate**.
- Enter the following input variables into the fields:

Field	Entry
Effective Date	[tomorrow's date]
CharacteristicName	Download Speed;Upload Speed
SourceProductCode	C-DSL-001
SourceProductName	DSL Service
CharacteristicValue	20 Mbps;2 Mbps



NOTE:

The sequence of the simulation parameters may be different in your org. Make sure to insert the correct value for each field.

- c. Click **Simulate** to run the expression set.

Input Variables
Enter input variable values to simulate your expression set.

Effective Date Range
Starts from Jun 30, 2022

Effective Date
Date: Jul 1, 2022
Time: 3:47 PM

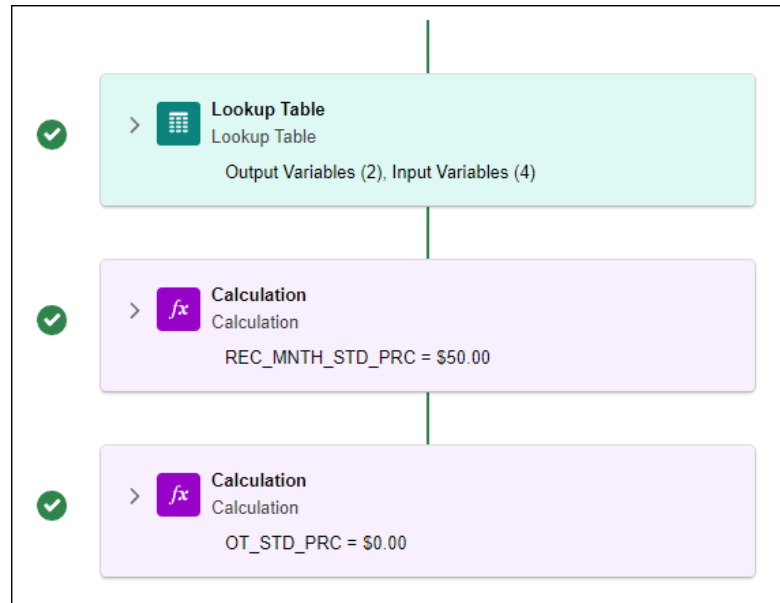
CharacteristicName
Text: Download Speed;Upload Spee

SourceProductCode
Text: C-DSL-001

SourceProductName
Text: DSL Service

CharacteristicValue
Text: 20 Mbps;2 Mbps

- d. Notice the green check marks that indicate that each element executed successfully. Also, notice the output of the REC_MNTH_STD_PRC calculation step is \$50.00. This price matches the value you entered in the 20 Mbps;2 Mbps row of the decision matrix.



17. Close the Expression Set Builder browser tab.

Task 8: Set up RangeAttributePricingProcedure Expression Set

1. Navigate to the **Expression Sets** tab. You may need to search for it again using the **Lightning App Launcher**.
2. Click **New**.
3. Enter `RangeAttributePricingProcedure` for the **Name** and select **Save**.
4. On the **Related** tab, select the dropdown arrow  next to **RangeAttributePricingProcedure V1** and click **Edit**.
5. In the **Rank** field, enter 1 and click **Save**.
6. Click **RangeAttributePricingProcedure V1**.
7. Select **Open in Expression Set Builder** from the header in the upper right.
8. In the Expression Set Builder window, click the plus + symbol and select **Lookup Table**.
9. In the **Lookup Table Details** field, search for and select **RangeAttributePricingMatrix**.
10. With the new Lookup Table element selected, click the **Element Details**  tab in the left panel, and check the **Include in Output** checkbox.
11. Click the **Resource Manager** tab  and select **Add Resource**.
12. As you did in the last exercise, add the recurring monthly standard price and one-time standard price variables:

Recurring Monthly Standard Price

Field	Entry
Resource Type	Variable
Resource Name	REC_MNTH_STD_PRC
Data Type	Currency
Decimal	2

One-Time Standard Price Variable

Field	Entry
Resource Type	Variable
Resource Name	OT_STD_PRC
Data Type	Currency
Decimal	2

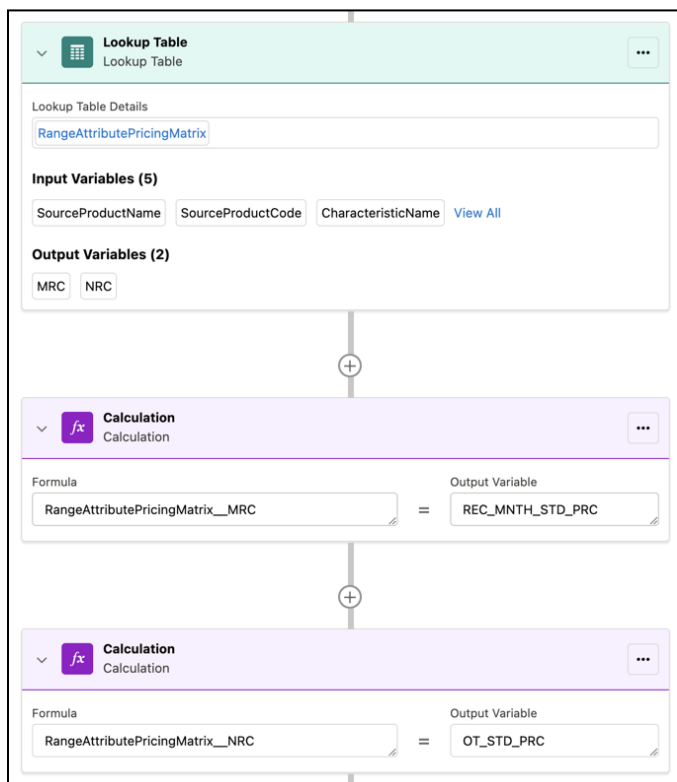
13. Add a new **Calculation** element and enter the following information:

Field	Entry
Formula	RangeAttributePricingMatrix__MRC
Output Variable	REC_MNTH_STD_PRC

14. Add another **Calculation** element and enter the following information:

Field	Entry
Formula	RangeAttributePricingMatrix__NRC
Output Variable	OT_STD_PRC

15. For all three elements, make sure that the **Include in Output** checkbox is selected in the **Element Details**  tab.



The screenshot shows the Expression Set Builder interface. At the top is a 'Lookup Table' element with the name 'RangeAttributePricingMatrix'. Below it are 'Input Variables (5)' (SourceProductName, SourceProductCode, CharacteristicName, View All) and 'Output Variables (2)' (MRC, NRC). Below the Lookup Table is a '+' connector. The next element is a 'Calculation' element with the formula 'RangeAttributePricingMatrix__MRC' and the output variable 'REC_MNTH_STD_PRC'. Below this is another '+' connector. The final element is another 'Calculation' element with the formula 'RangeAttributePricingMatrix__NRC' and the output variable 'OT_STD_PRC'.

16. Select **Save** and then click **Activate**.



TIPS:

To help troubleshoot any potential errors, try simulating the expression set by clicking **Simulate** and entering corresponding values from the connected decision matrix.

When simulating the RangeAttributePricingProcedure, you must enter the literal range values from the Quantity field of the RangeAttributePricingMatrix (e.g. "1-5") to return an accurate calculation. This is because the PricingPlanHelper.cls, which identifies the corresponding range based on the Quantity entered, is not called from the expression set. You'll test this calculation in the Cart in a later exercise step.

17. Close the Expression Set Builder browser tab.

Task 9: Set up SourceTargetAttributePricingProcedure Expression Set

Now that you've got the hang of expression sets, create and configure the SourceTargetAttributePricingProcedure to perform calculations using information from the SourceTargetAttributePricingMatrix.

1. Create an expression set and name it `SourceTargetAttributePricingProcedure`.
2. Enter 1 in the **Rank** field of the new version.
3. Open the expression set version in the Expression Set Builder.
4. Create a Lookup Table element to connect the **SourceTargetAttributePricingMatrix**.
5. Add the Recurring Standard Monthly Price (REC_MNTH_STD_PRC) and One-Time Standard Price (OT_STD_PRC) currency variables in the Resource Manager as you did in the last task.
6. Create a **Calculation** element to map the SourceTargetAttributePricingMatrix__MRC formula to the REC_MNTH_STD_PRC output variable as you did in the last task.
7. Create a **Calculation** element to map the SourceTargetAttributePricingMatrix__NRC formula to the OT_STD_PRC output variable as you did in the last task.
8. For all three elements, make sure that the **Include in Output** checkbox is selected.
9. Click **Save** and **Activate**.
10. Optional: Simulate the expression set and troubleshoot any errors that occur.

Task 10: Set up custom pricing steps

Eliza has created lookup tables and expression sets and she'll now configure steps in the Default Pricing Plan so that the attribute-based pricing calculations will display at runtime in the Cart.

1. Navigate to the **Vlocity Product Console**.
2. Next to Pricing Plan in the Pricing area, click the **search icon**  .
3. In the **Search Pricing Plan...** dialog box, press the return or enter key to display the Default Pricing Plan.
4. Click on the **Default Pricing Plan**.
5. Click on the **Pricing Plan Steps** facet.
6. Review the sequence of steps. Refer below for a description of each Pricing Plan Service Step.

10 Initialize	Pricing context information used by steps
20 Load PLEs	Loads PLEs for products in the cart (tightest match)
30 Initialize	Charges applied pricing variables for each line item
35 Apply Context Discounts	Apply order, account, or contract-based discounts
40 Apply Adjustments	Apply overrides/adjustments defined in offer
50 Apply Adjustments	Apply overrides/adjustments defined in promotion
60 Apply Adjustments	Apply overrides/adjustments done manually
70 Calculate	Calculates line level totals and rollup pricing variables
75 Load Margin Ranges	Invokes an expression set and matrix
80 Save	Saves updated line items to database
90 Apply/Save	Calculates/saves pricing variables of parent object

7. Create the Attribute Based Pricing step.

- a. Click **New Item**.
- b. Enter the following data in the **General Properties** pane:

Field	Entry
Name	Calculate Attribute Based Pricing
Implementation Name	CustomPricingPlanStepImpl
Method Name	GetMatrixPrice
Sequence	5
Active	✓



ALERT:

All attribute pricing steps must occur before the Initialize Pricing Context step; therefore, the sequence must be before 10.

- c. Click the **Edit** icon  to open the **Parameters** dialog box.
- d. Click the plus + symbol to add the following parameters:

First Box	Second Box
ProcedureName	AttributePricingProcedure
MatrixName	AttributePricingMatrix
DecisionMatrix	true

Parameters

ProcedureName	AttributePricingProcedure	🗑️
MatrixName	AttributePricingMatrix	🗑️
DecisionMatrix	true	🗑️

+

Done



TIPS:

Click on the + multiple times to create blank rows avoiding the need to click on the + each time a new row is needed.

- e. Click **Done**.
- f. Click **Save** in the General Properties pane.

General Properties

Pricing Plan Steps

New Item

SEQUENCE	NAME	DESCRIPTION	IMPLEMENTATION NAME	METHOD NAME
5	Calculate Attribute Based Pricing		CustomPricingPlanStepImpl	GetMatrixPrice
10	Initialize Pricing Context		DefaultPricingPlanImplementation	InitializePricingC
20	Load Price List Entries		DefaultPricingPlanImplementation	LoadTightestMat
30	Initialize Pricing Variables		DefaultPricingPlanImplementation	InitializePricingV
35	Apply Context Discounts		DefaultPricingPlanImplementation	ApplyContextDis
40	Apply Offer Adjustments		DefaultPricingPlanImplementation	ApplyOfferAdjust

General Properties

* Name Calculate Attribute Based Pricin

Description

* Implementation Name CustomPricingPlanStepImpl

* Method Name GetMatrixPrice

* Sequence 5

Active ☒

Parameters {"ProcedureName": "Attrib
utePricingProcedure"; "Ma
trixName": "AttributePricingMatrix"; "DecisionMatrix": true}

Cancel
Save

8. Create a new pricing plan step for Source Target Attribute Pricing using the following information.

- a. Click **New Item**.
- b. Enter the following information in the General Properties pane:

Field	Entry
Name	Calculate Source Target Attribute Pricing
Implementation Name	CustomPricingPlanStepImpl
Method Name	GetMatrixPrice
Sequence	6
Active	✓

- c. Click the plus + symbol to add the following parameters:

First Box	Second Box
ProcedureName	SourceTargetAttributePricingProcedure
MatrixName	SourceTargetAttributePricingMatrix
Path1	Back to School Student Offer<Back to School Backpack
DecisionMatrix	true



TIPS:

When you identify a path, you are pricing a target product differently. It prices the target product based on the bundle it is in. The target path must be formatted as: [root product name]<[parent1 product name][parent2 product name][...]<[product name].

In this example, Path1 corresponds to the Target Product Name in the SourceTargetAttributePricingMatrix.

Parameters

ProcedureName	SourceTargetAttributePricingProcedure	✕
MatrixName	SourceTargetAttributePricingMatrix	✕
Path1	Back to School Student Offer<Back to School Backpack	✕
DecisionMatrix	true	✕

+

Done

- d. Click **Done**.
 - e. Click **Save** in the General Properties pane.
9. Create a new pricing plan step for Range Attribute Pricing.
 - a. Click **New Item** and enter the following information:

Field	Entry
Name	Calculate Range Attribute Pricing
Implementation Name	CustomPricingPlanStepImpl
Method Name	GetMatrixPrice
Sequence	7
Active	✓

- b. Add the following parameters to the step:

First Box	Second Box
ProcedureName	RangeAttributePricingProcedure
MatrixName	RangeAttributePricingMatrix
RangeFields	Quantity
DecisionMatrix	true
IncludeAttrInfoInRangeKeys	false

- c. Click **Done**.
d. Click **Save** in the General Properties pane.

Parameters

ProcedureName	RangeAttributePricingProcedure	⛶
MatrixName	RangeAttributePricingMatrix	⛶
RangeFields	Quantity	⛶
DecisionMatrix	true	⛶
IncludeAttrInfoInRangeKeys	false	⛶

+

Done



NOTE:

When you want to use range attribute pricing, you specify the API names of fields or attributes using the RangeFields or RangeAttributes parameters. The fields and attributes listed need to have corresponding columns in the RangeAttributePricingMatrix.



NOTE:

The IncludeAttrInfoInRangeKeys parameter controls how the range column of the decision matrix is handled based on the conditional attributes in the other columns. In the range attribute matrix you created, the attributes are the same across all of the rows, and the quantity fields alone control the price of the product:

C-MOD-002 | Grade | Best | 1-4 | \$65

C-MOD-002 | Grade | Best | 5-9 | \$50

But what if you wanted to repeat the ranges and include another characteristic value (e.g. “Good”) as a condition, as shown below?

C-MOD-002 | Grade | Best | 1-4 | \$65

C-MOD-002 | Grade | Good | 1-4 | \$50

In this case, you’d set the IncludeAttrInfoInRangeKeys parameter to ‘true’ in the pricing plan step so that each attribute value is calculated along with the repeated range.

Task 11: Refresh the platform cache

Pricing plan steps and/or parameters are cached in the platform cache after they are executed for the first time in the Cart. Anytime you make changes to a pricing plan step/parameter you are required to run the maintenance jobs **Clear Managed Platform Cache** and **Refresh Platform Cache**. When the Cart is invoked, the platform cache will be populated with the new pricing plan step and/or parameter data.

1. Navigate to the **Vlocity CMT Administration** tab. You may need to click the **More** dropdown menu to expand for additional tabs.
2. Under **Admin Console**, click **Maintenance Jobs**.
3. For the job **Clear Managed Platform Cache**, click **Start** and **OK**.
4. For the job **Refresh Platform Cache**, click **Start** and **OK**.

Task 12: Add products to the Cart to initiate attribute-based pricing

1. Add products to the Cart.

- a. Click the **Orders** tab and click **New**.
- b. On the **New Order** dialog, enter the following:

Field	Value
Order Name	Attribute Based Pricing Order
Account Name	White, Noah
Order Start Date	[Today's date]
Price List	B2C Price List

- c. Click **Save**.
- d. Click on the Power Launcher's search field and select **Configure Order**.
- e. Search for the product `Back to School`.
- f. Add the **Back to School Student Offer** and **Back to School Backpack** to the Cart.
- g. Expand the **Messages** information bar to expose the **Take Me There** icon  to address the required missing attributes.
- h. For **DSL Service**, select **40 Mbps** for the **Download Speed** and **2 Mbps** for the **Upload Speed**.
- i. For **Home Hub Modem**, select **Better** for the **Grade**.
- j. Click **Close**.
- k. Search for the product **Home Hub Modem** and add it to the Cart.
- l. Click the **Take Me There** icon  to address the required missing attribute.
- m. Select **Best** for the **Grade** and click **Close**.

CART PROMOTIONS DISCOUNTS (0)								
Advanced Pricing ▼								
	Item Name	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	Action	Promotions
	Back to School B...	1	\$0.00	\$0.00	\$9.99	\$9.99	Add	▼
	Home Hub Modem	1	\$0.00	\$0.00	\$65.00	\$65.00	Add	▼
▼	Back to School St...	1	\$0.00	\$45.00	\$0.00	\$229.99	Add	▼
▼	Home Internet S...	1	\$0.00	\$45.00	\$0.00	\$229.99	Add	▼
	DSL Service	1	\$45.00	\$45.00	\$0.00	\$0.00	Add	▼
	Installation	1	\$0.00	\$0.00	\$99.99	\$99.99	Add	▼
	Home Hub Modem	2	\$0.00	\$0.00	\$65.00	\$130.00	Add	▼
	Back to School B...	1	\$0.00	\$0.00	\$0.00	\$0.00	Add	▼

2. Test the **AttributePricingMatrix**

- In the **Recurring Charge** column for **DSL Service**, click on the **charge** and select the **Price Details**  icon.
- Notice that the price returned is from the Attribute Pricing Procedure based on the data entered in the matrix.

Details	
Priced from AttributePricingProcedure	\$55.00
	\$55.00

- Click **Close**.
- Change the Download and Upload Speeds to those identified in the matrix. Does the recurring charge change according to the amounts entered in the matrix?

3. Test the **RangeAttributePricingMatrix**.

- In the **One Time Charge** column for the **Home Hub Modem**, (not part of the Back to School Student Offer bundle) notice the price.

CART

PROMOTIONS

Advanced Pricing

	Item Name	Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	Action
	Home Hub Modem	1	\$0.00	\$0.00	\$65.00	\$65.00	Add

- Change the **quantity** to **5**. Did the price change to the amount identified in your Range Attribute Pricing Matrix from Task 5?
- In the **One Time Charge** column, click on the **charge** and select the **Price Details**  icon.
- Notice that the price returned is from the Range Attribute Pricing Procedure based on the data entered in the matrix.

Details	
Priced from RangeAttributePricingProcedure	\$50.00
	\$50.00

- Click **Close**.

4. Test the **SourceTargetAttributePricingMatrix**.

- Notice the price of the **Back to School Backpack** when added to the Cart not as part of a bundle.
- In the **One Time Charge** column for **Back to School Backpack** (part of the Back to School Student Offer bundle), click on the **charge** and select the **Price Details**  icon.

- c. Notice that the price returned is from the Source Target Attribute Pricing Procedure based on the data entered in the matrix (Path1).

Details	
Priced from SourceTargetAttributePricingProcedure	\$0.00
	\$0.00

- d. Click **Close**.
- e. The custom pricing plan step parameter (Path1) and the target product (Path1) in the matrix states that if the Back to School Student Offer is added to the Cart with Full Service Install set as the Installation Type the Back to School Backpack is \$0.

Parameters	
ProcedureName	SourceTargetAttributePricingProcedure
MatrixName	SourceTargetAttributePricingMatrix
Path1	Back to School Student Offer<Back to School Backpack
DecisionMatrix	true
Done	

Matrix		Related		Details	
1 record • Sorted by SourceProductName					
Upload CSV FileGenerate CSV File					
Input Data				Output Data	
☐ SourceProductName (Text) ↑	SourceProductCode (Text)	CharacteristicName (Text)	CharacteristicValue (Text)	TargetProductName (Text)	MRC
1 ☐ Installation	C-INS-001	Installation Type	Full Service Install	Path1	\$0

Item Name	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	Action	Promotions
Back to School Bac...	1	\$0.00	\$0.00	\$9.99	\$9.99	Add	
Back to School Stu...	1	\$0.00	\$55.00	\$0.00	\$174.98	Add	
Home Internet Sol...	1	\$0.00	\$55.00	\$0.00	\$174.98	Add	
DSL Service	1	\$55.00	\$55.00	\$0.00	\$0.00	Add	
Installation	1	\$0.00	\$0.00	\$99.99	\$99.99	Add	
Home Hub Modem	1	\$0.00	\$0.00	\$74.99	\$74.99	Add	
Back to School Bac...	1	\$0.00	\$0.00	\$0.00	\$0.00	Add	

Troubleshooting Guide

If you encountered issues while performing the simulation of an expression set, or aren't getting the intended results in the Cart after completing the above tasks, try the following.

Lookup Table (Decision Matrix)

- Check that all matrix row information and expression set variables are entered correctly.
- Check that the matrix is enabled.
- Make sure that the MRC and NRC columns are configured as Currency and Output column types.

Expression Set

- Make sure you've set the Rank of the expression set version to 1.
- Check that each expression set element has the Include in Output checkbox selected in the Element Details tab in the Expression Set Builder.
- Make sure that the REC_MNTH_STD_PRC and OT_STD_PRC variables are configured with the type Currency in the expression set.
- Check that the expression set is activated.

Pricing Plan Step

- Verify that all data input is spelled correctly and has proper spacing.
- Make sure each step is active.

Exercise 9-2: Create Loyalty Pricing

Scenario

The marketing manager realizes the importance of retaining existing customers, as well as bringing in new customers, and wants to implement a customer loyalty program that rewards purchasing behavior. By implementing a loyalty program, Infiwave wants to send a message that they are interested in a relationship with their current and future customers. Eliza needs to enable and set up loyalty pricing, so customers can pay for products with loyalty points.

Goals

- Explain how to set up loyalty pricing
- Pay for a product using loyalty points

Tasks

1. Enable loyalty pricing
2. Create pricing elements for loyalty point charges
3. Assign loyalty pricing to products
4. Refresh the cache
5. Create orders using loyalty pricing

Time: 10 mins

Loyalty Pricing

Loyalty pricing allows customers to pay for any product with loyalty points using their loyalty credits. Once loyalty pricing is enabled, the following are available out of the box:

- One Time Loyalty pricing variable to set the one-time price type.
- Pricing variable with pricing variable bindings to ensure the correct calculation of each line item in the Cart.
- Additional columns for the Cart to display the one-time loyalty price and one-time loyalty total for each line item.
- Loyalty points total on the Cart's total bar.

Industries CPQ provides the ability to pay for an item in currency or loyalty points. Administering and managing point balances is handled by an external back office system.

Creating Pricing Elements for Loyalty Pricing

Pricing elements are stored within their respective price lists. When you create price list entries for products that have charges in loyalty points, you must have existing pricing elements that contain those charges.

\$89.99 RM	RM-89.99	Recurring Monthly Std Price	Currency	89.99	USD
\$499.99 OT	OT-499.99	One Time Std Price	Currency	499.99	USD
2500 OT	LP-2500-OT	One Time Loyalty	Loyalty	2500	PTS
500 OT	LP-500-OT	One Time Loyalty	Loyalty	500	PTS


Samsung Fast Charge Wireless Charging Stand

General Properties
CHARGES
COSTS
ADJUSTMENTS
OVERRIDES

Attribute Properties
Attributes and Fields
Attachments
Product Structure
Pricing
Context Rules
Layout Management
Debug

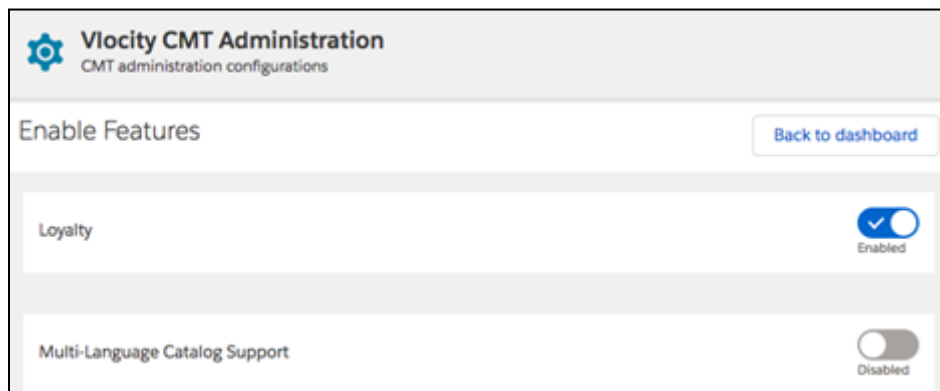
New

Showing 1 - 2 of 2 results

PRICE LIST	DISPLAY TEXT	PRICING VARIABLE	PRICING ELEMENT
  B2C Price List	\$49.99	One Time Std Price	Name: \$49.99 OT Code: OT-49.99 Display Text: \$49.99 Calculation Type: Simple Charge: 49.99 USD
  B2C Price List	500 LP	One Time Loyalty	Name: 500 OT Code: LP-500-OT Display Text: LP-500-OT Calculation Type: Simple Charge: 500 PTS

Task 1: Enable loyalty pricing

1. Navigate to the **Vlocity CMT Administration** tab. You may need to click the **More** dropdown menu to expand for additional tabs.
2. Under **Custom Settings**, click **Enable Features**.
3. Next to **Loyalty**, click the button currently set to Disabled.
4. The Enable Loyalty dialog window opens, click **Enable**.
5. Click **OK** to the message that the feature has been enabled successfully.



Task 2: Create pricing elements for loyalty point charges

1. Navigate to the **Vlocity Product Console**.
2. Next to **Price List** in the **Pricing** area, click the **search icon** .
3. In the **Search Price List...** dialog box, enter **B2C** and click the search icon.
4. Click the **Edit**  icon next to the **B2C Price List**.
5. Click on the **Pricing Elements** facet.
6. Click **New**.
7. Select the following data for the **Pricing Variable** fields:

Field	Value
Charge Type	One-time
Sub-Type	Standard

8. Click **Search** and select **One Time Loyalty**.
9. Enter the following data for the **General Properties** fields:

Field	Value
Name	500 OT
Code	LP-500-OT
Display Text	LP-500-OT

10. Enter the following data for the **Loyalty Value** fields:

Field	Value
Loyalty Amount	500
Loyalty Code	PTS

11. Enter the following data for the **Effectivity** fields:

Field	Value
Effective From	[today's date]
Active	✓

12. Click **Save**.

Pricing Variable

Charge Type *

One-time

Sub-Type *

Standard

Search

Selected Pricing Variable: One Time Loyalty

General Properties

Name *

500 OT

Code *

LP-500-OT

Display Text *

LP-500-OT

Help Text

Loyalty Value

Calculation Type

Simple

Loyalty Amount *

500

Loyalty Code *

PTS

Effectivity

Effective From *

6/14/2019

12:00 AM

Effective Until

Active

☒

Save

Task 3: Assign loyalty pricing to products

1. Navigate to the **Vlocity Product Console**.
2. Next to **Product** in the **Product Management** area, click the **search icon** .
3. In the **Search Product...** dialog box, enter *Samsung* and click the **search icon**.
4. Click the **Edit icon**  next to the **Samsung Fast Charge Wireless Charging Stand**.
5. Click on the **Pricing** facet for the product.
6. Click **New** from the **CHARGES** tab.
7. Enter the following data for the **General Properties** fields:

Field	Value
Price List	B2C Price List
Display Text	500 LP
Base Price	✓

8. Select the following data for the **Pricing Variable** fields:

Field	Value
Charge Type	One-time
Sub-Type	Standard

9. Click **Search** and select **One Time Loyalty**.
10. Select **500 OT** for the **Pricing Element**.
11. Enter the following data for the **Effectivity** fields:

Field	Value
Effective From	[today's date]
Active	✓

12. Click **Save**.


Samsung Fast Charge Wireless Charging Stand

General Properties
CHARGES
COSTS
ADJUSTMENTS
OVERRIDES

Attribute Properties
Attributes and Fields
Attachments
Product Structure
Pricing
Context Rules
Layout Management
Debug

New

Showing 1 - 3 of 3 results

PRICE LIST	DISPLAY TEXT	PRICING VARIABLE	PRICING ELEMENT
  B2C Price List	\$49.99	One Time Std Price	Name: \$49.99 OT Code: OT-49.99 Display Text: \$49.99 Calculation Type: Simple Charge: 49.99 USD
  B2B Price List	\$50	One Time Std Price	Name: \$50 OT Code: OT-50 Display Text: \$50 Calculation Type: Simple Charge: 50 USD
  B2C Price List	500 LP	One Time Loyalty	Name: 500 OT Code: LP-500-OT Display Text: LP-500-OT Calculation Type: Simple Charge: 500 PTS



NOTE:

This task is required as a one-time setup due to enabling the Loyalty Pricing feature and creating price list entries for loyalty pricing in a new training playground. Refreshing the platform cache will not be required when creating additional price list entries to be used with loyalty pricing.

Task 4: Refresh the cache

1. Navigate to the **Vlocity CMT Administration** tab. You may need to click the **More** dropdown menu to expand for additional tabs.
2. Under **Admin Console**, click **Maintenance Jobs**.
3. For the job **Clear Managed Platform Cache**, click **Start** and **OK**.
4. For the job **Refresh Platform Cache**, click **Start** and **OK**.

Task 5: Create orders using loyalty pricing

1. Navigate to the **Orders** tab and click **New**.
2. On the New Order, enter the following:

Field	Value
Order Name	Loyalty Pricing Order
Account Name	White, Noah
Order Start Date	[today's date]
Price List	B2C Price List

3. Click **Save**.
4. Click on the Power Launcher's search field and select **Configure Order**.
5. Search for the product **Samsung**.
6. For the **Samsung Fast Charge Wireless Charging Stand** notice the List Price is now \$49.99 or 500 LP.

Samsung Fast Charge Wireless Charging Stand[More](#)

RECURRING PRICE \$0.00

LIST PRICE \$49.99 or 500 LP

Add to Cart

7. Change the **Payment Choice** in the header to **Loyalty**.

**Loyalty Pricing Order**
READY TO SUBMIT

Price List
B2C Price List ▼

Payment Choice
Loyalty ▼

8. Add the **Samsung Fast Charge Wireless Charging Stand** to the Cart. Notice the One Time Loyalty Price column is 500 PTS.

9. Click the **One Time Loyalty Price** link and select the **Payment Type** button to open the Payment Choice dialog box.

CART PROMOTIONS DISCOUNTS (0)									
AdvancedPricing + Loyalty ▼									
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total		One Time Loyalty Total	Action	Promotions
Samsung Fast...	1	\$0.00	\$0.00	\$0.00	\$0.00	500 PTS	500 PTS	Add	

10. Change the Payment Choice to **Currency** and click **Apply**.

Payment Choice

You are configuring

Samsung Fast Charge Wireless Charging Stand

One Time Loyalty Price

☒ Currency

\$49.99 USD

☐ Loyalty

500 PTS

Close

Apply

11. Notice the One Time Loyalty Price is now 0 PTS and the One Time Charge is \$49.99.

CART PROMOTIONS DISCOUNTS (0)									
AdvancedPricing + Loyalty ▼									
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	One Time Loyalty Price	One Time Loyalty Total	Action	
Samsung Fast...	1	\$0.00	\$0.00	\$49.99	\$49.99	0 PTS	0 PTS	Add	

12. Change the Payment Choice back to **Loyalty** from the One Time Loyalty Price or One Time Charge links and click **Apply**.
13. Click **Submit Order**.
14. Click **Next**.
15. Notice the asset is priced according to the loyalty points.

ASSETS									
2022-07-04		Change to Quote Change to Order Move							
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	One Time Loyalty Price	One Time Loyalty Total	Promotions	
☐ Samsung ...	1	\$0.00	\$0.00	49.99 \$0.00	\$0.00	500	500		

Exercise 9-3: Work with Repricing

Scenario

Infiwave needs the ability to reprice products based on competition, supply and demand, market value and consumer interest. Eliza realizes the importance of staying competitive with the competition and informs management about the automatic repricing feature. Eliza knows that all Platinum customers are to receive special pricing on the Blast! Pro Internet product but learns that Acme is no longer a Platinum customer. Eliza needs to reprice the Blast! Pro Internet asset on Acme's account.

Goal

- Describe how to use repricing to reprice order line items and assets
- Explain how repricing adheres to context rules and effectivity dates on price list entries
- Explain how to invoke the repricing service

Tasks

1. Activate the RepricingElementServiceImplementation
2. Reprise order line items to reflect a price effectivity change
3. Reprice an asset to reflect a context rule condition for an account

Time: 20 mins

Repricing

Repricing allows you to reprice line items in opportunities, orders, quotes, and assets as needed by your business operations. You can use repricing to support products with prices that fluctuate over time, like natural resources, or products that are priced based on usage or currency exchange rates.

Repricing is done as a background process across one or more records. During repricing, prices are evaluated to ensure that they still apply based upon the effectivity dates of the price list entry for the product or the applied promotion. In addition, rules are re-run to ensure that the line items still meet the criteria for any context rules applied to the price list entry. Context rules for any manual adjustments can also be re-run to ensure that they are still valid. If the manual adjustment is no longer valid, the pricing for that line item reverts back to original pricing. Any price list entries that do not have a current effectivity date yields no change when repricing is run.

There are two different implementations available in the `RepricingInterface`:

- **DefaultRepricingImplementation:** will not make any changes to the existing prices.
- **RepricingElementServiceImplementation:** will reprice line items or parent objects of the line items (such as order line items or the order header) based on the price list entry effectivity dates attached to the product or promotion at that point in time when the repricing service is run.

There is also the `RepricingManAdjEligibilityInterface`, which must have the `RepricingManualAdjEligibilityService` implementation active to support repricing using context rules for pricing adjustments.

When repricing, set the `RepricingInterface` implementation to `RepricingElementServiceImplementation`. Repricing large updates in real time could cause performance degradation to occur if not run via a batch job. Consider repricing in batches with input parameters such as batch size or reprice provided line items only.

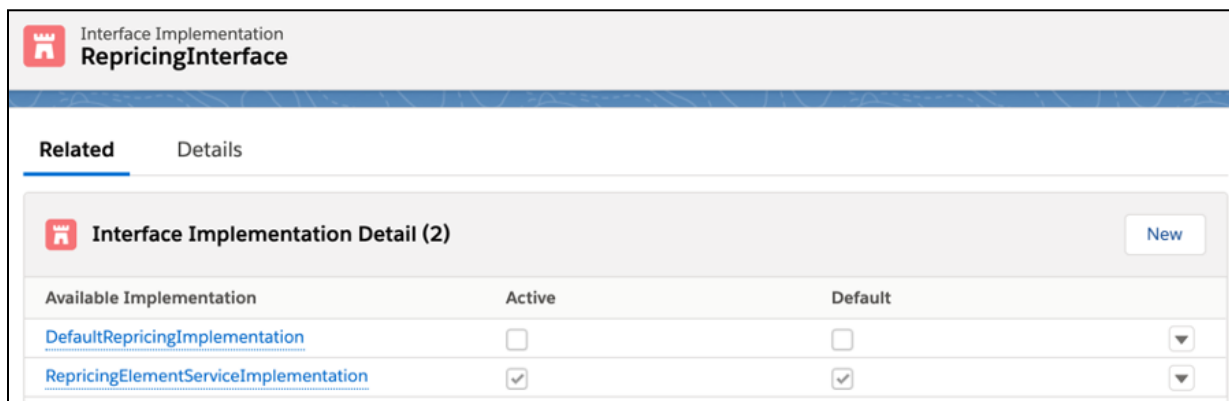


ALERT:

Repricing utilizes the `RepricingElementServiceImplementation` whereas attribute-based pricing utilizes the `PricingPlanService` implementation. Therefore, repricing does not support attribute-based pricing.

Task 1: Activate the RepricingElementServiceImplementation

1. Navigate to the **Interface Implementations** tab.
2. Change the **List Views** to **All**.
3. Click on **RepricingInterface**.
4. Using the dropdown menu for **RepricingElementServiceImplementation**, click **Edit**.
5. Check the **Active** and **Default** checkboxes and click **Save**.



The screenshot shows the Salesforce 'Interface Implementation' page for 'RepricingInterface'. It has tabs for 'Related' and 'Details'. Below the tabs is a table titled 'Interface Implementation Detail (2)' with a 'New' button. The table has three columns: 'Available Implementation', 'Active', and 'Default'. There are two rows of data.

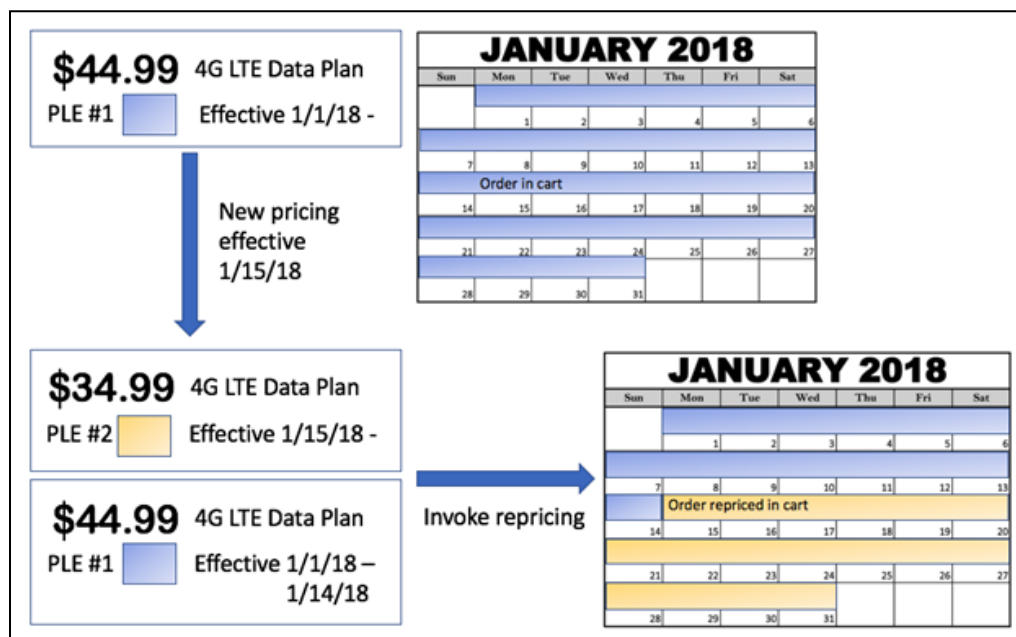
Available Implementation	Active	Default
DefaultRepricingImplementation	☐	☐
RepricingElementServiceImplementation	☒	☒

Task 2: Reprice order line items to reflect a price effectivity change

Due to changes in market conditions, the marketing team decides to lower the price of the 4G LTE Data Plan product starting on a certain date, and they want this new price to be available to any orders that are still pending and have not been submitted for fulfilment. To accomplish this, create a new price list entry and end-date the existing price list entry. Once the price list entries are created and updated, run the repricing service in order to reflect the new pricing on existing orders to ensure customers can take advantage of this new pricing.

Product	Price List Entry	Effectivity Start Date	Effectivity End Date
4G LTE Data Plan	\$44.99 RM	Jan 1, 2018	Jan 14, 2018
4G LTE Data Plan	\$34.99 RM	Jan 15, 2018	

Product	Order Date	Price in cart
4G LTE Data Plan	Jan 15, 2018	\$44.99 \$34.99



1. Create a new price list entry and end-date an existing price list entry.
 - a. Navigate to the **Vlocity Product Console**.
 - b. Next to **Product** in the **Product Management** area, click the **search icon** .
 - c. In the **Search Product...** dialog box, press the return or enter key to display the list of products.
 - d. Click on the **4G LTE Data Plan**.
 - e. Click on the **Pricing** facet.
 - f. Click **New** on the **CHARGES** tab to create a new price list entry.
 - g. Enter the following data for the **Price List Entry** fields:

Field	Value
Price List	B2C Price List
Display Text	\$24 . 99
Base Price	✓
Charge Type	Recurring
Frequency	Monthly
Sub-Type	Standard

- h. Click **Search** and select **Recurring Monthly Std Price**.
 - i. Select **\$24.99 RM** for the **Pricing Element**.
 - j. Enter the following data for the **Effectivity** fields:

Field	Value
Effective From	[today's date]
Active	✓

PRICE LIST ENTRY

CONTEXT RULES

General Properties

Price List *

B2C Price List

Display Text *

\$24.99

Base Price

☒

Virtual Price

☐

Pricing Variable

Charge Type *

Recurring

Frequency *

Monthly

Sub-Type *

Standard

Search

Selected Pricing Variable: Recurring Monthly Std Price

Pricing Element

Selected Pricing Element: \$24.99 RM

Time Plan/Policy

Time Plan

Time Policy

Effectivity

Effective From

6/14/2019

12:00 AM

Effective Until

Active

☒

Cancel

Save

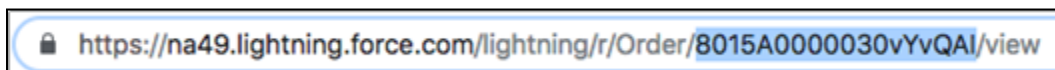
- k. Click **Save**. Notice the WARNING, you currently have two price list entries that overlap.



- l. Click the **X** to close the Warning message.
- m. Click the **Edit** icon  to modify the \$34.99 RM price list entry.
- n. Enter the following data for the **Effectivity** fields:

Field	Value
Effective Until	[yesterday's date]

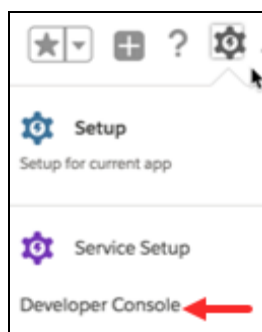
- o. Click **Save**.
- p. Notice the WARNING, you currently have two price list entries that overlap.
- q. Click the **X** to close the Warning message.
2. Review an existing order for the 4G LTE Data Plan product.
- Navigate to the **Orders** tab.
 - Change the **List Views** to **All Orders**.
 - Select the **Adv Pricing Data Plan Order**.
 - From the URL, copy the **OrderId** from after the Order/ and before /view and paste it to a text editor. You will need this Id for the Apex code when repricing the order.



- e. Click on the Power Launcher's search field and select **Configure Order**.
- f. Notice the **Recurring Charge** is **\$34.99**.

CART PROMOTIONS DISCOUNTS (0)				
AdvancedPricing + Loyalty ▼				
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge
4G LTE Data Plan	1	\$34.99	\$34.99	\$0.00

- g. Run the repricing service on orders.
3. Navigate to the **Setup** icon and select **Developer Console**.



- a. From the **Developer Console**, click **Debug > Open Execute Anonymous Window**.
- b. The following Apex code will be used to reprice the Adv Pricing Data Plan Order. Copy this code and paste it in the **Enter Apex Code** box from the **Developer Console**.

Apex Code

```
List<Order> a = [SELECT Id FROM Order WHERE Id
='8015A0000030vYvQAI'];
Map<String,Object> repricingInput = new
Map<String,Object>{'objectList' => a};
Map<String,Object> repricingOutput = new Map<String,Object>();
Map<String,Object> repricingOptions = new Map<String,Object>();
vlocity_cmt.VOIInvoker voi =
vlocity_cmt.VOIInvoker.getInstance();
```

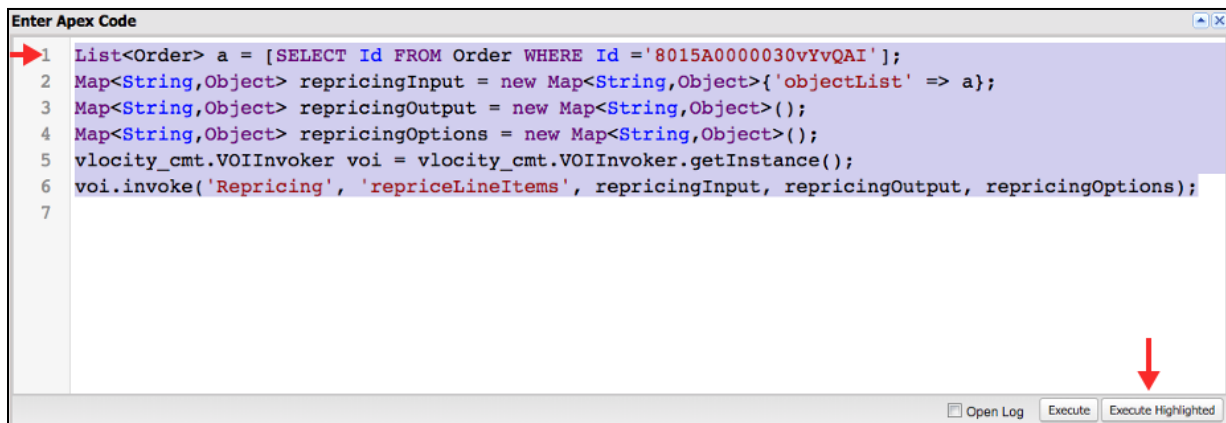
```
voi.invoke('Repricing', 'repriceLineItems', repricingInput,  
repricingOutput, repricingOptions);
```



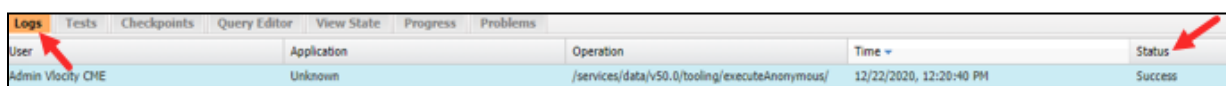
ALERT:

You will replace the OrderId in the Apex code with the OrderId from your training playground. The OrderId used in this example will not be the same in your environment.

- c. Copy the OrderId from your text editor and paste it over the OrderId in the Apex code in Line 1.



- d. Select the code and click **Execute Highlighted**.
- e. Verify the status is **Success** in the logs by clicking the Logs option in the bottom window.

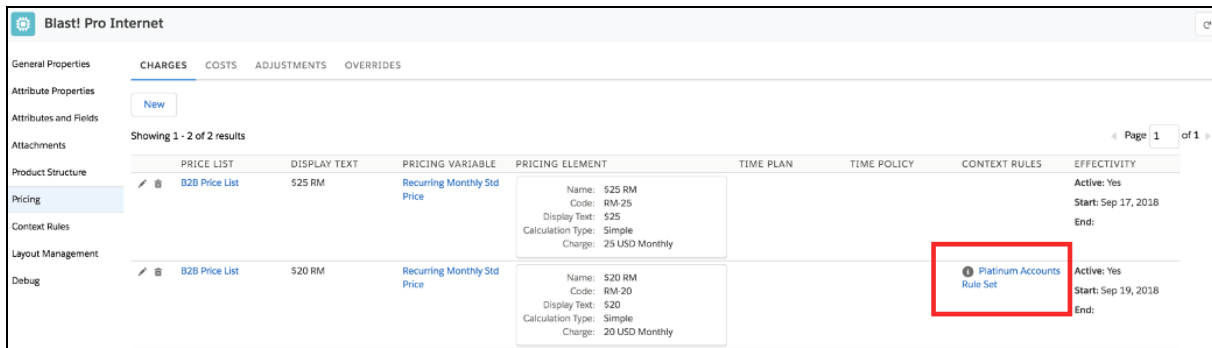


4. Review the repriced order.
 - a. Navigate back to the tab where the 4G LTE Data Plan is in the cart for the **Adv Pricing Data Plan Order**.
 - b. Refresh the browser page.
 - c. Notice the **Recurring Charge** is now **\$24.99** from the new price list entry.

CART PROMOTIONS DISCOUNTS (0)					
AdvancedPricing + Loyalty ▼					
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total
4G LTE Data Plan	1	\$24.99	\$24.99	\$0.00	\$0.00

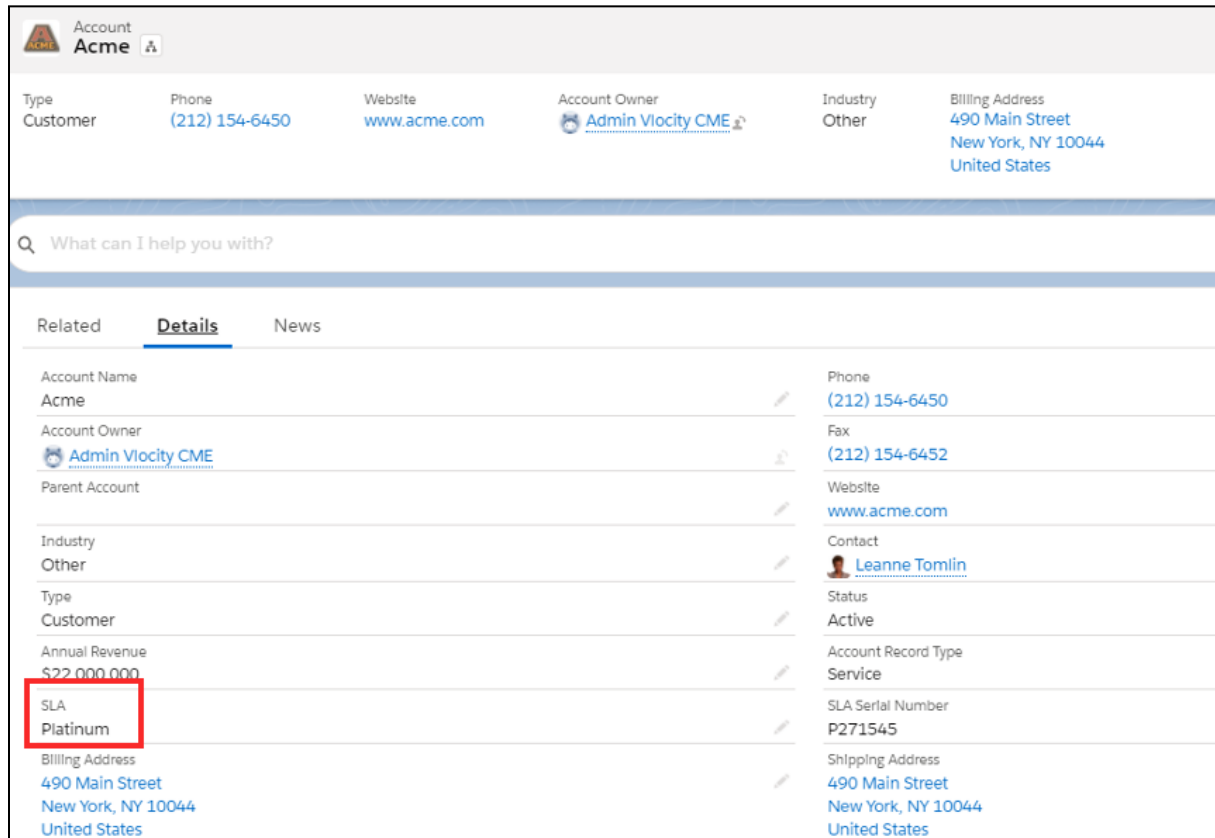
Task 3: Reprice an asset to reflect a context rule condition for an account

1. Review Blast! Pro Internet price list entries.
 - a. Navigate to the **Vlocity Product Console**.
 - b. Next to **Product** in the **Product Management** area, click the **search icon** .
 - c. In the **Search Product...** dialog box, enter **Blast!** and click the **search icon**.
 - d. Select the **Blast! Pro Internet** product and click on the **Pricing** facet.
 - e. Notice the pricing for both price list entries. The \$20 RM price list has a context rule attached to the price list entry giving Platinum account customers a reduced price on the product.



PRICE LIST	DISPLAY TEXT	PRICING VARIABLE	PRICING ELEMENT	TIME PLAN	TIME POLICY	CONTEXT RULES	EFFECTIVITY
B2B Price List	\$25 RM	Recurring Monthly Std Price	Name: \$25 RM Code: RM-25 Display Text: \$25 Calculation Type: Simple Charge: 25 USD Monthly				Active: Yes Start: Sep 17, 2018 End:
B2B Price List	\$20 RM	Recurring Monthly Std Price	Name: \$20 RM Code: RM-20 Display Text: \$20 Calculation Type: Simple Charge: 20 USD Monthly			Platinum Accounts Rule Set	Active: Yes Start: Sep 19, 2018 End:

2. Review Acme's Platinum SLA account status.
 - a. Navigate to the **Accounts** tab.
 - b. Change the **List Views** to **All Accounts**.
 - c. Select **Acme**.
 - d. Click on the **Details** tab.
 - e. Notice the **SLA** is set to **Platinum**.



The screenshot shows the Salesforce interface for an Account record named 'Acme'. At the top, there's a header with the account name and a search bar. Below this, a summary row displays key information: Type (Customer), Phone ((212) 154-6450), Website (www.acme.com), Account Owner (Admin Vlocity CME), Industry (Other), and Billing Address (490 Main Street, New York, NY 10044, United States). A search bar with the placeholder 'What can I help you with?' is located below the summary row. The main content area has tabs for 'Related', 'Details' (which is selected), and 'News'. Under the 'Details' tab, there's a table of account information. The 'SLA' row is highlighted with a red box, showing the value 'Platinum'. Other rows include Account Name, Account Owner, Parent Account, Industry, Type, Annual Revenue (\$22,000,000), Billing Address, Phone, Fax, Website, Contact (Leanne Tomlin), Status (Active), Account Record Type (Service), SLA Serial Number (P271545), and Shipping Address.

Related	Details	News
Account Name	Acme	Phone (212) 154-6450
Account Owner	Admin Vlocity CME	Fax (212) 154-6452
Parent Account		Website www.acme.com
Industry	Other	Contact Leanne Tomlin
Type	Customer	Status Active
Annual Revenue	\$22,000,000	Account Record Type Service
SLA	Platinum	SLA Serial Number P271545
Billing Address	490 Main Street New York, NY 10044 United States	Shipping Address 490 Main Street New York, NY 10044 United States

3. Review an existing asset that was discounted based on the account's SLA status.
 - a. Using the Lightning App Launcher, select **Assets** and open in a new tab.
 - b. Change the **List Views** to **All Assets**.
 - c. Select **Blast! Pro Internet** for **Acme**.

- d. On the **Details** tab notice the **Recurring Charge** for the asset is \$20 according to the context rule for platinum accounts.

 Asset

Blast! Pro Internet

Account	Contact	Quantity
 Acme		1.00

Related

Details

Asset Name

Blast! Pro Internet

Product

[Blast! Pro Internet](#)

Product Code

C-ISP-003

Serial Number

Install Date

Status

Quantity

1.00

Price

\$0

Billing Account

 Acme

Service Account

 Acme

Line Number

0001

One Time Calculated Price

\$0.00

One Time Discount Price

One Time Total

\$0.00

Recurring Calculated Price

\$20.00

Recurring Charge

\$20.00

Recurring Discount Price

One Time Manual Discount

0.00%

Recurring Manual Discount

0.00%

Recurring Total

\$20.00

4. Change Acme's SLA status to Gold.
 - a. Navigate back to the open browser tab for **Acme**.
 - b. Click on the **Edit** icon to change the **SLA** to **Gold**.
 - c. Click **Save**.
 - d. From the URL, copy the Account Id from after the Account/ and before /view and paste it to a text editor. You will need this Id for the Apex code for repricing accounts.

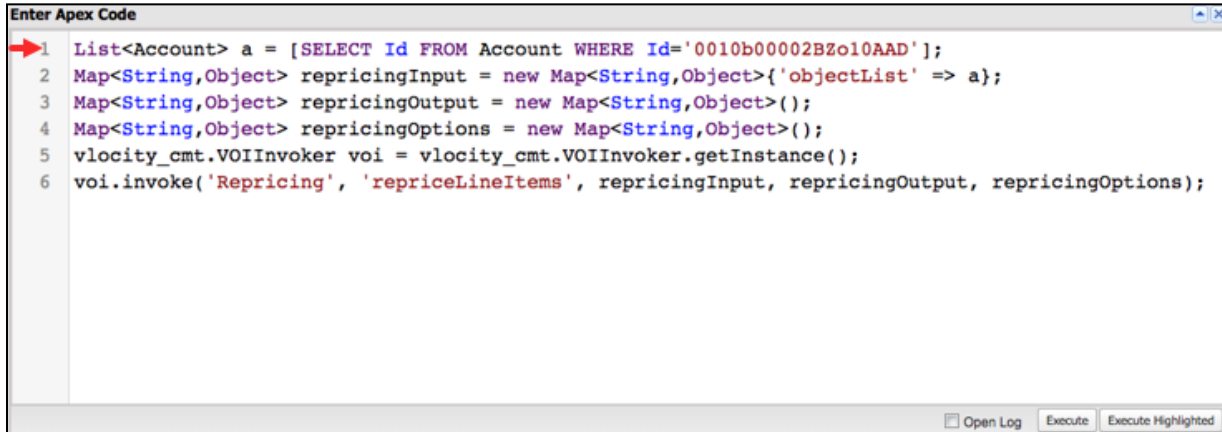
 Secure | <https://na57.lightning.force.com/lightning/r/Account/0010b00002BZo10AAD/view>

5. Run the repricing service on assets.
 - a. Navigate to the **Setup** icon and select **Developer Console**.
 - b. From the **Developer Console**, click **Debug > Open Execute Anonymous Window**.
 - c. The following Apex code will be used to reprice the Blast! Pro Internet asset on Acme's account. Copy the below Apex code and paste it in the **Enter Apex Code** box from the **Developer Console**.

Apex Code

```
List<Account> a = [SELECT Id FROM Account WHERE  
Id='00111I00000DBYUK'];  
Map<String,Object> repricingInput = new  
Map<String,Object>{'objectList' => a};  
Map<String,Object> repricingOutput = new  
Map<String,Object>();  
Map<String,Object> repricingOptions = new  
Map<String,Object>();  
vlocity_cmt.VOIInvoker voi =  
vlocity_cmt.VOIInvoker.getInstance();  
voi.invoke('Repricing', 'repriceLineItems',  
repricingInput, repricingOutput, repricingOptions);
```

- d. In line 1, replace the Account Id currently in the code with the Account Id copied from Acme's account.



```
1 List<Account> a = [SELECT Id FROM Account WHERE Id='0010b00002BZo10AAD'];
2 Map<String,Object> repricingInput = new Map<String,Object>{'objectList' => a};
3 Map<String,Object> repricingOutput = new Map<String,Object>();
4 Map<String,Object> repricingOptions = new Map<String,Object>();
5 vlocity_cmt.VOIInvoker voi = vlocity_cmt.VOIInvoker.getInstance();
6 voi.invoke('Repricing', 'repriceLineItems', repricingInput, repricingOutput, repricingOptions);
```

☐ Open Log Execute Execute Highlighted

- e. Select the code and click **Execute Highlighted**.
 - f. Verify the status is **Success** in the logs.
6. Review the repriced asset.
- a. Navigate back to the open Asset browser tab.
 - b. Refresh the browser page.

- c. Notice the **Recurring Charge** for the asset is now \$25 according to the price list entry for all customers other than Platinum.

 Asset

Blast! Pro Internet

Account	Contact	Quantity
 Acme		1.00

Related

Details

Asset Name

Blast! Pro Internet

Product

[Blast! Pro Internet](#)

Product Code

C-ISP-003

Serial Number

Install Date

Status

Quantity

1.00

Price

\$0

Billing Account

 Acme

Service Account

 Acme

Line Number

0001

One Time Calculated Price

\$0.00

One Time Discount Price

One Time Total

\$0.00

Recurring Calculated Price

\$25.00

Recurring Charge

\$25.00

Recurring Discount Price

Exercise 9-4: Create a Pricing Hook

Scenario

Management has come to Eliza with the requirement that they'd like the ability to price products in the Cart based on a customer's billing zip code. Eliza knows she can price products based on their attributes using attribute-based pricing but using the customer's billing zip code has her stumped. Eliza scratches her head wondering how this can be accomplished. In Eliza's research, she comes across the topic in software development called a hook. This is the answer to Eliza's dilemma. Using a hook, she can modify the pricing behavior in the cart without changing the underlying programming code. Eliza saddles up to the challenge and starts by mapping out the steps required to fulfill management's request.

Goal

- Explain what a hook is used for
- Describe how to implement a pricing hook
- Explain how to modify the Cart to display a custom field

Tasks

1. Review the billing zip code custom field
2. Create an attribute pricing matrix for billing zip code
3. Create an attribute pricing procedure expression set for billing zip code
4. Create a custom pricing plan step for billing zip code
5. Create a new Apex class
6. Create the CpqAppHandlerHookImplementation
7. Add a new column to the Cart
8. Add billing zip code as a custom field in the Cart
9. Test in the Cart

Time: 60 mins

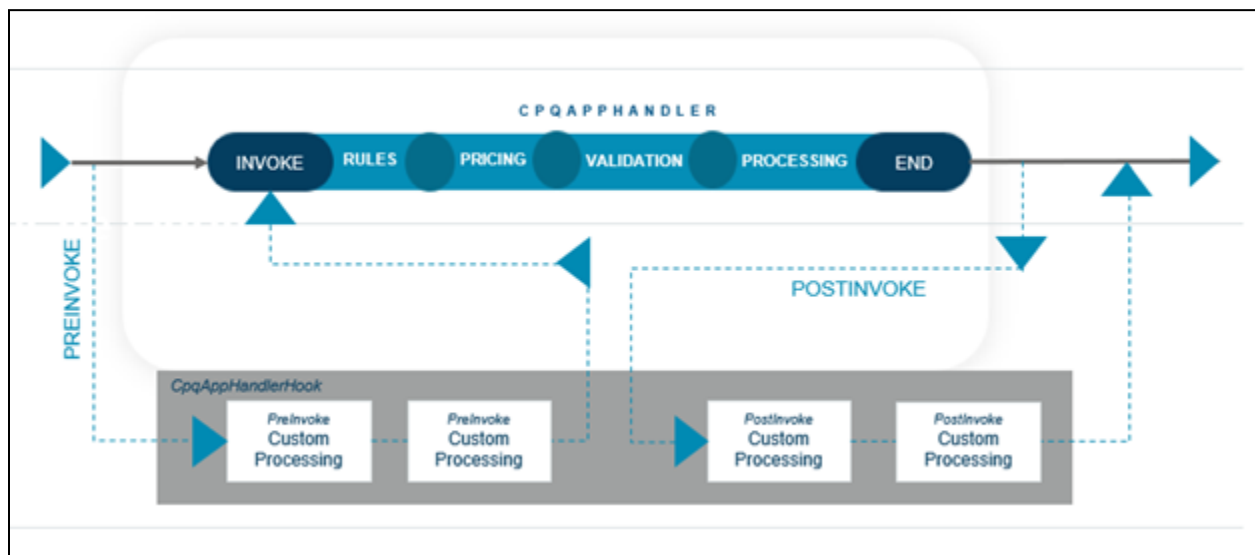
What is an Event Hook?

In software development, hooking is a concept that allows you to modify the original behavior of the application without changing your code. By intercepting the commands, you can change the action that would have been performed originally. It is very useful in the case of adding new functionality to applications, exposing additional fields in pricing calculations or altering how products are priced, to name a few of its applications.

How can I use an event hook in Industries CPQ?

Industries CPQ allows you to write custom Pre-invoke and Post-invoke hooks for each interface to implement custom logic. Pre-invoke hooks get executed before the actual interface is executed whereas Post-invoke hooks get executed after the actual interface is executed.

In Industries CPQ, CPQ functionality is managed by a global interface called `CpqAppHandler`. It includes a wide range of methods to perform CPQ processes in the Cart. (These methods are also available as a RESTful API, called the Cart-based APIs.) So, when you need to change the default behavior or processing of a CPQ operation, you create a custom implementation in `CpqAppHandler`'s hook interface, which is appropriately named `CpqAppHandlerHook`.



The `CpqAppHandlerHook` can be used on inputs and outputs of the CPQ methods to:

- Add, change, or delete input parameters.
- Add, change, or delete output responses.
- Debug issues by verifying the input and output are correct.
- In some cases, provide a workaround when waiting for a patch for an issue.

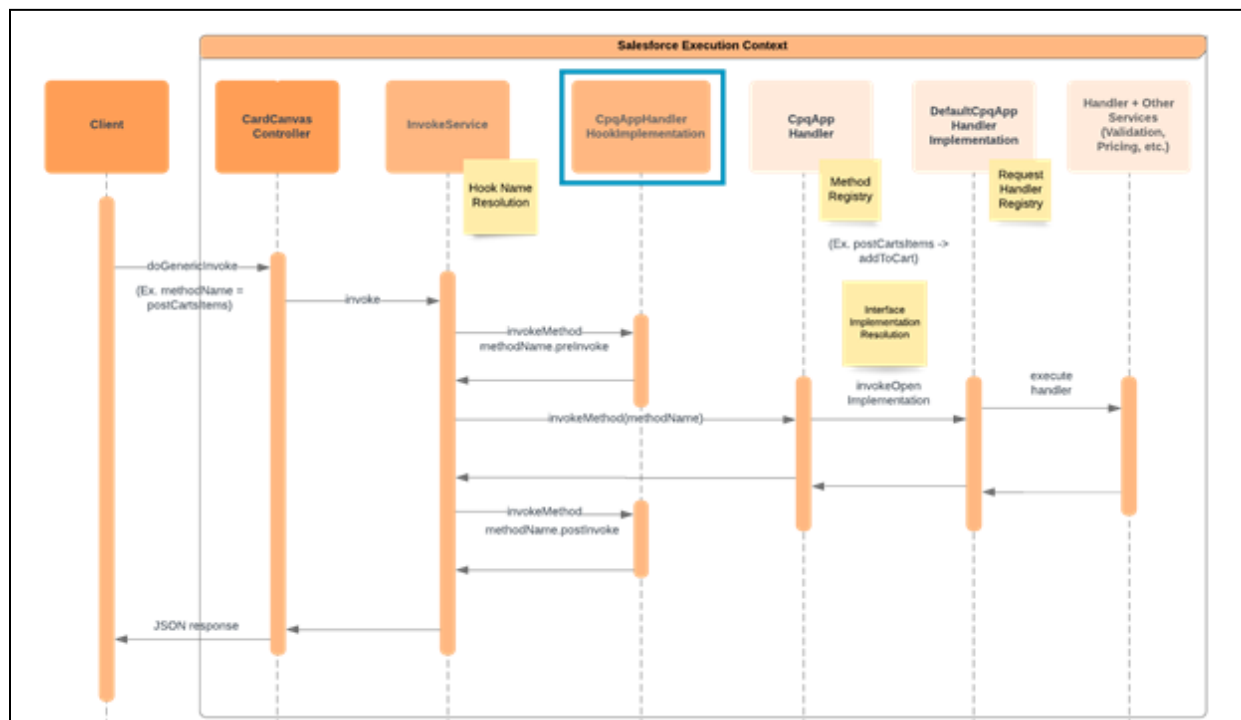
In addition, you can add business logic such as:

- Conditional processing.
- Preload information that will be used further down the execution chain.
- Process custom fields.
- Implement custom functionality.
- Do callouts.

How is the `CpqAppHandlerHook` invoked?

The `CpqAppHandlerHook` is automatically invoked when the `InvokeService` is called. The `InvokeService` simply appends a hook string, e.g., “Hook”, to the interface name and determines if there is an active interface with that name. If there is, the `InvokeService` will initiate the active implementation in the hook interface. Then, it will add a `.preInvoke` suffix to the method name and invoke the `PreInvoke` method in the hook implementation. A corresponding `.postInvoke` method is invoked on the hook implementation after the hooked class method has completed its processing.

The graphic below provides an example of the sequence when using a `CpqAppHandlerHook`.



What do I need to look out for when using a hook?

Injecting custom code is powerful when executed correctly, but you also need to be aware of the ramifications if executed poorly. Salesforce recommends the following when implementing a hook:

- Validate that a feature doesn't already exist that addresses your need for a hook.
- Ensure the custom code in the hook does not cause the execution to exceed any Apex governor limits.
- Put a try-catch around the hook code to catch any exceptions thrown by the custom code.
- When unit testing the hook code, it may be possible to mock the inputs expected by the hook code to minimize test setup and get better code coverage. Use `Test.isRunningTest()` to extract the mock input from the `inputMap` or from a static map.

Examples of using a hook

Let's look at an example based on the following business requirement:

Users must not be allowed to order the same product more than once in the same order. And, check to see if the product exists as an active asset on the account.

Solution using a hook:

- Implement a `CpqAppHandlerHook` to check before `postCartsItems` logic runs to see if the product exists in other active line items or active assets.
- In the `Preinvoke` hook, if the product exists in active line items or assets, null the items and create a new node that will be checked in the post-hook.
- In the `Postinvoke` hook, if the new node exists, display an error in the UI.
- When the user tries to add a duplicate product, they will get an error in the cart.

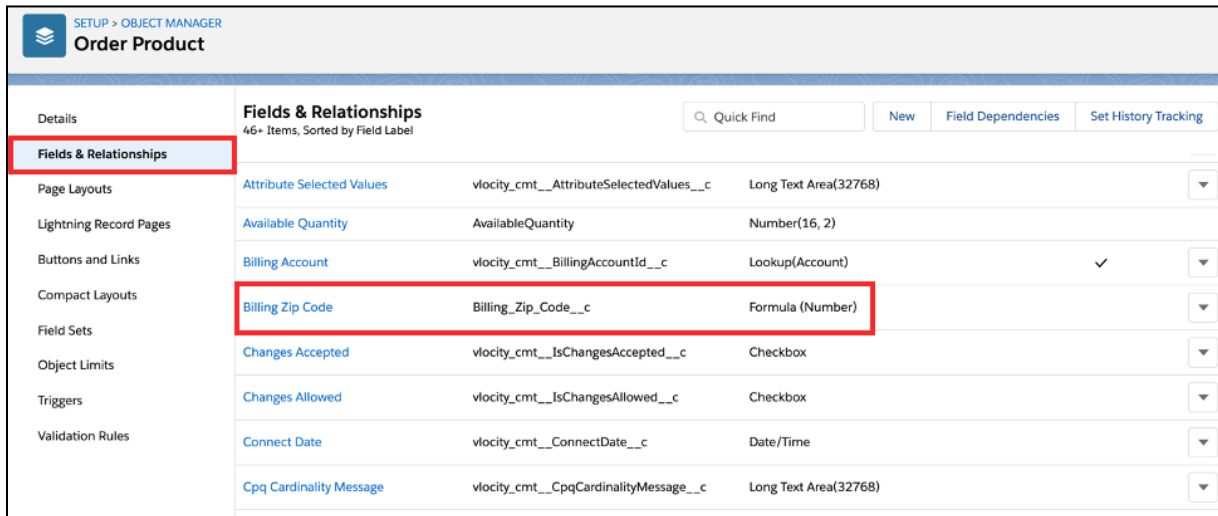
Let's take a look at another example. Consider how pricing works in the Cart. You add an iPhone X to the cart. It displays a one-time charge of \$1000. That's all good, but as a customer I want to know the total price of the iPhone X, which includes the sales tax cost. How can I display the cost of the iPhone X plus applicable taxes in the one-time charge column in the Cart? You would do this by implementing a pricing hook to alter how the pricing is calculated. Instead of calculating $\text{Quantity} \times \text{Price}$ and displaying the price in the One Time Charge column, you would create a pricing hook to calculate the One Time Charge as $\text{Quantity} \times \text{Price} \times \text{Sales Tax Rate}$. What if I didn't want to include the sales tax in the one-time charge column, but rather wanted to display it as a separate column? No problem, you can modify the cart to display a custom field for the applicable sales tax.

In this exercise, we will price products using the account's zip code. This would allow you to vary pricing for each line item based on the customer's location, such as the billing or shipping location. To do this, we must add a custom field to store the zip code on each order line item, and then use attribute-based pricing to calculate the price for each zip code. However, the pricing service has no idea that this new field exists. So, we also must use a pricing hook to alert the pricing service to the existence of this new field and ensure that it is passed in the input map to attribute-based pricing. Then, as the icing on the cake, we'll also expose the custom field (Billing Zip Code) in the Cart, so the user can see the values that affect the price of the line item.

Task 1: Review the billing zip code custom field

In your training playground we have created a custom field, Billing Zip Code, on the Order Product. The Billing Zip Code field will be used throughout this exercise, so we need to be familiar with its setup.

1. Go to **Setup** and click **Object Manager**.
2. In the **Quick Find** box, type `Order Product`.
3. Under Label, click **Order Product**.
4. Click **Fields & Relationships** from the navigation pane on the left.



The screenshot shows the Salesforce Object Manager interface for the 'Order Product' object. The 'Fields & Relationships' tab is selected, displaying a list of 46+ items sorted by field label. The 'Billing Zip Code' field is highlighted with a red box. The field details are as follows:

Field Label	Field Name	Field Type
Attribute Selected Values	velocity_cmt__AttributeSelectedValues__c	Long Text Area(32768)
Available Quantity	AvailableQuantity	Number(16, 2)
Billing Account	velocity_cmt__BillingAccountId__c	Lookup(Account) ✓
Billing Zip Code	Billing_Zip_Code__c	Formula (Number)
Changes Accepted	velocity_cmt__IsChangesAccepted__c	Checkbox
Changes Allowed	velocity_cmt__IsChangesAllowed__c	Checkbox
Connect Date	velocity_cmt__ConnectDate__c	Date/Time
Cpq Cardinality Message	velocity_cmt__CpqCardinalityMessage__c	Long Text Area(32768)

5. Click **Billing Zip Code**.
6. Make a notation of the **API name**, here _____.
7. Make a notation of the **formula**, here _____.
8. Close the Setup browser tab.

Task 2: Create an attribute pricing matrix for billing zip code

In this task, Eliza needs to create an attribute pricing matrix for all US billing zip codes.

1. Using the Lightning App Launcher in the upper left, search for and click **Files**.
2. Locate the **BillingZipCodeAttributePricingMatrix v2.csv** file.
3. Click the drop-down menu on the **BillingZipCodeAttributePricingMatrix v2** line and select **Download**.
4. Navigate to the **Lookup Tables** tab. You may need to reopen it using the Lightning App Launcher.
5. Click **New**.
6. Select **Decision Matrix** and click **Next**.
7. In the New Decision Matrix dialog box, type `BillingZipCodeAttributePricingMatrix` for the **Name**.
8. Select **Standard** for the **Type**.
9. Click **Save**.
10. On the **Related** tab, select **BillingZipCodeAttributePricingMatrix V1**.
11. Click **Upload CSV File**.
12. In the dialog box that appears, select **Upload Files**.
13. Select the downloaded .csv file and click **Open**.

14. Edit the Header Information with the following and click **Upload**:

Column Header	Data Type	Header Type
SourceProductName	Text	Input
SourceProductCode	Text	Input
CharacteristicName	Text	Input
CharacteristicValue	Text	Input
Billing_Zip_Code__c	Text	Input
MRC	Currency	Output
NRC	Currency	Output

15. Click **Done** and wait for the file upload to process. Click **Refresh**, if necessary.

7 records • Sorted by SourceProductName

[Upload CSV File](#)
[Generate CSV File](#)
[Add Column](#)
[Add Row](#)
[Delete Rows](#)
[Refresh](#)
[Save](#)

Input Data					Output Data		
☐	SourceProductName (Text) ↑	SourceProductCode (Text) ↓	CharacteristicName (Text) ↓	CharacteristicValue (Text) ↓	Billing_Zip_Code__c (Text) ↓	MRC (Currency) ↓	NRC (Currency) ↓
1	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	00000-29999	\$0	\$200
2	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	30000-39999	\$0	\$300
3	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	40000-58900	\$0	\$400
4	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	59000-59999	\$0	\$500
5	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	60000-69999	\$0	\$600
6	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	70000-79999	\$0	\$700
7	☐ BeatsX Wireless Earphones	C-ACC-002	Accessory Type	Audio	80000-99999	\$0	\$800

16. In the BillingZipCodeAttributePricing V1 header, click **Edit**.

17. Check the **Enabled** checkbox and select **Save**.

Edit BillingZipCodeAttributePricingMatrix V1

Information

Name

BillingZipCodeAttributePricingMatrix V1

Version Number

1

Matrix Type

Standard

Decision Matrix

BillingZipCodeAttributePricingMatrix

Load Process Status

Completed

Enabled

☒

Start Date Time

Date

7/4/2022

Time

8:56 AM

End Date Time

Date

Time

Rank

1

Cancel

Save & New

Save

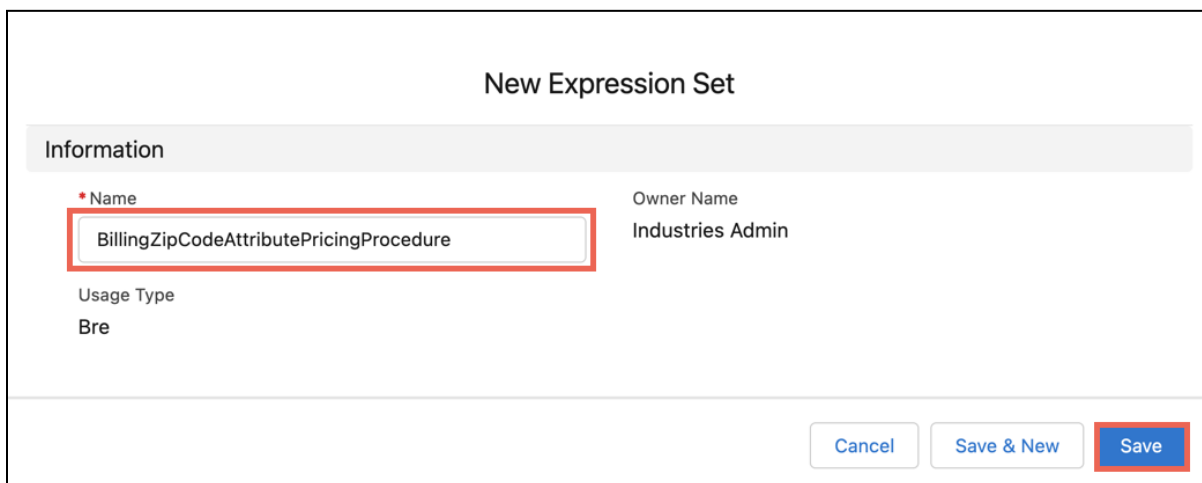
© Copyright 2022 Salesforce.com, inc. All rights reserved.

91

Task 3: Create an attribute pricing procedure expression set for billing zip code

In this task, Eliza needs to create an attribute pricing procedure so that product pricing is calculated based on the Billing Zip Code attribute pricing matrix.

1. Navigate to the **Expression Sets** tab. You may need to reopen it using the Lightning App Launcher.
2. Click **New**.
3. Type `BillingZipCodeAttributePricingProcedure` for the **Name**.
4. Click **Save**.



5. Click the **Related** tab, and select **Edit** from the dropdown arrow  next to **BillingZipCodeAttributePricingProcedure V1**.
6. In the **Rank** field, enter 1 and click **Save**.
7. Click **BillingZipCodeAttributePricingProcedure V1**.
8. Select **Open in Expression Set Builder** in the upper right.
9. In the **Expression Set Builder** window, click the **plus +** sign.
10. In the Add Element dialog box, select **Lookup Table**.

11. In the **Lookup Table Details** field, search for and select **BillingZipCodeAttributePricingMatrix**.
12. Click the **Element Details**  tab and select the **Include in Output** checkbox.
13. Select the **Resource Manager** tab  and click **Add Resource**.
14. Enter the following information to create the recurring monthly standard price variable and click **Done and New**:

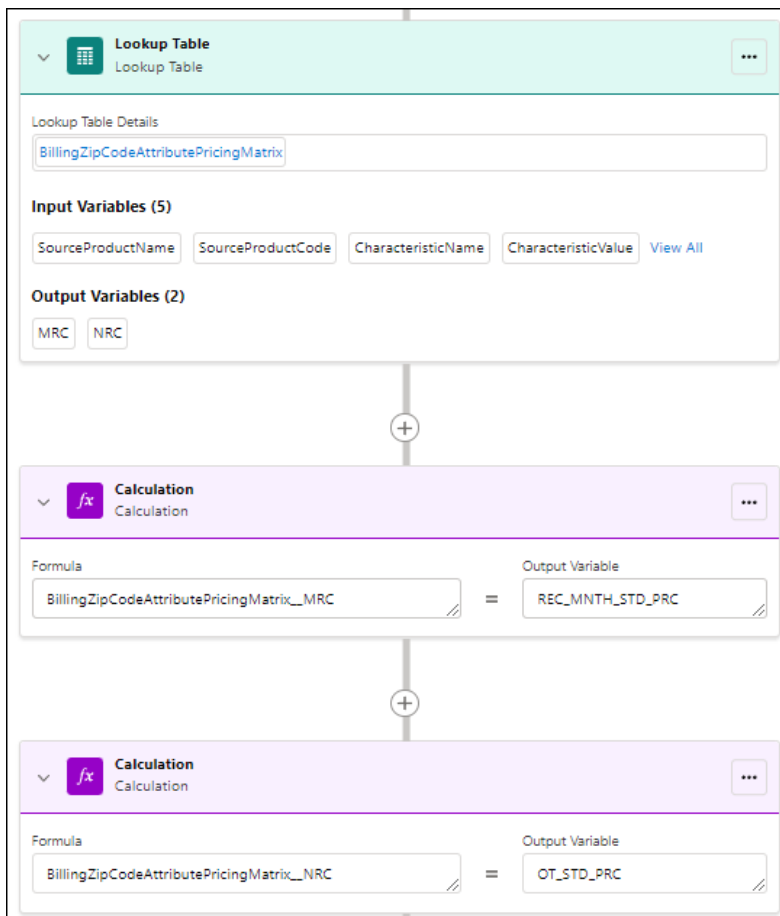
Field	Entry
Resource Type	Variable
Resource Name	REC_MNTH_STD_PRC
Data Type	Currency
Decimal	2

15. Enter the following information to create the one-time standard price variable and click **Done**:

Field	Entry
Resource Type	Variable
Resource Name	OT_STD_PRC
Data Type	Currency
Decimal	2

16. Click the **plus +** sign and select **Calculation** from the **Add Element** dialog.
17. In the **Formula** field, search for and select **BillingZipCodeAttributePricingMatrix__MRC**.
18. In the **Output Variable** field, search for and select **REC_MNTH_STD_PRC**.

19. With the Calculation element selected, click the **Element Details**  tab and select the **Include in Output** checkbox.
20. Click the **plus +** sign and select **Calculation** from the **Add Element** dialog.
21. In the **Formula** field, search for and select **BillingZipCodeAttributePricingMatrix__NRC**.
22. In the **Output Variable** field, search for and select **OT_STD_PRC**.
23. With the Calculation element selected, click the **Element Details**  tab and select the **Include in Output** checkbox.



The screenshot displays the Salesforce Advanced Pricing configuration interface. It features a vertical stack of elements connected by plus signs. The top element is a **Lookup Table** (teal header) with the name **BillingZipCodeAttributePricingMatrix**. Below it, the **Input Variables (5)** section lists **SourceProductName**, **SourceProductCode**, **CharacteristicName**, and **CharacteristicValue**, with a **View All** link. The **Output Variables (2)** section lists **MRC** and **NRC**. The second element is a **Calculation** (purple header) with the formula **BillingZipCodeAttributePricingMatrix__MRC** and the output variable **REC_MNTH_STD_PRC**. The third element is another **Calculation** (purple header) with the formula **BillingZipCodeAttributePricingMatrix__NRC** and the output variable **OT_STD_PRC**.

24. Click **Save** and then select **Activate** in the top menu.

Task 4: Create a custom pricing plan step for billing zip code

In this task, Eliza needs to add a pricing plan step to the Default Pricing Plan so that products in the Cart are priced according to the Billing Zip Code Attribute Pricing Procedure and Matrix.

1. Navigate to the **Vlocity Product Console**.
2. Next to **Pricing Plan** in the **Pricing** area, click the **search icon**  .
3. In the **Search Pricing Plan...** dialog box, press the return or enter key to display the Default Pricing Plan.
4. Click on the **Default Pricing Plan**.
5. Click on the **Pricing Plan Steps** facet.
6. Click **New Item**.
7. Enter the following data in the **General Properties** pane:

Field	Entry
Name	Calculate Billing Zip Code Attribute Pricing
Implementation Name	CustomPricingPlanStepImpl
Method Name	GetMatrixPrice
Sequence	9
Active	✓

8. Add the following parameters to the step:

First Box	Second Box
ProcedureName	BillingZipCodeAttributePricingProcedure
MatrixName	BillingZipCodeAttributePricingMatrix
RangeFields	Billing_Zip_Code__c
DecisionMatrix	true
IncludeAttrInfoInRangeKeys	false

9. Select **Done** and then select **Save**.
10. Run the **Clear Managed Platform Cache** and **Refresh Platform Cache** maintenance jobs in CMT Administration.

Task 5: Create a new Apex class

In this task, Eliza needs to create a new Apex class to inject logic for the new custom field – Billing Zip Code. Without this logic the new custom field will not populate.

1. Using the Lightning App Launcher in the upper left, click **Files**.
2. Locate the **CpqAppHandlerHookImplementation** file by navigating to **Libraries -> Advanced CPQ -> Advanced Pricing**.



NOTE:

The `CpqAppHandlerHookImplementation` is an Apex Class that has been written for this lab exercise.

3. Click the drop-down menu on the **CpqAppHandlerHookImplementation** line and select **Download**.
4. Navigate to the **Setup** icon and select **Developer Console**.
5. From the **Developer Console**, click **File > New > Apex Class**.
6. Name the Apex Class `CpqAppHandlerHookImplementation` and click **OK**.
7. Open the downloaded file and copy the Apex Class code.
8. Paste the code in the **Enter Apex Code** box. You will have 48 lines of code.

```
File Edit Debug Test Workspace Help
CpqAppHandlerHookImplementation.apex
Code Coverage: None API Version: 46 Go To

1 global with sharing class CpqAppHandlerHookImplementation implements vlocity_cmt.VelocityOpenInterface
2 {
3     global Boolean invokeMethod(String methodName, Map<String, Object> input, Map<String, Object> output,
4     {
5         try
6         {
7             System.debug('___ CpqAppHandlerHookImplementation ' + methodName);
8
9             if (methodName == 'postCartsItems.PreInvoke' || methodName == 'putCartsItems.PreInvoke' ||
10                methodName == 'deleteCartsItems.PreInvoke' || methodName == 'getCartsItems.PreInvoke')
11             {
12                 String customFields = 'Billing_Zip_Code__c'; // comma separated list of custom fields
13                 // if fields is passed in, append the custom fields to the parameter value
14                 if (String.isNotBlank((String)input.get('fields')))
15                 {
16                     input.put('fields', input.get('fields') + ',' + customFields);
17                 }
18                 else
19                 {
20                     // if the fields is not passed in, initialize to default price detail fields and inclu
21                     input.put('fields', 'vlocity_cmt__BillingAccountId__c,vlocity_cmt__ServiceAccountId__c
22                     'vlocity_cmt__OneTimeTotal__c,vlocity_cmt__OneTimeManualDiscount__c,vlocity_cmt__R
23                     'vlocity_cmt__ProvisioningStatus__c,vlocity_cmt__RecurringCharge__c,vlocity_cmt__C
24                     'vlocity_cmt__ParentItemId__c,vlocity_cmt__BillingAccountId__r.Name,vlocity_cmt__S
25                     'vlocity_cmt__PremisesId__r.Name,vlocity_cmt__InCartQuantityMap__c,vlocity_cmt__Ef
26                 }
27                 // if priceDetailsFields is passed in, append the custom fields to the parameter value
28     }
```

9. Click **File > Save**.
10. Review line 12. Note the String customFields API here, _____.
11. Close the Developer Console.

Task 6: Create the CpqAppHandlerHookImplementation

In this task, Eliza needs to create the interface implementation hook. The hook interface provides a hook to the implementation method for pre-invoke and post-invoke handling.

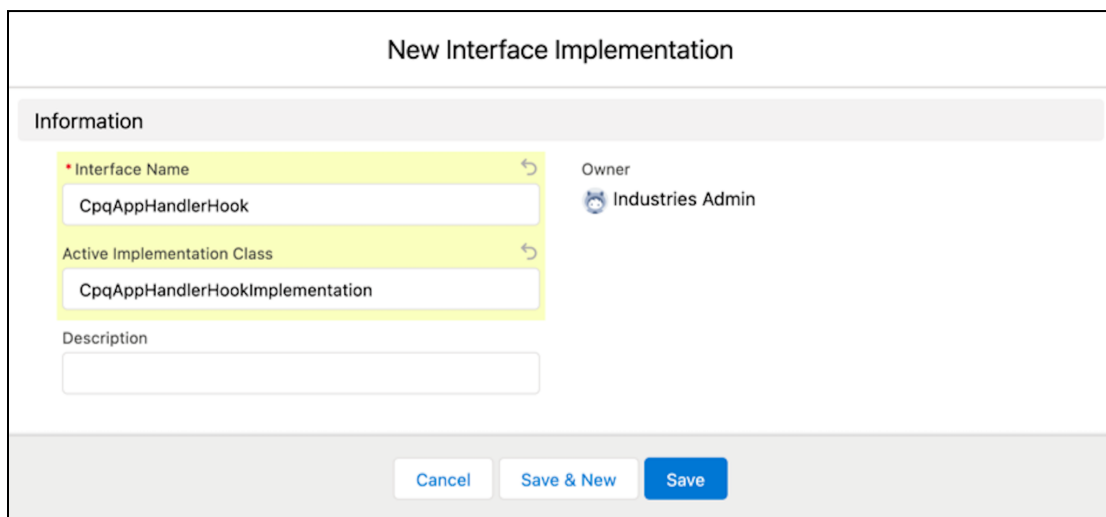
1. Navigate to the **Interface Implementations** tab in the Configure Price Quote (CPQ) app.
2. Click **New**.
3. In the **Interface Name** field, type `CpqAppHandlerHook`.



NOTE:

The maximum length for a hook interface name is 36 characters.

4. In the Active Implementation Class field, type `CpqAppHandlerHookImplementation`.



5. Click **Save**.
6. From the Related tab, click **New** next to **Interface Implementation Detail (0)**.
7. In the New Interface Implementation Detail window, type `CpqAppHandlerHookImplementation` in the **Available Implementation** field.

8. Check the **Active** and **Default** box and click **Save**.

 Interface Implementation
CpqAppHandlerHook

Related

Details

 **Interface Implementation Detail (1)**

New

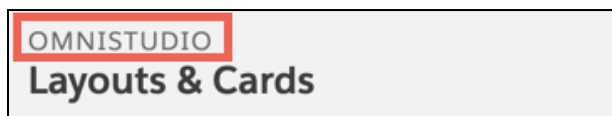
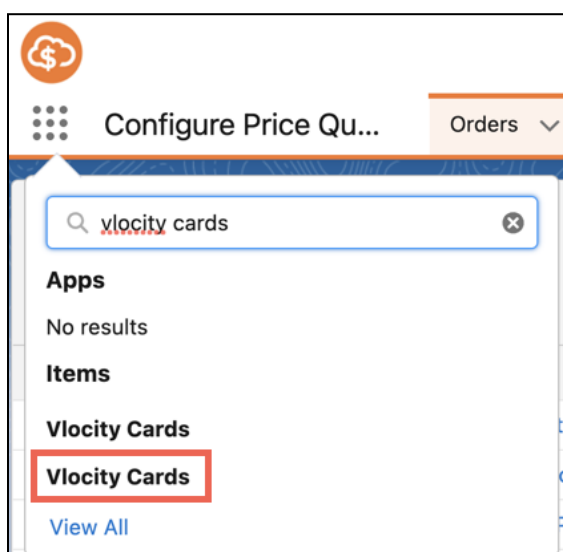
Available Implementation	Active	Default	
CpqAppHandlerHookImplementation	☒	☒	

[View All](#)

Task 7: Add a new column to the Cart

In this task, Eliza needs to modify the default CPQ product cart so that the new field – Billing Zip Code – displays as a column in the Cart.

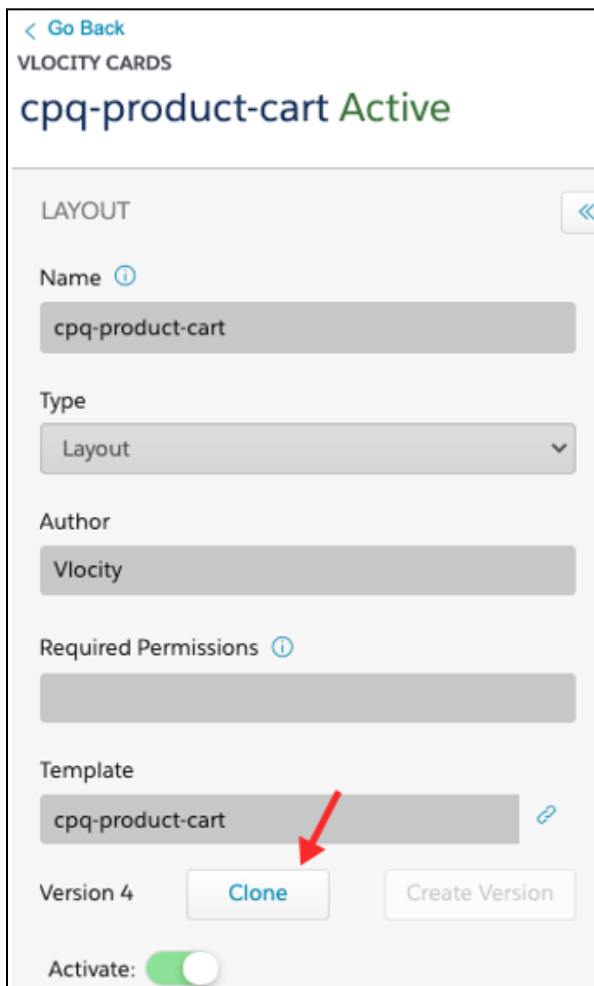
1. Using the Lightning App Launcher in the upper left, search for the **Vlocity Cards** app. Two options appear. Choose the second Vlocity Cards app to open the Omnistudio Layouts & Cards app. Verify you have opened the correct app by checking to see if Omnistudio is listed in the header.



2. In the **Find in page** field, enter `cpq-product`.
3. Expand **cpq-product-cart** and select **Version 4**.

OMNISTUDIO Layouts & Cards						
As of - Viewing as Industries Admin						
☐	LAYOUT NAME	TYPE	LWC ENABLED	AUTHOR	LAST MODIFIED DATE	LAST MODIFIED BY ID
☒	cpq-product-cart	Layout			6/29/22 11:26 PM	Industries Admin
☒	cpq-product-cart (Version 4)	Layout		Vlocity	6/29/22 11:26 PM	Industries Admin
☐	cpq-product-cart (Version 3)	Layout		Vlocity	6/29/22 11:26 PM	Industries Admin

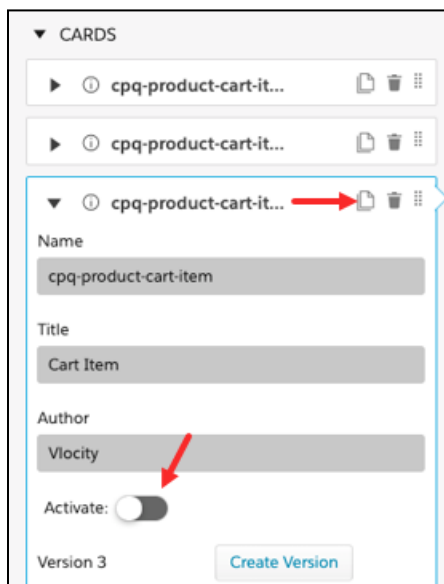
4. In the **LAYOUT** pane, scroll down to the **DATA SOURCE** section. Notice the selection is **Dual**. This means that the API query will be performed using Apex when running on a desktop, while a REST call will be performed while running on a mobile.
5. Notice the **Remote Class**, **Remote Method** and **fields** under the Desktop ApexRemote Config section. The CPQ API CpqAppHandler.getCartItems returns all products currently in the cart. The fields parameter controls which fields are returned in the API for each product.
6. Scroll back to the top of the **LAYOUT** pane and click **Clone**.



The screenshot shows the 'cpq-product-cart' layout configuration page. At the top, there is a 'Go Back' link and the text 'VLOCITY CARDS cpq-product-cart Active'. Below this is a 'LAYOUT' section with a double arrow icon. The fields are: Name (cpq-product-cart), Type (Layout), Author (Vlocity), Required Permissions (empty), Template (cpq-product-cart), Version 4, and an Activate toggle (checked). A red arrow points to the 'Clone' button, which is next to the 'Create Version' button.

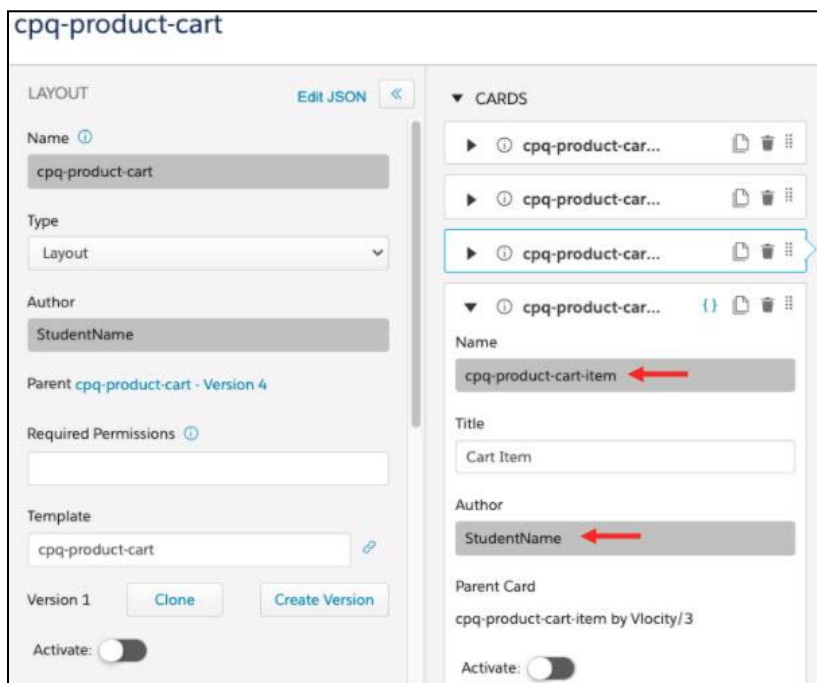
7. In the **CLONE LAYOUT** window, change the **Layout Author** to your name. Do NOT change the Layout Name or Layout Type.

8. Click **Clone**.
9. After cloning the card layout, it should contain an active card called **cpq-product-cart-item**.
10. Open the active **cpq-product-cart-item** card. You can ignore any inactive versions of this card that may also display.
11. Click the **Activate** button to deactivate it.
12. Click the **Clone Card** icon.



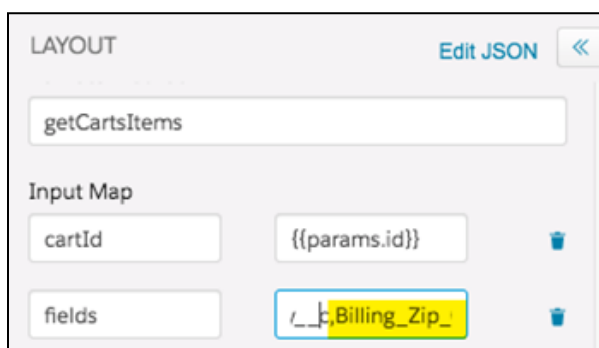
13. In the **CLONE CARD** window, type `cpq-product-cart-item` as the **Card Name**.
14. Type `your` name for the Card Author.

15. Click **Clone**.

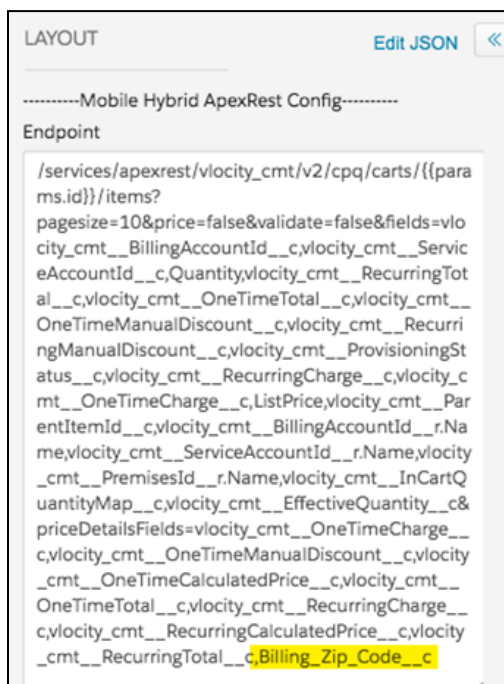


16. On the **LAYOUT** pane, navigate to the **fields** parameter from the Input Map. This is a comma-separated list of product fields that will be queried as part of the API call to `getCartItems()`. We need to append this list of fields to include the Billing Zip Code custom field.

17. In the box next to **fields**, scroll to the end of the list to enter the following: `,Billing_Zip_Code__c`. You must precede the custom field name with a comma.



18. On the **LAYOUT** pane, navigate to the **Mobile Hybrid ApexRest Config Endpoint** field, expand the field, and type the following: `,Billing_Zip_Code__c`. Remember, you must precede the custom field name with a comma.



19. Remember, the **DATA SOURCE** field is **Dual**. If you want the new column to be added to mobile versions of the CPQ cart you need to add the field to the Endpoint under Mobile Hybrid ApexRest Config.
20. Make sure you have the correct **cpq-product-cart-item** card selected. It will be the one with your name as the Author.
21. If you have enabled Loyalty Pricing, select **CustomView_CPQAdvancedPricingLoyalty** in the **STATES** pane. Otherwise, select **CustomView_CPQAdvancedPricing**.
22. In the blank **Fields** box, select **<<Custom Field>>** and click **Add**.

23. Delete <<Custom Field>> and replace it with the following:

Name	Label
<code>['Billing_Zip_Code__c']['value']</code>	<code>['Billing_Zip_Code__c']['label']</code>



ALERT:

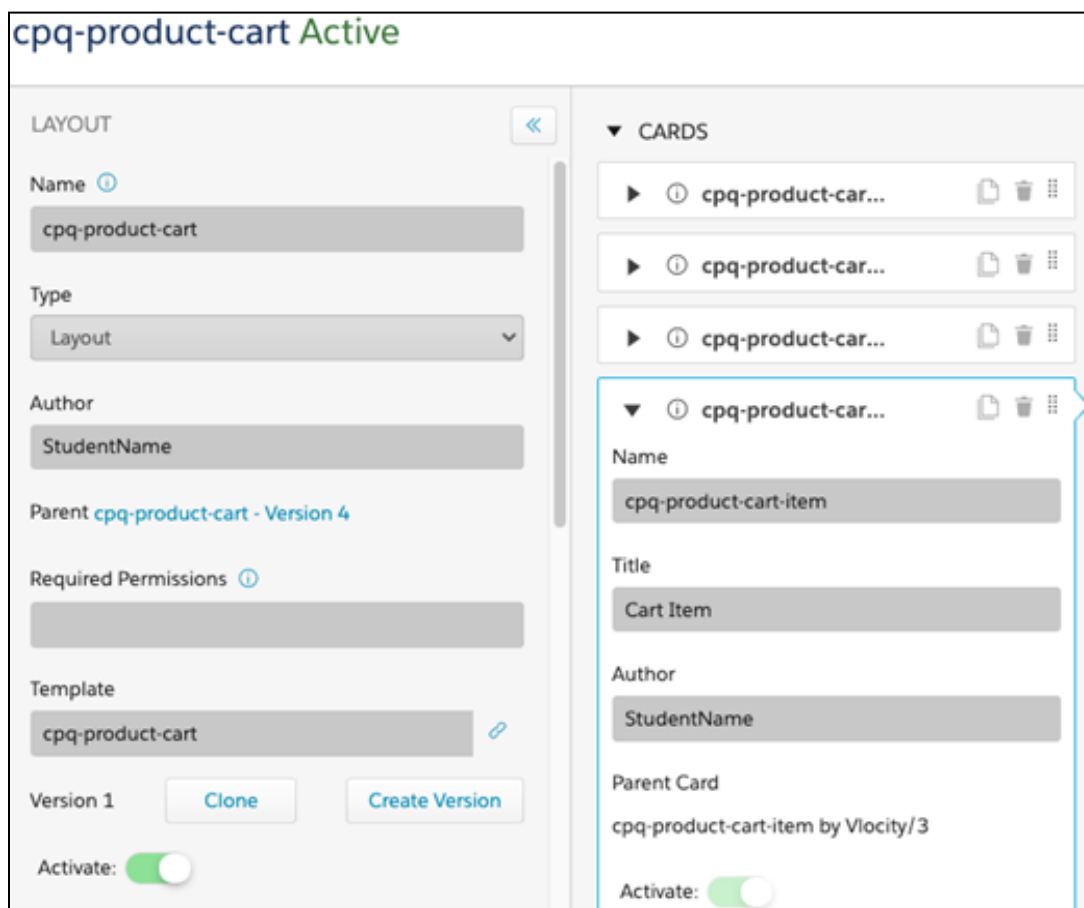
Do not perform a copy/paste from the exercise guide. The Name and Label field input must be entered exactly as displayed using straight quotes and no spaces between brackets.

STATES PREVIEW

Fields

Name	string	Label	+
<code>['Quantity']['value']</code>	input	<code>['Quantity']['label']</code>	+
<code>['vlocity_cmt__RecurringCharge__c']['value']</code>	currency	<code>['vlocity_cmt__RecurringCharge__c']['label']</code>	+
<code>['vlocity_cmt__RecurringTotal__c']['value']</code>	currency	<code>['vlocity_cmt__RecurringTotal__c']['label']</code>	+
<code>['vlocity_cmt__OneTimeCharge__c']['value']</code>	currency	<code>['vlocity_cmt__OneTimeCharge__c']['label']</code>	+
<code>['vlocity_cmt__OneTimeTotal__c']['value']</code>	currency	<code>['vlocity_cmt__OneTimeTotal__c']['label']</code>	+
<code>['vlocity_cmt__OneTimeLoyaltyPrice__c']['value']</code>	loyalty	<code>['vlocity_cmt__OneTimeLoyaltyPrice__c']['label']</code>	+
<code>['vlocity_cmt__OneTimeLoyaltyTotal__c']['value']</code>	loyalty	<code>['vlocity_cmt__OneTimeLoyaltyTotal__c']['label']</code>	+
<code>['vlocity_cmt__Action__c']['value']</code>	string	<code>['vlocity_cmt__Action__c']['label']</code>	+
promotions	promo	CPQPromotions	+
<code>['Billing_Zip_Code__c']['value']</code>	string	<code>['Billing_Zip_Code__c']['label']</code>	+

24. Click the **Activate** button on the **CARDS** and **LAYOUT** pane.



The screenshot shows the Salesforce CPQ configuration interface for a product named "cpq-product-cart". The interface is divided into two main panes: "LAYOUT" on the left and "CARDS" on the right. The "LAYOUT" pane contains fields for Name, Type, Author, Parent, Required Permissions, Template, and Version, along with an "Activate" toggle switch. The "CARDS" pane shows a list of cards, with one card selected and its details displayed in a sub-pane. The selected card is named "cpq-product-cart-item" and has a title of "Cart Item". The "Activate" toggle switch for the selected card is also visible.

cpq-product-cart Active

LAYOUT

Name: cpq-product-cart

Type: Layout

Author: StudentName

Parent: cpq-product-cart - Version 4

Required Permissions:

Template: cpq-product-cart

Version 1: Clone Create Version

Activate: ☒

CARDS

cpq-product-car... cpq-product-car... cpq-product-car...

cpq-product-car... (Selected)

Name: cpq-product-cart-item

Title: Cart Item

Author: StudentName

Parent Card: cpq-product-cart-item by Vlocity/3

Activate: ☒

Task 8: Add billing zip code as a custom field in the Cart

In this task, Eliza needs to create a Field Setting so that `getCartsItems` (called when the cart is loaded) and `postCartsItems` (called when you click Add to Cart) APIs return the Billing Zip Code field and it displays in the Cart.

1. Go to **Setup**.
2. In the **Quick Find** box, type `Custom Settings`.
3. Click **Custom Settings**.
4. Click **Manage** next to **Field Settings**.

SETUP Custom Settings									
Custom List									
Manage		Custom Object Map	Public	List	velocity_cmt		580	33	1917
Manage		DataRaptor Encryption	Public	Hierarchy	velocity_cmt		110	0	0
Manage		DataRaptor Workers	Public	List	velocity_cmt	Each worker represents a possible Apex Batch Job that can be created at run time by the Velocity DataRaptor service. Use this to control the number of DataRaptor workers processing batches from the batch queue simultaneously.	1,150	2	2306
Manage		DocuSign Anchor Tab Setting	Public	List	velocity_cmt		610	1	610
Manage		DocuSign Callout Configuration Setup	Public	List	velocity_cmt		355	0	0
Manage		DocuSign Envelope Object Setting	Public	List	velocity_cmt		420	4	1680
Manage		Field Settings	Public	List	velocity_cmt		903	0	0
Manage		FieldsConfiguration	Public	List	velocity_cmt	Used for defining the configuration for the fields in UI	1,130	0	0
Manage		FieldsLookupConfiguration	Public	List	velocity_cmt		865	0	0
Manage		General Settings	Public	List	velocity_cmt		355	1	355

5. Click **New**.

6. Enter the following for the new Field Setting:

Field	Value	Notes
Name	Billing_Zip_Code__c	
Feature	CPQV2	Links to the CPQ APIs
Field Name	Billing_Zip_Code__c	API name of the field
Inclusion	✓	
Object Name	OrderItem	API name of the object
Sequence	1	

Field Settings Edit

[Help for this Page](#)

Provide values for the fields you created. This data is cached with the application.

Edit Field SettingsSaveSave & NewCancel

Field Settings Information! = Required Information

Name	Billing_Zip_Code__c
Feature	CPQV2
Field Name	Billing_Zip_Code__c
Inclusion	☒
IsEditable	☐
Object Name	OrderItem
Sequence	1

7. Click **Save**.

8. Close the Setup browser tab.

Task 9: Test in the Cart

1. Navigate to the **Orders** tab and click **New**.
2. On the **New Order** dialog, enter the following:

Field	Value
Order Name	Adv Pricing Hooks Order
Account Name	Acme
Order Start Date	[Today's date]
Price List	B2B Price List

3. Click **Save**.
4. Click on the Power Launcher's search field and select **Configure Order**.
5. Add the **BeatsX Wireless Earphones** to the cart.
6. In the **One Time Charge** column, click on the **charge** and select the **Price Details**  icon.
7. Notice that the price returned is from the BillingZipCodeAttributePricingProcedure based on the data entered in the matrix.

Details	
Priced from BillingZipCodeAttributePricingProcedure	\$200.00
	\$200.00

8. Click **Close**.
9. Navigate to the **Accounts** tab and **open link in new tab**.
10. Change the **List Views** to **All Accounts**.

11. Click **Acme**.
12. Click on the **Details** tab.
13. Note the Billing Address zip code, here _____.
14. Close the **Accounts** tab.
15. Notice the new Billing Zip Code column in the cart and the billing zip code returned for Acme.

AdvancedPricing + Loyalty ▼										
	Total Quantity	Recurring Charge	Recurring Total	One Time Charge	One Time Total	One Time Loyalty Price	One Time Loyalty Total	Action	Promotions	Billing Zip Code
BeatsX Wirele...	1	\$0.00	\$0.00	\$200.00	\$200.00	0	0	Add		10044 ▼



ALERT:

The custom field value can't be null. If null, it will not be read and will not display.

Troubleshooting Guide

Attribute-Based Pricing

Is the BeatsX Wireless Earphones product not displaying the correct price in the cart based on the BillingZipCodeAttributePricingMatrix? Use the troubleshooting headings below to assist in your evaluation.

BillingZipCodeAttributePricingMatrix (Lookup Table)

- Verify that all matrix headers are spelled correctly.
- Verify that all input/output data in the matrix has proper spacing, punctuation, and correct spelling.
- Verify that the matrix is enabled and has the highest rank (1).

BillingZipCodeAttributePricingProcedure (Expression Set)

- Verify that the procedure is working using the Simulate button on the Vlocity Procedure page.
- Verify all variables has proper spacing, punctuation and correct spelling
- Verify the calculation steps:
 - The Decision Matrix element is pointing to the BillingZipCodeAttributePricingMatrix.
 - The Calculation elements are pointing to the MRC/NRC BillingZipCodeAttributePricingMatrix outputs.
- Include in Output is checked for each element.
- The expression set is enabled.

Custom Billing Zip Code Pricing Plan Step

- Verify all data input is spelled correctly and has proper spacing
- Step is active

Modifying the Cart

Is the Billing Zip Code column not displaying in the cart? Use the troubleshooting headings below to assist in your evaluation.

Billing Zip Code Field Setting

- The API names are spelled correctly with proper punctuation.
- The correct CPQ API is linked.

Card Template

- Fields Input Map includes the Billing Zip Code custom field and is preceded with a comma.
- Correct custom view State is selected on the card.
 - Loyalty Pricing enabled? Use the CustomView_CPQAdvancedPricingLoyalty state.
 - Loyalty Pricing not enabled? Use the CustomView_CPQAdvancedPricing state.
- The layout and card are both activated.
- Have you verified all the above and the column still isn't displaying? Try clearing your browser cache.

Exercise 9-5: Create Usage Pricing

Scenario

Infiwave has decided to get into the business of providing energy to their B2B customers. One of the big differences in quoting and selling energy products is that customers need to see estimated prices for commodity products included in their quotes and orders. So, before these B2B customers can purchase energy products, Eliza needs to enable the Usage Pricing feature and configure its related components so that the estimated usage consumption and price is calculated accurately and displayed in the Cart.

Goal

- Explain the concept of usage pricing
- Define how to configure usage pricing for products
- Use usage pricing in the Cart

Tasks

1. Enable usage pricing
2. Create charge measurements and assign usage pricing to products
3. Configure an attribute and an attribute binding for estimated usage quantity
4. Test usage pricing in the Cart
5. Configure attribute-based pricing for usage pricing
6. Edit an order to use attribute-based pricing

Time: 45 mins

What is Usage Pricing?

Usage pricing is a type of pricing used when a customer is charged based on their usage (consumption) of a product or service, such as water, gas or electricity. Typically, at the end of a period (often, a billing cycle), a service provider collects information about actual service consumption and calculates total charges for that period. Unlike fixed charges, which are predefined, usage-based charges fluctuate from period to period. Then, the customer is charged for their actual usage by the billing system.

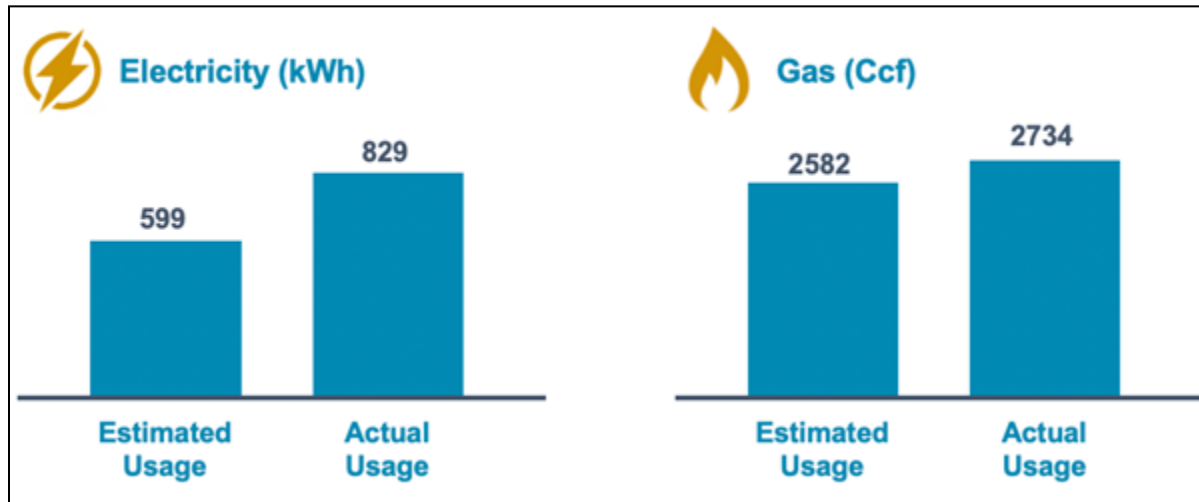
This makes sense when you're talking about services that are installed and already sold to a customer. Most of us have experienced this type of pricing as home or business owners. But things get more complicated when you think about how to price and sell these services *before* they're installed—because the usage or consumption hasn't happened yet! So, during quoting and order capture, you need a way to estimate usage, so you have a defined price that you can share and discuss with your customer.

In Industries CPQ, usage pricing provides the ability to price a product or service during quoting and order capture based on an *estimate* of usage.

It's important to understand that the customer may not be charged the estimated usage price that displays in the quote or order. Instead, that information can be shared with the customer to help them understand the price they may pay in the future, or it can be used in contracts to lock in a rate for a set period of time. (Note that administering and billing *actual* usage is handled by an external back office system.)

Some of the most typical use cases for usage pricing are:

- Long-distance call charges depending on the destination country and call duration.
- Internet data overage charges when a service provider charges for each gigabyte over a certain threshold.
- Cloud backup service proportional to the total volume of backups.
- Cloud computing charges based on the total time of resources being used.
- Utility services, including commodities like electricity, gas, or water.



Eliza has been asked to create pricing for a fixed rate electricity product. Just like it sounds, this product provides customers with a fixed rate for electricity for the length of their plan, even if electricity prices fluctuate in the market. Sounds great, right? The tricky part is that the price must be based on past usage or consumption data for their location, and it must include a component for electricity as a commodity and also a standing charge for service to the house or location.

So, Eliza creates a new Fixed Energy product bundle that contains two child products, Unit Rate and Standing Charges. The Unit Rate product represents the rate that the customer will pay for electricity based on their estimated usage, and the Standing Charges represent the standard fee to supply the location (or service point) with electricity service.

At the completion of this exercise, you'll be able to order the Fixed Energy product in the Cart with a calculated usage price based on an estimate as shown below. The calculation will multiply the Estimated Usage Quantity by the Usage Price for each product to get a Usage Price Total amount, which then will be rolled up the product hierarchy to get a total for the product bundle. We'll also implement attribute-based pricing to make the Usage Price for each child product variable based on Distribution Region and Payment Method using attribute-based pricing.


Energy Usage Pricing Order
READY TO SUBMIT

Price List
 B2B Price List

[View Record](#)
[Generate Document PDF](#)
[New](#)

CART
 PROMOTIONS
 DISCOUNTS (0)

Usage Pricing

	Quantity	Usage Price		Usage Price Total	Recurring Charge	Recurring Total	One Time Charge	One Time Total
Fixed Energy	1	\$0.00		\$273.60	\$0.00	\$0.00	\$0.00	\$0.00
Unit Rate	1	\$0.19673	kWh	\$206.57	\$0.00	\$0.00	\$0.00	\$0.00
Standing Charges	1	\$0.13405	day	\$67.03	\$0.00	\$0.00	\$0.00	\$0.00

Unit Rate: (1) Estimated Usage Quantity * (2) Usage Price = (3) Usage Price Total
 1050 * 0.19673 = \$206.57

Commodity Cost Components

Distribution Region
 B

Commodity Commercial Offers
 Direct Debit

Usage Pricing
 Estimated Usage Quantity
 1050

Attribute-based pricing



ALERT:

Several elements, such as object types, attribute categories, attributes, picklists, and products, have been pre-configured in your training playground to expedite the usage pricing exercise. In a sandbox or production environment, the setup of these elements would have been done in conjunction with the decision to enable usage pricing.

Task 1: Enable usage pricing

Eliza's first task at hand is enabling the usage pricing feature so that B2B customers can purchase energy products. Under the hood, there are a number of activities occurring to ensure the feature is set up correctly, so you don't have to. Once enabled, usage pricing variables and usage pricing variable bindings (we'll learn about bindings later) are created. In addition, usage pricing fields are available in the Cart and are used for calculations based on the usage pricing variables defined. You'll see all this in action when we create a usage pricing order later in this exercise.

1. Navigate to the **Vlocity CMT Administration** tab. You may need to click the **More** dropdown menu to expand for additional tabs.
2. Under **Custom Settings**, click **Enable Features**.
3. Next to **Usage Pricing**, click the button currently set to Disabled.
4. The **Enable Usage Pricing** dialog window opens, click **Enable**.
5. Click **OK** to the message that the feature has been enabled successfully.

Task 2: Create charge measurements and assign usage pricing to products

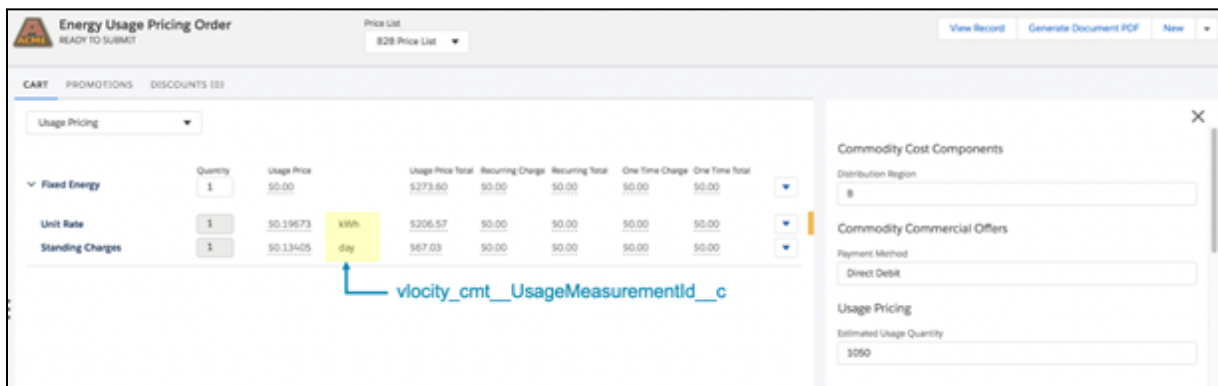
What is a charge measurement?

Charge measurements describe units of measure that are used to price products or services using usage pricing. For example, electricity can be measured in kilowatt hours (kWh). To price based on electricity usage, you would need to create a charge measurement for “kWh”.

So, in our scenario, Eliza needs to create two charge measurements, kWh and day. The kWh charge measurement will be used with the Unit Rate product to price electricity usage, and the day charge will be used with the Standing Charges product to price daily service charges.

1. Create charge measurements
 - a. Using the **Lightning App Launcher**, enter `Charge Measurements` in the search box and click to select.
 - b. Click **New** on the Charge Measurement page.
 - c. Enter `kWh` for the **Charge Measurement Name** and click **Save & New**.
 - d. Enter `day` for the **Charge Measurement Name** and click **Save**.

Below you can see how the charge measurements you’ve just created will display in the Cart.



The screenshot shows the 'Energy Usage Pricing Order' interface. The 'CART' tab is active, displaying a table of usage pricing items. The table has columns for Quantity, Usage Price, Usage Price Total, Recurring Charge, Recurring Total, One Time Charge, and One Time Total. There are three items listed: Fixed Energy, Unit Rate, and Standing Charges. The Unit Rate item is highlighted with a yellow background and has a blue arrow pointing to the 'kWh' unit. The Standing Charges item is also highlighted with a yellow background and has a blue arrow pointing to the 'day' unit. The right sidebar shows 'Commodity Cost Components' and 'Commodity Commercial Offers'.

	Quantity	Usage Price	Usage Price Total	Recurring Charge	Recurring Total	One Time Charge	One Time Total
Fixed Energy	1	\$0.00	\$273.60	\$0.00	\$0.00	\$0.00	\$0.00
Unit Rate	1	\$0.19673 kWh	\$206.57	\$0.00	\$0.00	\$0.00	\$0.00
Standing Charges	1	\$0.13405 day	\$67.03	\$0.00	\$0.00	\$0.00	\$0.00

Next, Eliza needs to create two pricing elements for the B2B price list that will be used to create price list entries for the Unit Rate and Standing Charges products.

Creating pricing elements should be familiar to you as this topic was covered in earlier modules. However, when creating pricing elements for usage pricing, you need to use a special Charge Type: Usage Std Price. This allows you to specify a Usage Unit of Measure (a.k.a. Charge Measurement). In this way, we can create one pricing element for the electricity rate that will be priced using kWh and another pricing element for standing charges that will be priced per day of service.

2. Create a pricing element for usage pricing charges
 - a. Navigate to the **Vlocity Product Console**.
 - b. Click on the search icon for **Price List**.
 - c. Search for the **B2B Price List** and select it.
 - d. Click on the **Pricing Elements** facet of the **B2B Price List**.
 - e. On the **CHARGES** tab, click **New**.
 - f. Enter the following to create a new **Pricing Element** and click **Save**:

Field	Value	Notes
Charge Type	Usage	
Search / Select a Pricing Variable:	Usage Std Price	Usage is a legacy variable that should not be used.
Name	\$0.0123	
Code	U-0.0123	
Display Text	\$0.0123/kWh	
Usage Unit of Measure	kWh	
Amount	0.0123	
Effective From	[today's date]	
Active	✓	

Pricing Variable

Charge Type * Usage

[Search](#)

Selected Pricing Variable: Usage Std Price

General Properties

Name * \$0.0123

Code * U-0.0123

Display Text * \$0.0123/kWh

Help Text

Currency Value

Calculation Type Simple

Usage Unit of Measure * kWh

Amount * 0.0123

Currency Code USD

Effectivity

Effective From * 6/1/2020 12:00 AM

Effective Until

Active ☒

[Save](#)

- g. Click **New** to create another pricing element.

h. Enter the following and click **Save**:

Field	Value	Notes
Charge Type	Usage	
Search / Select a Pricing Variable:	Usage Std Price	Usage is a legacy variable that should not be used.
Name	\$0.87123	
Code	U-0.87123	
Display Text	\$0.87123/day	
Usage Unit of Measure	day	
Amount	0.87123	
Effective From	[today's date]	
Active	✓	

Lastly, Eliza needs to create price list entries and assign usage pricing to two child products, Unit Rate and Standing Charges. Creating price list entries should be familiar to you as this topic was covered in earlier modules.

3. Create price list entries for usage pricing products
 - a. Navigate to the **Vlocity Product Console**.
 - b. Click on the search icon next to **Product** and search for **Unit Rate** to select it.
 - c. Click on the **Pricing** facet.
 - d. On the **CHARGES** tab, click **New**.

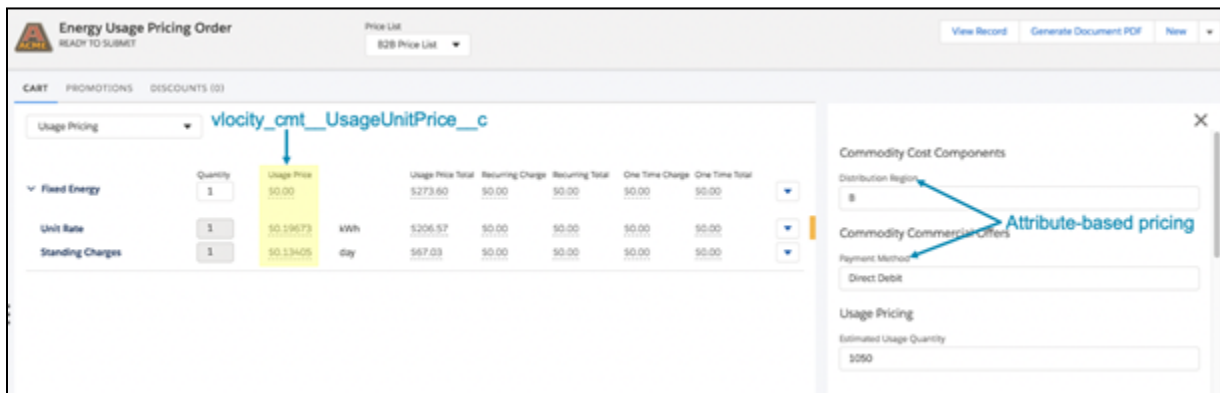
- e. Enter the following on the **PRICE LIST ENTRY** tab and click **Save**:

Field	Value
Price List	B2B Price List
Display Text	\$0.0123/kWh
Base Price	✓
Charge Type	Usage
Search / Select a Pricing Variable:	Usage Std Price
Pricing Element	\$0.0123
Effective From	[today's date]
Active	✓

- f. Search for the product **Standing Charges** from the Vlocity Product Console and select it.
- g. Click on the **Pricing** facet.
- h. On the **CHARGES** tab, click **New**.
- i. Enter the following on the **PRICE LIST ENTRY** tab and click **Save**:

Field	Value
Price List	B2B Price List
Display Text	\$0.87123/day
Base Price	✓
Charge Type	Usage
Search / Select a Pricing Variable:	Usage Std Price
Pricing Element	\$0.87123
Effective From	[Today's date]
Active	✓

The screen capture below displays the price of the energy products (using attribute-based pricing – we'll get to that soon!) and the field name on the Order Product (`OrderItem`) object in the Cart.



Now that Eliza has created charge measurements and pricing elements for usage pricing and assigned a usage price list entry to each product, next she needs to configure the variable element in the pricing calculation: Estimated Usage Quantity.

Estimated Usage Quantity will store the *estimated* amount of electricity the customer will use, so that it can be used in the usage pricing calculations for the quote or order. Remember that the Usage Price Total is calculated by multiplying the Usage Price by the Estimated Usage Quantity. But notice that the estimated quantity has a double burden; not only does it need to be visible and editable by users within the Cart, it also needs to be included in the pricing engine's calculations and rollups. To achieve this, Eliza will create an attribute binding.

What is an attribute binding?

An attribute binding links an attribute to a field on an object, so that as you change the attribute's value in the Cart, it is also changed in the bound field.



ALERT:

Attribute bindings are a one-way street. You cannot change the bound field's value and have the new value populate the attribute's value in the `JSONAttribute` field. The attribute always drives the bound field—and not the other way around.

Why is this useful?

Usually, it's because you want to use the value in some kind of calculation or process. When an attribute value is embedded within the JSONAttribute field, it's *very* accessible to the end users in the Cart, allowing them to change its value easily. But, as it turns out, it's not very accessible to any other processes! You can't monitor its value and trigger a workflow from it, for example. And importantly for our usage pricing scenario, the pricing engine can't use attribute values embedded in the JSONAttribute field for its calculations. They need to be stored in a field on the line item.

So, in order to enable usage pricing, we must create an attribute binding to link the attribute to a field, and then the field can be added to the pricing engine's calculation logic (via a pricing variable binding).

In our usage pricing scenario, we will bind the Estimated Usage Quantity to an existing field that's provided as part of the application, Usage Quantity (`vLOCITY_cmt__UsageQuantity__c`) on the Order Product (`OrderItem`) object.

The steps to create an attribute binding are:

- Create the attribute.
- Create (or select) the field and object that will be bound to the attribute.
- Create a VLOCITY Object to link the field's object to the shared catalog's metadata.
- Create the attribute binding to link the field to the attribute via the VLOCITY Object.

Then, if you plan to use the attribute binding in pricing, you must also:

- Create a pricing variable binding to the bound field.



NOTE:

In our scenario, we will use an existing field that is already bound to an out-of-the-box pricing variable, so essentially, it's been done for us. But it's important to note that if you were using a custom field, this is a key step that would need to be done.

What is a Vlocity Object?

You might have noticed in the steps above that you need to create a Vlocity Object as part of creating an attribute binding, and naturally, you're probably wondering what that is.

While Salesforce Industries has many objects that it has added to Salesforce's data model, there are some specific objects that the shared catalog considers its responsibility to manage. These objects are called Vlocity Objects, and they can be viewed in the Vlocity Objects and Object Types tab. You'll recognize most of them because they are precisely the objects you work with using the Product Console—objects like Product2, Attribute, Price List, Rules, and so on. Vlocity Objects adds a layer of metadata around the object itself, and the shared catalog uses the metadata to perform its functions.

So, when you create an attribute binding to a field on an object that is outside the shared catalog's "field of vision", so to speak, you must also define the object as a Vlocity Object so that the shared catalog knows about it. You only need to do this once for each object. In our scenario, we need to do it for the Order Product (`OrderItem`) object, and then you could create multiple attribute bindings to different fields on the Order Product object. If we wanted to enable usage pricing during quoting, we'd need to also create a Vlocity Object for the Quote Line Item (`QuoteLineItem`) object. And so on, for each object that has a field for which you want to create an attribute binding.

Task 3: Configure an attribute and an attribute binding for estimated usage quantity

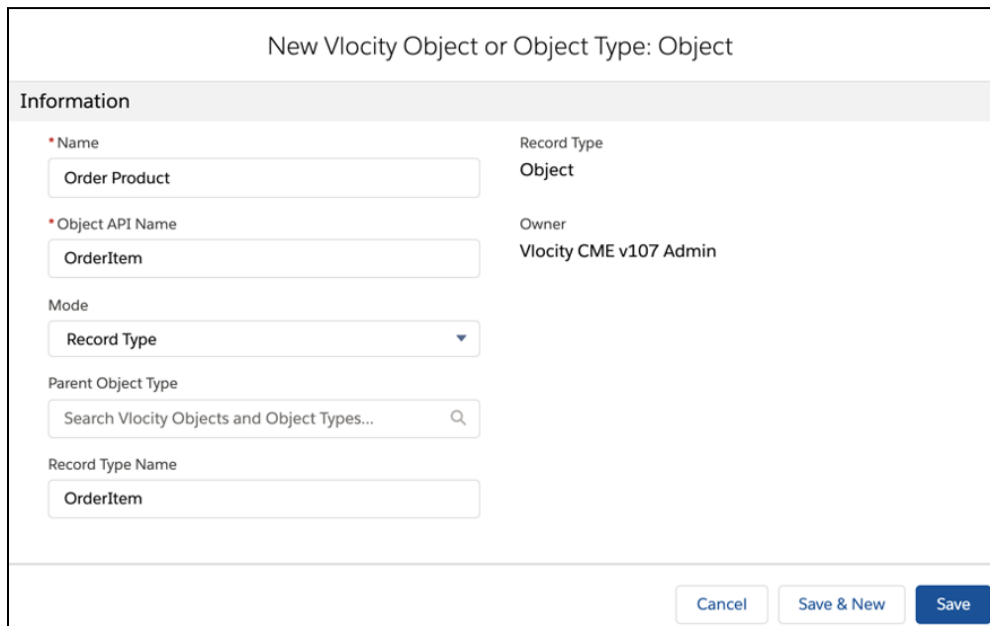
1. Create an attribute.
 - a. Navigate to the **Vlocity Product Console**.
 - b. Click the + icon next to **Attribute**.
 - c. Enter the following and click **Save**:

Field	Value
Name	Estimated Usage Quantity
Code	ATT_EST_USAGE_QTY
Active	✓
Attribute Category	Usage Pricing
Value Type	Number

2. Create a Vlocity Object for Order Product (`OrderItem`).
 - a. Using the **Lightning App Launcher**, enter **Vlocity Objects and Object Types** in the search box and click to select.
 - b. Click **New**.
 - c. Select **Object** as the record type on the New Vlocity Object or Object Type window and click **Next**.

- d. Enter the following and click **Save**:

Field	Entry
Name	Order Product
Object API Name	OrderItem
Mode	Record Type
Parent Object Type	Leave blank
Record Type Name	OrderItem



- e. Navigate to the **Vlocity Product Console**.
- f. Click on the **search** icon next to **Object** in the Metadata section and search for all objects.
- g. Notice the **Order Product** object record that you just created.

Dashboard Search Object x			
Search Object: Q			
Search Results:			
Showing 1 - 33 of 33 results			
NAME	OBJECT API NAME	RECORD TYPE NAME	IS VIRTUAL
AdjustmentData	ObjectClass__c		☐
Attribute Object	Attribute__c		☐
Context Action Object	ContextAction__c		☐
Context Dimension Object	ContextDimension__c		☐
Context Scope Object	ContextScope__c		☐
Order Product	OrderItem	OrderItem	☐
Picklist Object	Picklist__c		☐

Next, Eliza needs to bind the Estimated Usage Quantity attribute to the Usage Quantity field on the Order Product (`OrderItem`) object.

3. Create an attribute binding.

- Navigate to the **Vlocity Attribute Categories** tab. You may need to click the **More** dropdown menu to expand for additional tabs.
- Select List View **All**.
- Select the **Usage Pricing** category.
- Click on the **Estimated Usage Quantity** attribute.
- Click on the **Related** tab and click **New**.
- Enter the following to create a New Vlocity Attribute Binding:

Field	Value
Attribute Binding Name	Estimated Usage Quantity Binding
Object Class	Order Product
Field Name	vlocity_cmt__UsageQuantity__c

New Vlocity Attribute Binding

Information

• Attribute Binding Name

Estimated Usage Quantity Binding

• Vlocity Attribute

Estimated Usage Quantity

• Object Class

Order Product

Field Name

vlocity_cmt__UsageQuantity__c

Cancel

Save & New

Save

g. Click **Save**.

Lastly, Eliza will assign the bound attribute, Estimated Usage Quantity, to the Fixed Energy child products, Unit Rate and Standing Charges.

4. Assign bound attribute to a product
 - a. Navigate to the **Vlocity Product Console**.
 - b. Click the **search** icon next to **Product** to search for the **Unit Rate** product and select it.
 - c. Click on the **Attributes and Fields** facet.
 - d. Click on the **Assign Attributes/Fields** button. Check the box next to **Estimated Usage Quantity** and click **Assign**.


Unit Rate

General Properties

Attribute Properties

Attributes and Fields

Attachments

Product Structure

Pricing

Context Rules

Layout Management

Debug

Attributes and Fields

Unassign

Assign Attributes/Fields

☐ NAME	TYPE
☐ Distribution Region	Attribute
☐ Payment Method	Attribute
☐ Active	Field
☐ Approved By	Field
☐ Approved On	Field
☐ Attribute Default Values	Field
☐ Attribute Metadata	Field
☐ Category Data	Field
☐ ChangeDetectorImplementation	Field
☐ Class	Field
☐ Configurable	Field

Assign Attributes and Fields

Assign

☐ NAME	TYPE
☐ Connection Type	Attribute
☐ Customer Category	Attribute
☐ Download Speed	Attribute
☒ Estimated Usage Quantity	Attribute
☐ Install Account Name	Attribute
☐ Meter Type	Attribute
☐ Number of Accounts	Attribute
☐ Pricing Scheme	Attribute
☐ Spotify Subscription Type	Attribute
☐ Upload Speed	Attribute

- e. Repeat these steps to assign the **Estimated Usage Quantity** attribute to the product **Standing Charges**.

5. Review usage pricing variables

Remember earlier when we enabled the usage pricing feature and we mentioned that several activities occur behind the scenes when features are enabled? Well, when usage pricing was enabled, pricing variables for usage and usage pricing variable bindings were created. You'll notice that these usage pricing variable bindings are similar to the attribute binding you just created.

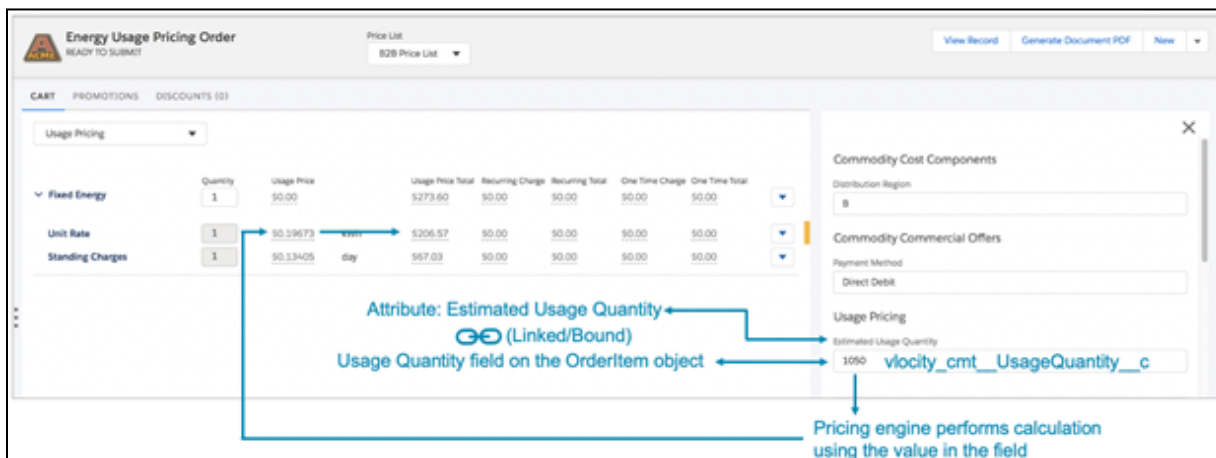
- a. Navigate to the **Vlocity Product Console**.
- b. Click on the search icon for **Pricing Variable**.
- c. Enter `usage` in the search box and click the search icon.
- d. Notice the usage pricing variables created when usage pricing was enabled.

PRICING VARIABLE NAME	CODE	CHARGE TYPE	SUB-TYPE	FREQUENCY	TYPE	ADJUSTMENT METHOD
Effective Usage Quantity	EFFECTIVE_USAGE_QUANTITY	Usage				
Effective Usage Std Price Total	EFF_USAGE_STD_PRC_TOTAL	Usage	Standard		Price	
Group Rollup Usage Std Price Total	GROUP_ROLLUP_USAGE_STD_PRC_TOTAL	Usage	Standard		Price	
Master Rollup Usage Std Price Total	MASTER_ROLLUP_USAGE_STD_PRC_TOTAL	Usage	Standard		Price	
Parent Rollup Usage Std Price Total	PARENT_ROLLUP_USAGE_STD_PRC_TOTAL	Usage	Standard		Price	
Rollup Usage Std Price Total	ROLLUP_USAGE_STD_PRC_TOTAL	Usage	Standard		Price	
Usage	USAGE	Usage			Price	
Usage Line Quantity	USAGE_LINE_QUANTITY	Usage				
Usage Std Price	USAGE_STD_PRC	Usage	Standard		Price	
Usage Std Price Adjustment Abs	USAGE_STD_PRC_ADJ_ABS	Adjustment	Standard		Price	Absolute
Usage Std Price Adjustment Pct	USAGE_STD_PRC_ADJ_PCT	Adjustment	Standard		Price	Percent
Usage Std Price Total	USAGE_STD_PRC_TOTAL	Usage	Standard		Price	

- Click on the edit icon next to **Usage Std Price** and then click on the **Pricing Variable Bindings** facet.
- Notice the field binding associated with each sObject type.

Usage Std Price			
General Properties	Context Mappings		
Pricing Variable Bindings	New Pricing Variable Binding		
	PRICING VARIABLE BINDING NAME	DESTINATION OBJECT TYPE	DESTINATION FIELD API NAME
	Usage Std Price - OpportunityLineItem	OpportunityLineItem	velocity_cmt__UsageUnitPrice__c
	Usage Std Price - QuoteLineItem	QuoteLineItem	velocity_cmt__UsageUnitPrice__c
	Usage Std Price - OrderItem	OrderItem	velocity_cmt__UsageUnitPrice__c
	Usage Std Price - Asset	Asset	velocity_cmt__UsageUnitPrice__c

The screen capture below displays the attribute (Estimated Usage Quantity) bound to the velocity_cmt__UsageQuantity__c field on the OrderItem object where the pricing engine detects it and can perform calculations, based on the pricing variables defined, in the Cart.



The screenshot shows the 'Energy Usage Pricing Order' interface. The cart contains a 'Fixed Energy' unit rate of \$0.19673 and standing charges of \$0.13405/day. A diagram illustrates the 'Attribute: Estimated Usage Quantity' (Linked/Bound) to the 'Usage Quantity field on the OrderItem object'. This field is linked to the 'velocity_cmt__UsageQuantity__c' field, which has a value of 1050. The diagram notes that the 'Pricing engine performs calculation using the value in the field'.

Task 4: Test usage pricing in the Cart

1. Navigate to the **Orders** tab and click **New**.
2. On the New Order, enter the following and click **Save**.

Field	Value
Order Name	Energy Usage Pricing Order
Account Name	Acme
Order Start Date	[Today's date]
Price List	B2B Price List

3. Click on the Power Launcher's search field and select **Configure Order**.
4. Add the **Fixed Energy** product to the cart.
5. Click the down arrow to **Configure** the **Unit Rate** order line item.
6. Enter 1000 for the **Estimated Usage Quantity** attribute. Close the Configure window.
7. Click the down arrow to **Configure** the **Standing Charges** order line item.
8. Enter 500 for the **Estimated Usage Quantity** attribute. Close the Configure window.
9. Notice how the pricing engine calculates the Usage Price Total from the bound attribute and pricing variable bindings created.

CART PROMOTIONS DISCOUNTS (0)								
Usage Pricing ▼								
	Total Quantity	Usage Price		Usage Price Total	Recurring Charge	Recurring Total	One Time Charge	One Time Total
▼ Fixed Energy	1	\$0.00		\$447.92	\$0.00	\$0.00	\$0.00	\$0.00
Unit Rate	1	\$0.0123	kWh	\$12.30	\$0.00	\$0.00	\$0.00	\$0.00
Standing Cha...	1	\$0.87123	day	\$435.62	\$0.00	\$0.00	\$0.00	\$0.00



NOTE:

If usage pricing doesn't display or calculate correctly in the cart verify that you have configured usage pricing correctly for each step. Then, run the maintenance jobs -- Clear Managed Platform Cache and Refresh Platform Cache.

In many Energy & Utilities deployments, usage pricing alone is not enough. The cost of energy commodities fluctuates and will vary based on customers' geographic location. Therefore, in addition to usage pricing, Eliza needs to create a decision matrix to change the usage price of the Unit Rate and Standing Charges products using attribute-based pricing. And, in order to use attribute-based pricing with usage pricing, you need to extend attribute-based pricing to work with new usage pricing variables.

Task 5: Configure attribute-based pricing for usage pricing

1. Create an attribute-based pricing matrix.
 - a. Using the **Lightning App Launcher** in the upper left, select **Files**.
 - b. Locate the **UsagePricingMatrix v1.csv** file.
 - c. Click the drop-down menu on the **UsagePricingMatrix v1** line and select **Download**.
 - d. Navigate to the **Lookup Tables** tab. You may need to reopen it using the Lightning App Launcher.
 - e. Click **New**, select **Decision Matrix**, and click **Next**.
 - f. In the New Decision Matrix dialog box, type `UsagePricingMatrix` for the **Name**, select **Standard** as **Type**, and click **Save**.
 - g. Click the **Related** tab and select **UsagePricingMatrix V1**.
 - h. Click **Upload CSV Files**.
 - i. In the dialog box that appears, select **Upload Files**.
 - j. Select the downloaded .csv file.
 - k. Edit the Header Information with the following and click **Upload**:

Column Header	Data Type	Header Type
SourceProductName	Text	Input
SourceProductCode	Text	Input
CharacteristicName	Text	Input
CharacteristicValue	Text	Input
MRC	Currency	Output
NRC	Currency	Output
UP	Currency	Output
CM	Text	Output

- l. Click **Done** and wait for the file upload to process. Click **Refresh**, if necessary.
- m. Notice the following:
 - i. Products in the matrix, along with their attributes and values in the Characteristic Name and Characteristic Value columns.
 - ii. UP column = usage pricing
 - iii. CM column = charge measurement

Matrix							
10 records • Sorted by SourceProductName							
Input Data				Output Data			
SourceProductName (Text) ↑	SourceProductCode (Text)	CharacteristicName (Text)	CharacteristicValue (Text)	MRC (Currency)	NRC (Currency)	UP (Currency)	CM (Text)
Standing Charges	CC_FIXED_ENERGY_STANDING_CHA...	Distribution RegionPayment Method	A:Direct Debit	\$0	\$0	\$0.13405	day
Standing Charges	CC_FIXED_ENERGY_STANDING_CHA...	Distribution RegionPayment Method	B:Direct Debit	\$0	\$0	\$0.17805	day
Standing Charges	CC_FIXED_ENERGY_STANDING_CHA...	Distribution RegionPayment Method	C:Direct Debit	\$0	\$0	\$0.32405	day
Standing Charges	CC_FIXED_ENERGY_STANDING_CHA...	Distribution RegionPayment Method	D:Direct Debit	\$0	\$0	\$0.28405	day
Standing Charges	CC_FIXED_ENERGY_STANDING_CHA...	Distribution RegionPayment Method	E:Direct Debit	\$0	\$0	\$0.33405	day
Unit Rate	CC_FIXED_ENERGY_UNIT_RATE	Distribution RegionPayment Method	A:Direct Debit	\$0	\$0	\$0.13673	kWh
Unit Rate	CC_FIXED_ENERGY_UNIT_RATE	Distribution RegionPayment Method	B:Direct Debit	\$0	\$0	\$0.19673	kWh
Unit Rate	CC_FIXED_ENERGY_UNIT_RATE	Distribution RegionPayment Method	C:Direct Debit	\$0	\$0	\$0.22673	kWh
Unit Rate	CC_FIXED_ENERGY_UNIT_RATE	Distribution RegionPayment Method	D:Direct Debit	\$0	\$0	\$0.37673	kWh
Unit Rate	CC_FIXED_ENERGY_UNIT_RATE	Distribution RegionPayment Method	E:Direct Debit	\$0	\$0	\$0.31673	kWh

- n. In the UsagePricingMatrix V1 header, click **Edit**.
- o. Select **yesterday's date** from the calendar icon for the **Start Date Time / Date** field.
- p. Check the **Enabled** checkbox and select **Save**.

Once the decision matrix is defined, Eliza then needs to create a usage pricing procedure expression set so that the energy products assigned usage pricing are calculated based on the UsagePricingMatrix.

2. Create an attribute pricing procedure expression set.
 - a. Navigate to the **Expression Sets** tab. You may need to reopen it using the Lightning App Launcher.
 - b. Click **New**.
 - c. In the New Expression Set dialog box, enter `UsagePricingProcedure` for the **Name** and click **Save**.
 - d. Click the **Related** tab and next to UsagePricingProcedure V1, click the dropdown arrow  and select **Edit**.
 - e. In the **Rank** field, enter 1.
 - f. Click the Start Date Time / Date field and select **yesterday's date** from the calendar, and select **Save**.

- g. Select **UsagePricingProcedure V1**.
- h. Click **Open in Expression Set Builder**.
- i. Click the **plus +** sign and select **Lookup Table** in the Add Element dialog box.
- j. In the Lookup Table Details field, search for and select **UsagePricingMatrix**.
- k. With the Lookup Table element selected, click the **Element Details** tab  in the left panel, and check the **Include in Output** checkbox.
- l. Click the **Resource Manager** tab  and select **Add Resource**.
- m. In the **Add New Resource** dialog, and enter the following:

Field	Value
Resource Type	Variable
Resource Name	REC_MNTH_STD_PRC
Data Type	Currency
Decimal	2

- n. Click **Done And New** and enter the following:

Field	Value
Resource Type	Variable
Resource Name	OT_STD_PRC
Data Type	Currency
Decimal	2

- o. Click **Done And New** and enter the following:

Field	Value
Resource Type	Variable
Resource Name	USAGE_STD_PRC
Data Type	Currency
Decimal	5

- p. Click **Done And New** and enter the following:

Field	Value
Resource Type	Variable
Resource Name	CM
Data Type	Text

- q. Click **Done**.

- r. Click the **plus +** sign and add a **Calculation** step to the expression set.

- In the Formula field, search for and select **UsagePricingMatrix__MRC**.
- In the Output Variable, search for and select **REC_MNTH_STD_PRC**.

- Click the **Element Details** tab  in the left panel, and check the **Include in Output** checkbox.

- s. Click the **plus +** sign and add a **Calculation** step to the expression set.

- In the Formula field, search for and select **UsagePricingMatrix__NRC**.
- In the Output Variable, search for and select **OT_STD_PRC**.

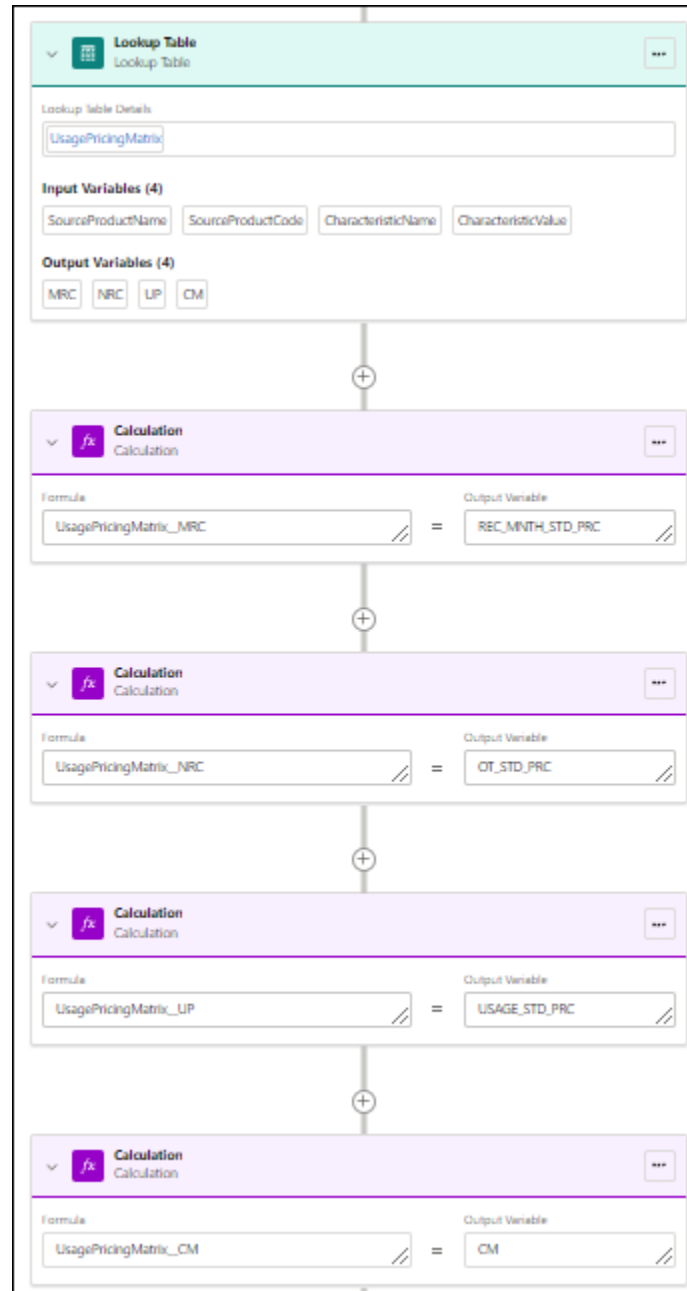
- Click the **Element Details** tab  in the left panel, and check the **Include in Output** checkbox.

- t. Click the **plus +** sign and add a **Calculation** step to the expression set.

- In the Formula field, search for and select **UsagePricingMatrix__UP**.
- In the Output Variable, search for and select **USAGE_STD_PRC**.

- Click the **Element Details** tab  in the left panel, and check the **Include in Output** checkbox.

- u. Click the **plus +** sign and add a **Calculation** step to the expression set.
 - i. In the Formula field, search for and select **UsagePricingMatrix__CM**.
 - ii. In the Output Variable, search for and select **CM**.
 - iii. Click the **Element Details** tab  in the left panel, and check the **Include in Output** checkbox.



- v. Click **Save** and then click **Activate**.
- w. Optional: Simulate the expression set and troubleshoot any errors that occur.
- x. Close the Expression Set Builder browser tab.

3. Create a custom pricing plan step.

- a. Navigate to the **Vlocity Product Console**.
- b. Next to **Pricing Plan** in the **Pricing** area, click the **search icon** .
- c. In the **Search Pricing Plan...** dialog box, press the return or enter key to display the Default Pricing Plan.
- d. Click on the **Default Pricing Plan**.
- e. Click on the **Pricing Plan Steps** facet.
- f. Click **New Item**.
- g. Enter the following data in the **General Properties** pane:

Field	Value
Name	Calculate Usage ABP
Implementation Name	CustomPricingPlanStepImpl
Method Name	GetMatrixPrice
Sequence	9.5
Active	✓



ALERT:

All attribute pricing steps must occur before the Initialize Pricing Context step; therefore, the sequence number must be before 10.

- h. Click the **Edit** icon  to open the **Parameters** dialog box.
- i. Click the + symbol to add the following parameters:

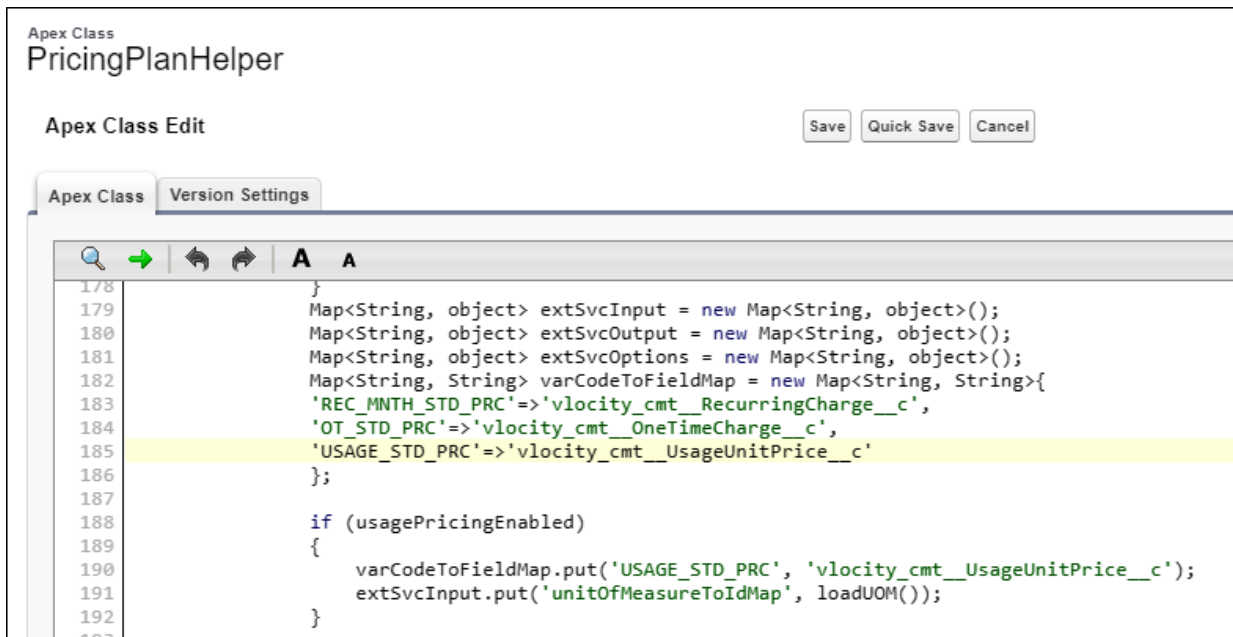
Field	Value
ProcedureName	UsagePricingProcedure
MatrixName	UsagePricingMatrix
DecisionMatrix	true

- j. Click **Done**.

- k. Click **Save** in the General Properties pane.

Finally, Eliza needs to extend the PricingPlanHelper Apex Class to include the USAGE_STD_PRC variable mapping to ensure the USAGE_STD_PRC value coming from the expression set is saved into the vlocity_cmt__UsageUnitPrice__c field.

4. Configure PricingPlanHelper Apex Class for usage pricing.
 - a. Click on the **Setup** icon  and select **Setup**.
 - b. In the **Quick Find** search box, enter and select **Apex Classes**.
 - c. On the Apex Classes page, click **Edit** next to **PricingPlanHelper**.
 - d. Place the cursor at the end of line 184. Enter a comma (,) and then press the enter key to create a new line 185.
 - e. Enter 'USAGE_STD_PRC'=>'vlocity_cmt__UsageUnitPrice__c' on the new line 185.



```
Apex Class
PricingPlanHelper

Apex Class Edit
Save Quick Save Cancel

Apex Class Version Settings

178 }
179 Map<String, object> extSvcInput = new Map<String, object>();
180 Map<String, object> extSvcOutput = new Map<String, object>();
181 Map<String, object> extSvcOptions = new Map<String, object>();
182 Map<String, String> varCodeToFieldMap = new Map<String, String>{
183     'REC_MNTH_STD_PRC'=>'vlocity_cmt__RecurringCharge__c',
184     'OT_STD_PRC'=>'vlocity_cmt__OneTimeCharge__c',
185     'USAGE_STD_PRC'=>'vlocity_cmt__UsageUnitPrice__c'
186 };
187
188 if (usagePricingEnabled)
189 {
190     varCodeToFieldMap.put('USAGE_STD_PRC', 'vlocity_cmt__UsageUnitPrice__c');
191     extSvcInput.put('unitOfMeasureToIdMap', loadUOM());
192 }
```

- f. Click **Save**.

Task 6: Edit an order to use attribute-based pricing

Eliza will now see all her hard work come together to price an energy product in the Cart based on values defined in the ABP matrix and estimated usage consumption attribute field.

1. Navigate to the **Orders** tab and select the **Energy Usage Pricing Order**.
 2. Click on the Power Launcher's search field and select **Configure Order**.
 3. In the cart line item for **Unit Rate**, click the down arrow and select **Configure**.
 4. Enter **A** for the **Distribution Region** attribute. Close the Configure window.
 5. What price displays in the Usage Price column and Usage Price Total column?
-
6. Change the **Distribution Region** to **B** and the Estimated Usage Quantity to **1050**. Close the Configure window.
 7. In the **Usage Price** column for **Unit Rate**, click on the price and select the **Price Details**  icon. Notice that the price returned is from the UsagePricingProcedure based on the data entered in the matrix. **Close** the Price Details window.



NOTE:

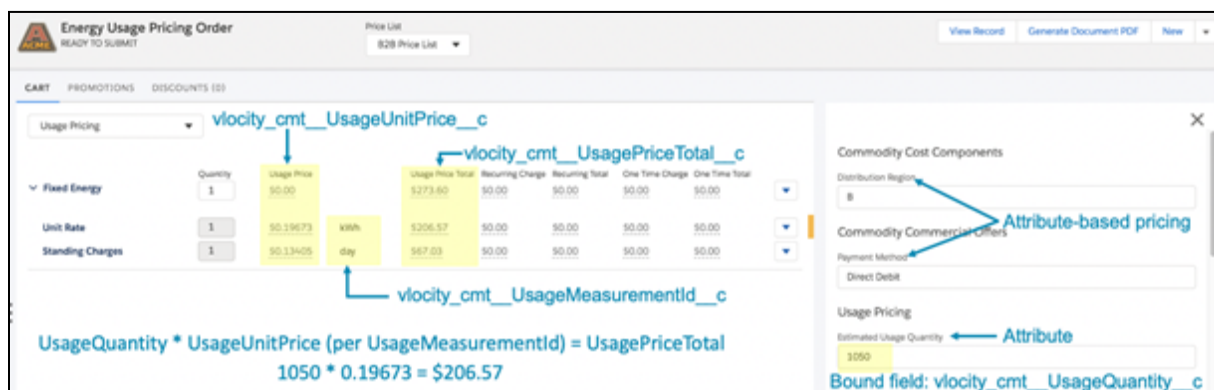
The Usage Price column displays as five decimal places which was defined when you created the UsagePricingProcedure. However, it defaults to two decimal places in the Details window. Custom configuration of the Details window would be required to alter the precision of decimal places.

8. In the **Usage Price Total** column for **Unit Rate**, click on the price and select the **Price Details** icon. Notice the calculation to arrive at the total usage price. **Close** the Price Details window.
9. In the cart line item for **Standing Charges**, click the down arrow and select **Configure**.
10. Enter **A** for the **Distribution Region** attribute. **Close** the Configure window.

CART PROMOTIONS DISCOUNTS (0)								
Usage Pricing ▼								
	Total Quantity	Usage Price		Usage Price Total	Recurring Charge	Recurring Total	One Time Charge	One Time Total
▼ Fixed Energy	1	\$0.00		\$273.60	\$0.00	\$0.00	\$0.00	\$0.00
Unit Rate	1	\$0.19673	kWh	\$206.57	\$0.00	\$0.00	\$0.00	\$0.00
Standing Cha...	1	\$0.13405	day	\$67.03	\$0.00	\$0.00	\$0.00	\$0.00

11. Click **Submit Order**.

The screen capture below displays all the relevant fields in the Cart after enabling Usage Pricing and binding an attribute to a field.



The screenshot shows the 'Energy Usage Pricing Order' interface. The cart table is updated with usage pricing details. Annotations highlight the attribute-based pricing configuration:

- Usage Unit Price:** `velocity_cmt_UsageUnitPrice__c` is mapped to the 'Usage Price' field.
- Usage Price Total:** `velocity_cmt_UsagePriceTotal__c` is mapped to the 'Usage Price Total' field.
- Usage Measurement ID:** `velocity_cmt_UsageMeasurementId__c` is mapped to the 'Usage Measurement' field.
- Usage Pricing Section:** The 'Usage Pricing' section shows 'Estimated Usage Quantity' set to 1050, with an 'Attribute' bound to the field `velocity_cmt_UsageQuantity__c`.

A formula is displayed at the bottom: $\text{UsageQuantity} * \text{UsageUnitPrice (per UsageMeasurementId)} = \text{UsagePriceTotal}$, with the calculation $1050 * 0.19673 = \$206.57$.

Exercise 9-6: Extending Attribute-Based Pricing Challenge

Scenario

Marketing has introduced the Apple iPhone Xs as a new product offering where customers can either pay in full for the phone or pay monthly. Marketing is also offering a volume discount when more than 50 Apple iPhone Xs' are paid in full. Eliza is tasked with creating attribute-based pricing matrices to accommodate the request by marketing.

Goal

- Review the requirements below and challenge yourself to create attribute-based pricing for the iPhone Xs product
- Challenge exercises solidify what you learned in the exercises, build your skills, and allow you to make mistakes -- which is how you will master this application

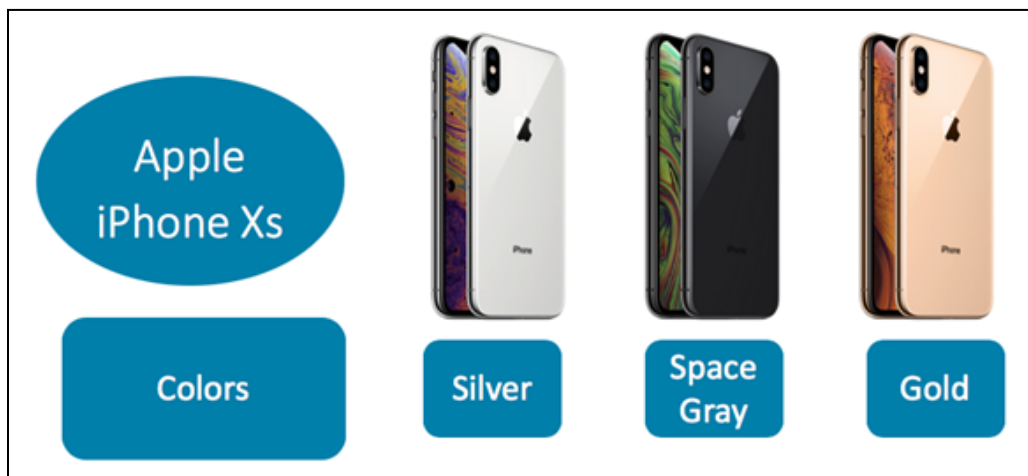
Tasks

1. Review product attributes
2. Set up attribute-based pricing matrix
3. Test attribute-based pricing in the Cart

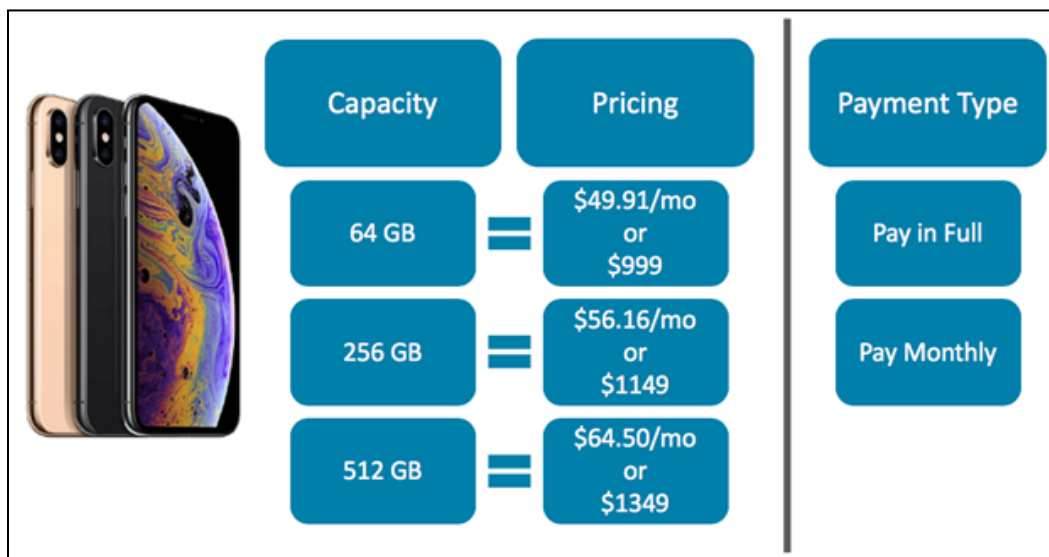
Time: 45 mins

Task 1: Review product attributes

1. The Apple iPhone Xs (Product Code C-PHO-010) is available in three finishes – Silver, Space Gray and Gold.

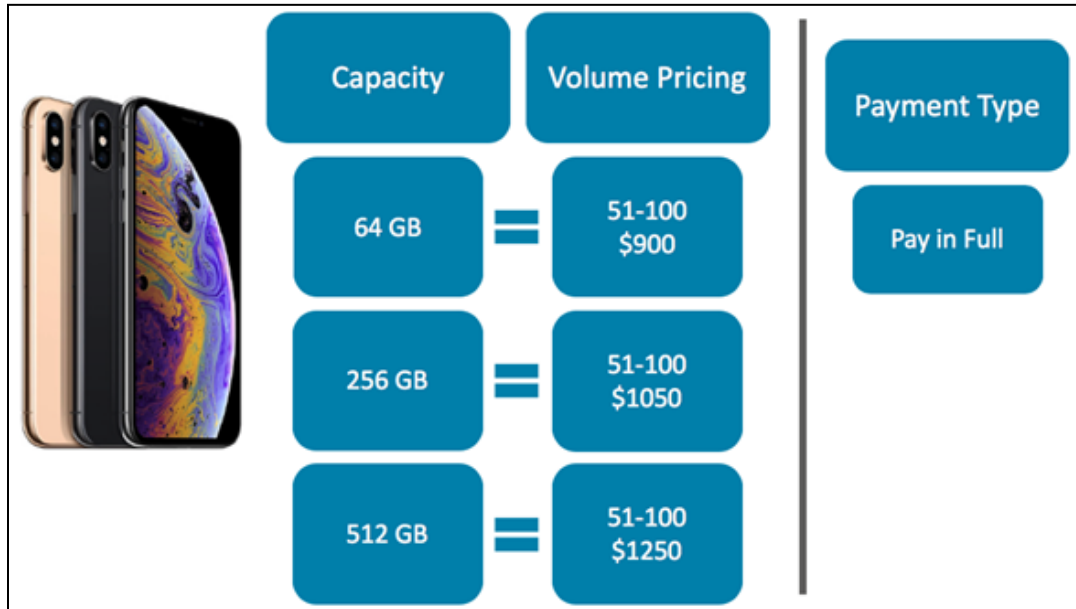


2. The iPhone Xs is available in three capacities – 64GB, 256GB, and 512GB.
3. Customers have two payment choices: Pay in Full or Pay Monthly. Use the graphic below to understand the one-time and recurring pricing models.



One-time and Recurring Pricing

4. Volume discounts are available to customers buying over 50 units. Use the graphic below for volume discount pricing.



Volume Discount Pricing

Task 2: Set up attribute-based pricing matrix

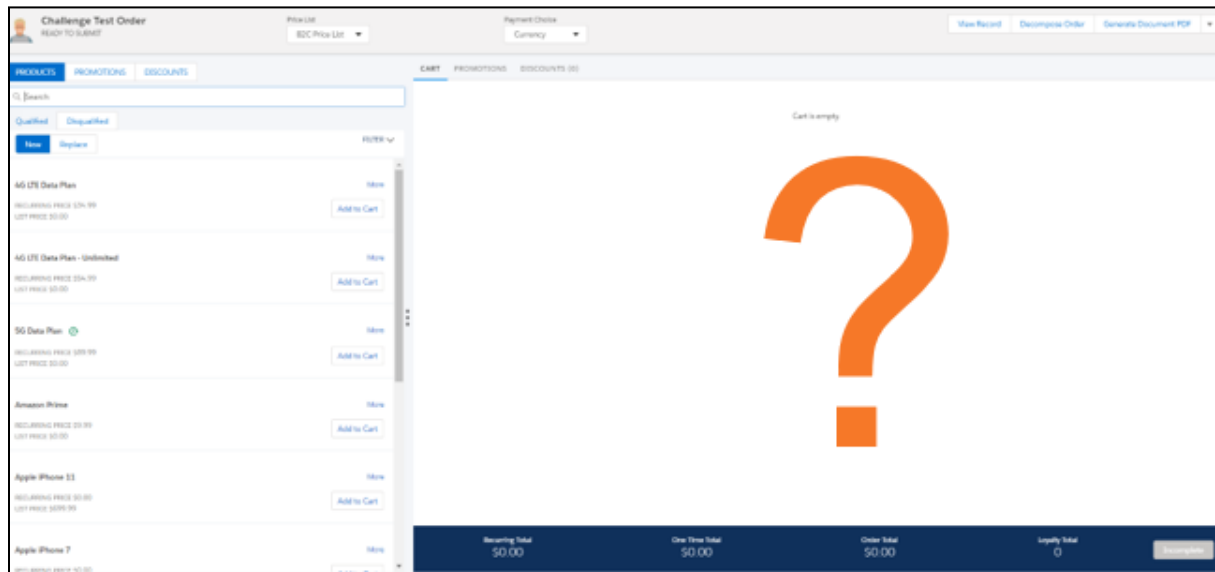
 Lookup Tables	
Decision Matrices	
Recently Viewed ▼	
6 items • Sorted by Name • Updated a few seconds ago	
Name ↑	
1	AttributePricingMatrix
2	BillingZipCodeAttributePricingMatrix
3	RangeAttributePricingMatrix



TIPS:

- Did you enter the data in the matrix using proper case and/or appropriate spacing?
- Are multiple attribute names separated with proper punctuation?
- Have you double-checked your spelling?
- Does the color of the Apple iPhone Xs affect pricing? Does it need to be included in the matrix?

Task 3: Test attribute-based pricing in the Cart





Yay! All done!